
ROBOT OPERATING SYSTEM 2

ROS2

프로그래밍 첫걸음

토픽에서 런치까지,
Python과 C++로 익히는 다섯 가지 통신

이수미 · 최규남 · 이하나 · 손민호
지음

발 행 | 2026년

저 자 | 이수미 · 최규남 · 이하나 · 손민호

펴낸이 | 한건희

펴낸곳 | 주식회사 부크크

출판사등록 | 2014.07.15.(제2014-16호)

주 소 | 서울특별시 금천구 가산디지털1로 119 SK트윈타워 A동 305호

전 화 | 1670-8316

이메일 | info@bookk.co.kr

ISBN | 979-11-410-0000-0

www.bookk.co.kr

© ROS2 프로그래밍 첫걸음

본 책은 저작자의 지적 재산으로서 무단 전재와 복제를 금합니다.

ROBOT OPERATING SYSTEM 2

ROS2

프로그래밍 첫걸음

토픽에서 런치까지,
Python과 C++로 익히는 다섯 가지 통신

이수미 · 최규남 · 이하나 · 손민호
지음

머리말

로봇을 로봇으로 만드는 것은 겉의 모터와 센서가 아니라 그 안의 소프트웨어입니다. 로봇 소프트웨어는 인식·판단·제어를 여러 모듈로 나눠 만들고, 그 모듈들이 대화하는 공통의 약속이 ROS(Robot Operating System)입니다. ROS2는 실시간 제어와 보안, 여러 대의 로봇을 함께 다루기 위해 산업 표준 통신 미들웨어 DDS 위에서 다시 설계되었습니다.

이 책은 ROS2를 처음 접하는 사람이 리눅스와 분산 통신, 빌드 시스템, 낮선 명령어를 한꺼번에 마주하는 부담을 덜도록 썼습니다. 핵심 예제를 Python과 C++로 함께 제공하고, 모든 예제를 Ubuntu 24.04와 ROS2 Jazzy에서 직접 검증했으며, 명령을 나열하기보다 왜 필요한지를 먼저 짚습니다.

1권인 이 책은 노드·토픽·서비스·액션·파라미터·런치의 기본을 다루고, 2권 『ROS2 시뮬레이션 첫걸음』은 URDF 모델링과 Gazebo 시뮬레이션, 자율주행으로 이어집니다. 로봇 소프트웨어를 처음 시작하는 학생과 개발자, ROS1에서 ROS2로 옮기려는 분에게 맞춘 책입니다.

함께 원고를 쓰고 다듬어 준 공저자와 동료, 학생들, 그리고 ROS를 오픈소스로 만들어 온 개발자들에게 감사드립니다. 이 책이 로봇 소프트웨어를 시작하는 데 도움이 되기를 바랍니다.

2026년, 이수미

머리말

로봇 개발은 혼자만의 연구가 아니라, 서로의 경험과 코드를 나누며 함께 문제를 해결해 가는 협업의 과정입니다. 오로카 판교 모임을 이끌며 수많은 학생, 연구원, 현업 엔지니어들과 함께 ROS를 공부하고 현장에 적용해 왔습니다. 그동안 수많은 질문을 마주하며 가장 절실히 느낀 것은, 복잡한 이론의 나열보다 핵심 원리를 꿰뚫고 실무에 바로 쓰는 단단한 기본기가 무엇보다 중요하다는 점이었습니다.

이 책에는 커뮤니티와 스터디 현장에서 마주했던 입문자들의 생생한 고민을 고스란히 녹여냈습니다. 노드가 왜 분산되어야 하는지, DDS 기반의 새로운 통신 체계가 왜 필요한지를 실습을 통해 자연스럽게 체득할 수 있도록 구성했습니다. Python과 C++ 예제를 함께 실어 두 언어의 실전 개발 감각을 고루 다질 수 있게 배려했고, 2026년 기준 최신 LTS 배포판인 ROS2 Jazzy 환경에서 모든 코드를 꼼꼼하게 검증했습니다.

이 책이 오로카 판교 스터디 모임을 비롯해 함께 공부하는 모든 분들에게 든든한 나침반이 되기를 바랍니다. 책장을 넘기는 데서 멈추지 않고, 동료들과 머리를 맞대어 코드를 실행하고 터미널의 에러 로그를 함께 풀어가는 살아 있는 스터디 교재로 널리 쓰이기를 기대합니다. 책의 예제를 발판 삼아 각자의 독창적인 로봇 프로젝트로 확장해 나가는 기쁨도 함께 경험해 보시길 권합니다.

언제나 배움에 대한 뜨거운 열정으로 지식을 나눠 주신 오로카 판교 커뮤니티 멤버 여러분과, 한마음으로 원고를 완성해 준 공저자분들께 깊은 감사를 전합니다. 로봇 소프트웨어 개발이라는 멋진 여정을 시작하는 모든 분들에게 이 책이 가장 든든한 동반자가 되어 주기를 진심으로 응원합니다.

2026년, 최규남

머리말

로봇과 인공지능을 가르치는 교육 현장에서 학생들을 만나며 늘 느끼는 것이 있습니다. 새로운 것을 배울 때 우리를 어렵게 만드는 것은 기술 자체의 난이도보다 낯섦이 주는 두려움일 때가 많다는 것입니다.

누구나 ROS2를 처음 접하면 노드, 토픽, 서비스, 액션과 같은 낯선 개념부터 리눅스 환경과 터미널 명령어까지 한꺼번에 익혀야 하는 어려움을 경험합니다. 그래서 이 책을 집필하며 가장 중요하게 생각한 것은 처음 배우는 사람이 낯섦을 넘어설 수 있도록 돕는 것이었습니다. 단순히 코드를 실행하는 데 그치지 않고, 왜 이런 구조가 필요한지, 각각의 개념이 어떻게 연결되는지를 이해하며 스스로 다음 단계로 나아갈 수 있기를 바랐습니다.

처음 ROS2를 마주한 독자 여러분도 너무 어렵다고 생각하지 않았으면 합니다. 지금은 그저 조금 낯선 것뿐입니다. 이 책과 함께 하나씩 직접 실행하고 경험하다 보면 어느새 ROS2가 익숙한 도구가 되어 있을 것입니다.

함께 고민하며 책을 완성해 주신 공저자분들께 감사드리며, 이 책이 여러분의 낯선 첫걸음을 든든하게 함께하는 길잡이가 되기를 바랍니다.

2026년, 이하나

머리말

30년 가까이 센서를 설계하고 액추에이터를 구동하고 계측 시스템을 자동화하는 일을 해 왔습니다. 직접 로봇을 만든 적은 없었지만, 로봇을 이루는 요소들은 대부분 손에 익은 것이었습니다. 4족 보행 로봇을 직접 조립하고 동작시키면서 확인한 것은, 부품도 쉽지는 않지만 그것들을 하나의 시스템으로 묶는 일이 더 어렵다는 점이었습니다. 그리고 그 묶음의 언어가 ROS2였습니다.

이 책을 쓰는 동안 저는 로봇팔과 자율주행 이동 로봇을 실제로 제어하는 집중 과정을 함께 마쳤고, 오로카에서 4족 보행 로봇 제작 스터디도 병행했습니다. 그 병행이 이 책에는 다행이었다고 생각합니다. 화면 안에서만 도는 코드와 실제 관절이 꺾이고 바퀴가 굴러가는 코드는 다릅니다. 문서대로 했는데 동작하지 않는 지점이 있었습니다. 원인이 코드가 아니라 통신 설정이나 하드웨어에 있던 순간도 많았습니다. 그런 경험이 본문의 설명과 트러블슈팅에 그대로 담겨 있습니다.

ROS의 세계에서 오래 헤매는 사람과 빠르게 나아가는 사람을 가르치는 것은 문법 지식이 아닙니다. 왜 ROS가 필요한지, 노드와 토픽이라는 약속이 어떤 설계 철학에서 나왔는지, 그 위에서 메시지가 실제로 어떻게 오가는지를 이해해야 비로소 코드가 의도대로 동작하고 로봇이 제어됩니다. 그래서 이 책은 명령어 뒤에 있는 구조를 함께 보여 주려 했습니다.

이 책은 혼자였다면 나오지 못했을 결과물입니다. 각자의 자리에서 쌓아 온 경험을 아낌없이 내어 준 공저자 여러분께 깊이 감사드립니다. 로봇을 제대로 만들어 보려는 분들에게 이 책이 단단한 출발점이 되기를 바랍니다.

2026년, 손민호

차례

1부. ROS2 시작하기

1장. ROS2 개요와 개발 환경 구축	10
1.1 로봇 소프트웨어와 ROS	12
1.2 ROS1에서 ROS2로	14
1.3 ROS2의 심장, DDS	15
1.4 배포판 선택 - 왜 Jazzy인가	18
1.5 다섯 가지 통신 미리보기	19
1.6 개발 환경 준비	20
1.7 ROS2 Jazzy 설치	21
1.8 환경 설정	22
1.9 설치 확인 - 첫 통신	24
1.10 [미니 실습] DDS 구현 바꿔 보기	26
1.11 자주 만나는 오류와 해결	26
2장. 워크스페이스와 패키지, 빌드 시스템	31
2.1 워크스페이스란 무엇인가	33
2.2 colcon과 ament - 빌드 시스템의 두 축	34
2.3 [따라하기] 워크스페이스 만들기	35
2.4 [따라하기] Python 패키지 만들기	37
2.5 [따라하기] C++ 패키지 만들기	39
2.6 패키지 구조 해부 - 무엇이 무엇을 정의하나	40
2.7 오버레이와 언더레이 - 내 코드가 인식되는 원리	41
2.8 트러블슈팅 - 막혔을 때	43
2.9 [워크드 예제] 빈 워크스페이스부터 첫 패키지까지 한 호흡	43
2.10 colcon 자주 쓰는 옵션 정리	44

2부. ROS2 노드 통신과 실행 (Python & C++)

3장. 토픽 - 발행/구독	49
3.1 토픽 통신 모델	52
3.2 퍼블리셔 만들기	54
3.3 서브스크라이버 만들기	58
3.4 C++로 구현하기	60
3.5 QoS 설정	65

3.6 ros2 topic 명령으로 살펴보기	66
4장. 서비스 — 요청/응답	78
4.1 서비스 통신 모델	80
4.2 서비스 서버와 클라이언트	81
4.3 C++로 구현하기	85
4.4 ros2 service 명령으로 살펴보기	87
5장. 액션 — 목표/피드백/결과	93
5.1 액션 통신 모델	96
5.2 액션 서버와 클라이언트	99
5.3 피드백과 목표 취소	116
5.4 ros2 action 명령으로 살펴보기	121
6장. 파라미터와 커스텀 인터페이스	130
6.1 파라미터란 — 코드를 안 고치고 바꾸기	132
6.2 [따라하기] 파라미터 사용 — Python	132
6.3 [따라하기] 파라미터 사용 — C++	134
6.4 YAML로 파라미터 일괄 주입	137
6.5 커스텀 인터페이스 — 나만의 타입	138
6.6 커스텀 타입 사용하기	140
6.7 [워크드 예제] 런타임 파라미터 변경 반영	142
6.8 트러블슈팅 — 막혔을 때	143
6.9 커스텀 인터페이스 — 흔한 실수 체크리스트	144
7장. 런치 시스템	149
7.1 런치란 무엇인가	152
7.2 파이썬 런치 파일 작성	154
7.3 노드 구성과 리매핑	157
7.4 파라미터와 실행 인자 전달	160
7.5 런치 조합하기	164
7.6 [미니 실습] 런치 하나로 시스템 띄우기	166
7.7 Foxy에서 Jazzy로 옮길 때	167
부록. ros2 명령어 치트시트	172
참고문헌	174

01

ROS2 개요와 개발 환경 구축

ROS1과 ROS2의 차이와 DDS를 이해한다.
Ubuntu 24.04에 Jazzy를 설치하고
첫 통신까지 확인한다.

이 장에서 다루는 내용

- ROS와 ROS2가 로봇 소프트웨어에서 맡는 역할과 활용 분야
- ROS1과 ROS2의 차이, 그 바탕인 DDS와 ROS2의 계층 구조
- 앞으로 배울 다섯 가지 통신 방식의 개요
- 이 책이 Jazzy 배포판과 Ubuntu 24.04를 선택한 이유
- ROS2 Jazzy 설치와 환경 설정(네이티브, WSL 2)
- talker/listener 예제로 설치를 검증하고 시스템을 확인하는 방법
- DDS 구현을 바꿔 같은 코드가 동작하는지 확인하는 미니 실습

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. ROS가 로봇 소프트웨어에서 맡는 역할과 활용 분야를 설명할 수 있습니다.
2. ROS1과 ROS2의 핵심 차이를 이해하고, ROS2가 DDS 위에서 동작하는 의미를 설명할 수 있습니다.
3. ROS2의 계층 구조를 이해하고, 다섯 가지 통신 방식을 구분할 수 있습니다.
4. 이 책이 Jazzy 배포판을 선택한 이유를 LTS 개념과 함께 설명할 수 있습니다.
5. Ubuntu 24.04에 ROS2 Jazzy를 설치하고 환경 변수를 설정할 수 있습니다.
6. talker/listener 예제로 설치를 검증하고, ros2 명령으로 노드와 토픽을 확인할 수 있습니다.
7. RMW 개념을 이해하고 DDS 구현을 바꿔 실행할 수 있습니다.

1.1 로봇 소프트웨어와 ROS

로봇은 소프트웨어 덩어리다

겉으로 보면 로봇은 모터와 센서, 금속 골격으로 이루어진 기계입니다. 하지만 그 기계를 로봇으로 만드는 것은 소프트웨어입니다. 카메라가 보낸 영상에서 사람을 찾아내고, 거리 센서로 벽까지의 거리를 재고, 그 정보로 어디로 갈지 결정하고, 바퀴 모터에 속도 명령을 내리는—이 모든 일이 소프트웨어의 몫입니다.

문제는 이 일들이 저마다 다른 전문 분야라는 점입니다. 영상 인식, 센서 처리, 위치 추정, 경로 계획, 모터 제어를 한 사람이 모두 잘 만들기는 어렵습니다. 그래서 현대의 로봇 소프트웨어는 하나의 거대한 프로그램이 아니라, **여러 개의 작은 프로그램이 서로 데이터를 주고받는 형태**로 만들어집니다. 이 작은 프로그램 하나하나를 ROS에서는 **노드(node)**라고 부릅니다.

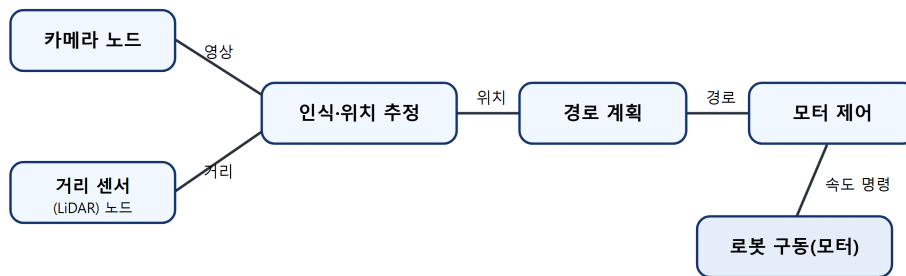


그림 1-1. 카메라·거리센서·경로계획·모터제어 노드가 데이터를 주고받는 로봇 소프트웨어 구조

ROS는 어디에 쓰이나

ROS는 특정 로봇 한 종류만을 위한 기술이 아닙니다. 오늘날 ROS와 ROS2는 우리 주변의 수많은 로봇 속에서 동작하고 있습니다.

- **물류·자율주행**: 창고를 누비는 운반 로봇(AMR), 실내외 배달 로봇
- **제조·협동로봇**: 공장에서 부품을 옮기고 조립하는 로봇 팔(매니퓰레이터)
- **드론·수중 로봇**: 비행 제어와 임무 수행, 해양 탐사
- **서비스 로봇**: 식당 서빙, 병원 물류 이송, 안내 로봇
- **연구·교육**: 전 세계 대학과 기업 연구소의 사실상 표준 플랫폼

물류·제조·우주 분야의 여러 기업과 연구기관이 ROS를 기반으로 로봇을 개발합니다. 이 책에서 다루는 기술은 학습용 예제에 그치지 않고 현업에서 그대로 쓰이는 표준입니다.

ROS의 등장 – 로봇 소프트웨어의 공통 골격

여러 노드가 협력하려면 공통의 규칙이 필요합니다. 데이터를 어떤 형식으로 어떻게 주고받을지 정해진 약속이 없다면, 부품을 새로 만들 때마다 연결 방식을 다시 정의해야 합니다. ROS(Robot Operating System) 는 이 약속과 도구를 모아 놓은 것입니다.

ROS는 2007년 미국의 윌로우 개라지(Willow Garage)에서 시작되어, 전 세계 연구자와 기업이 함께 키워 온 오픈소스 프로젝트입니다. ROS가 제공하는 것은 크게 세 가지입니다.

- **통신 방식:** 노드들이 데이터를 주고받는 표준 방법(토픽, 서비스, 액션 등)
- **도구:** 데이터를 들여다보고, 시각화하고, 기록하고, 다시 재생하는 유틸리티
- **생태계:** 센서 드라이버, 내비게이션, 시뮬레이터 등 이미 만들어진 수많은 패키지

덕분에 개발자는 바닥부터 만들지 않고, 검증된 부품을 가져와 자신의 로봇에 맞게 조립할 수 있습니다.

ROS는 운영체제가 아닙니다

이름에 'Operating System'이 들어가지만, ROS는 윈도우나 리눅스 같은 운영체제가 아닙니다. ROS는 운영체제 **위에서** 동작하며, 여러 노드가 협력하도록 돕는 **미들웨어(middleware)** 이자 개발 도구 모음입니다. 운영체제가 프로그램과 하드웨어 사이를 중개하듯, ROS는 로봇을 이루는 노드들 사이를 중개한다고 이해하면 됩니다.

정의(1) 미들웨어

서로 다른 프로그램이나 시스템이 데이터를 주고받을 수 있도록 중간에서 연결해 주는 소프트웨어 계층. ROS2는 로봇 소프트웨어 노드 간 통신을 책임지는 미들웨어다.

1.2 ROS1에서 ROS2로

ROS1의 한계

ROS1은 로봇 연구의 사실상 표준이 될 만큼 성공했지만, 설계상 몇 가지 한계가 있었습니다.

- **중앙 관리자(master) 의존:** ROS1은 **roscore**라는 중앙 관리자가 모든 연결을 중개합니다. 이 관리자가 멈추면 시스템 전체가 멈춥니다.
- **실시간성 부족:** 정해진 시간 안에 반드시 반응해야 하는 제어에는 적합하지 않았습니다.
- **보안 기능 부재:** 통신을 암호화하거나 접근을 제어하는 장치가 없었습니다.
- **다중 로봇·산업 환경의 어려움:** 여러 대의 로봇, 불안정한 네트워크를 견디는 설계가 아니었습니다.

ROS2의 재설계

ROS2는 이러한 한계를 보완하기 위해 기존 코드를 고치는 대신 **처음부터 다시 설계** 되었습니다. 가장 큰 변화는 통신의 토대를 산업 표준 미들웨어인 DDS(Data Distribution Service) 위에 올린 것입니다. 이를 통해 중앙 관리자 없이도 노드들이 서로를 찾아 연결되고, 통신 품질을 세밀하게 조절하며, 보안과 실시간 요구까지 끌어안을 수 있게 되었습니다.

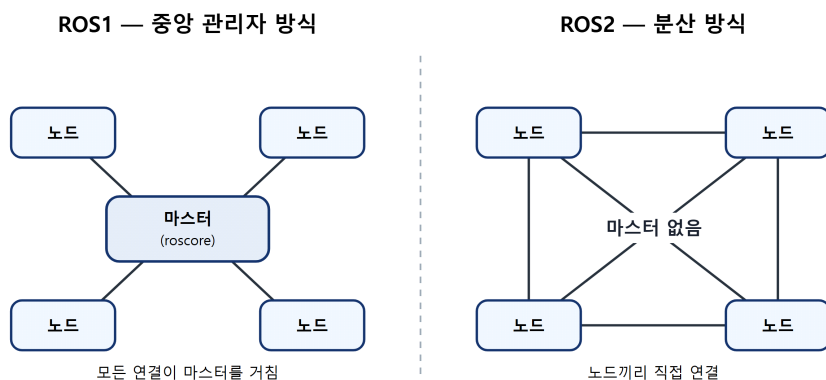


그림 1-2. ROS1(중앙 master 경유) vs ROS2(master 없는 분산 연결) 구조 비교

무엇이 달라졌나

ROS1과 ROS2의 주요 차이를 정리하면 다음과 같습니다.

구분	ROS1	ROS2
통신 기반	자체 프로토콜(TCPROS)	산업 표준 DDS
중앙 관리자	roscore 필요	불필요(분산 발견)
실시간 제어	어려움	지원
보안	없음	지원(SROS2)
빌드 도구	catkin	colcon / ament
Python 클라이언트	rospy	rclpy
C++ 클라이언트	roscpp	rclcpp
주요 대상	연구	연구 + 산업

표 1-1. ROS1과 ROS2의 주요 차이점

표에서 보듯 이름만 바뀐 것이 아니라 토대가 바뀌었습니다. 이 책에서 익히는 **rclpy**(파이썬)와 **rclcpp**(C++)는 ROS2의 표준 클라이언트 라이브러리입니다.

[칼럼] ROS2 버전 이름 이야기

ROS2 배포판은 알파벳 순서로 이름이 붙습니다. Foxy, Galactic, Humble, Iron, Jazzy처럼요. 각 이름에는 거북이(ROS의 상징) 종류나 지명이 담기는데, Jazzy의 정식 이름은 'Jazzy Jalisco'입니다. 새 배포판은 보통 매년 5월에 발표됩니다.

1.3 ROS2의 심장, DDS

DDS란

DDS(Data Distribution Service)는 분산 시스템에서 데이터를 안정적으로 주고받기 위한 국제 표준 통신 규격입니다. 항공, 국방, 금융, 의료처럼 한 치의 오차도 허용되지 않는 분야에서 오래 검증된 기술입니다. ROS2는 이 검증된 기반 위에서 통신을 처리합니다.

마스터가 없습니다 - 분산 발견

ROS1과 비교했을 때 가장 체감되는 변화는 **중앙 관리자가 사라진** 것입니다. ROS2에서는 각 노드가 네트워크에 자신을 알리고 상대를 스스로 찾아 연결합니다. 이를 **분산 발견(distributed discovery)**이라고 합니다. 덕분에 시스템 일부가 멈춰도 나머지는 계속 동작하며, 로봇을 켜고 끄는 순서에도 자유로워집니다.

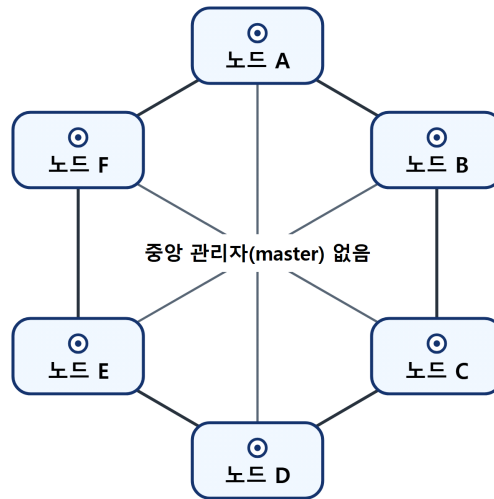


그림 1-3. 노드들이 서로를 직접 찾아 연결되는 DDS 분산 발견 개념

QoS - 통신 품질을 고른다

DDS의 강력한 기능 중 하나는 QoS(Quality of Service, 서비스 품질) 설정입니다. 모든 데이터를 똑같이 보내는 대신, 데이터의 성격에 맞게 전달 방식을 고를 수 있습니다.

- 신뢰성(reliability): 한 건도 빠뜨리지 않고 보낼지(reliable), 좀 놓쳐도 빠른 게 나은지(best effort)

- 이력(history): 최근 몇 개의 데이터를 보관할지

- 지속성(durability): 늦게 접속한 상대방에게도 과거 데이터를 전달할지

예를 들어 카메라 영상은 한두 프레임 놓쳐도 괜찮으니 빠른 전달을, 비상 정지 명령은 절대 놓치면 안 되니 신뢰성 있는 전달을 선택합니다. QoS는 3장에서 직접 다뤄 봅니다.

RMW - DDS를 갈아 끼우는 계층

DDS는 여러 회사가 만든 여러 제품이 있습니다. ROS2는 특정 제품에 묶이지 않도록 RMW(ROS Middleware) 라는 중간 계층을 두어, 코드를 바꾸지 않고도 DDS 구현을 갈아 끼울 수 있게 했습니다. Jazzy의 기본 구현은 eProsima의 Fast DDS이며, 필요하면 다른 구현으로 바꿀 수 있습니다. 이 장 끝의 미니 실습에서

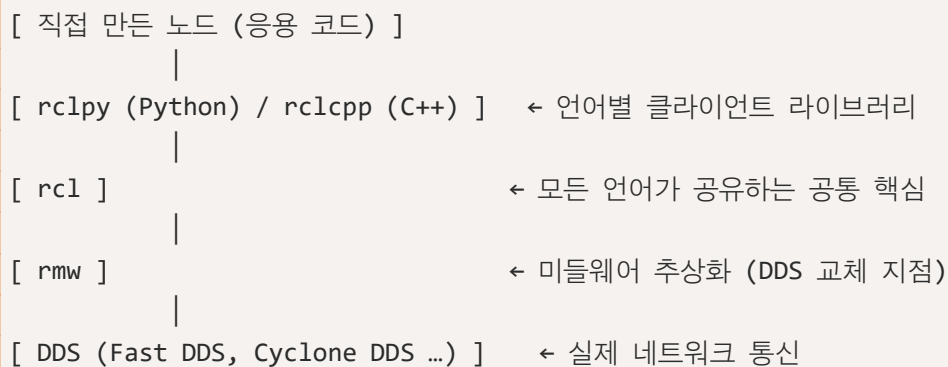
직접 바꿔 봅니다.

정의(2) RMW

ROS2와 DDS 사이를 연결하는 추상화 계층. 덕분에 같은 ROS2 코드가 서로 다른 DDS 제품 위에서 동일하게 동작합니다.

ROS2 계층 구조 한눈에 보기

지금까지 ROS2, DDS, RMW를 따로 살펴봤습니다. 이들이 어떻게 층층이 쌓여 동작하는지 한 장의 그림으로 정리해 봅시다. 여러분이 작성한 노드 코드가 맨 위에 있고, 그 아래로 여러 계층을 거쳐 실제 통신을 담당하는 DDS까지 내려갑니다.



위쪽 두 계층(**rclpy/rclcpp**)만 우리가 직접 코드로 다룹니다. 그 아래는 ROS2가 알아서 처리합니다. 여기서 두 가지 중요한 사실이 보입니다. 첫째, **rclpy**와 **rclcpp**가 똑같이 **rcl** 위에 올라가 있기 때문에 **파이썬 노드와 C++ 노드가 아무 문제 없이 통신** 할 수 있습니다(1.9절에서 직접 확인합니다). 둘째, **rmw** 계층 덕분에 맨 아래 DDS를 갈아 끼워도 윗부분 코드는 그대로입니다(1.10절 미니 실습에서 확인합니다).

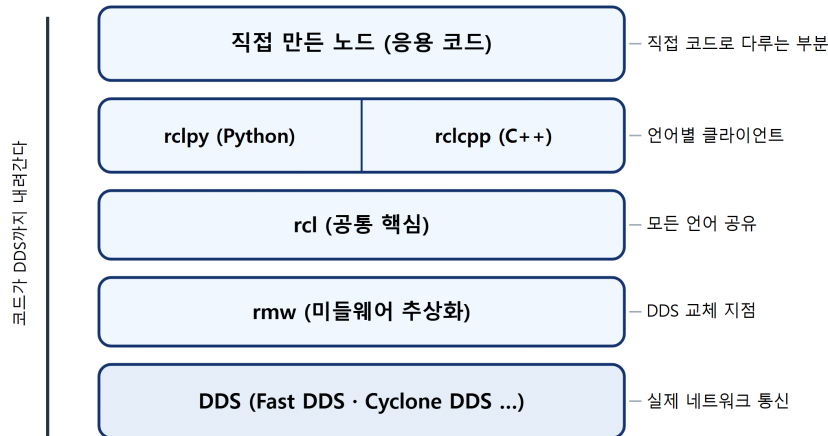


그림 1-4. 응용 코드부터 DDS까지 — ROS2의 계층 구조

1.4 배포판 선택 — 왜 Jazzy인가

ROS2 배포판과 LTS

ROS2는 대략 1년에 한 번 새로운 **배포판(distribution)** 을 내놓습니다. 각 배포판에는 알파벳 순서의 이름이 붙습니다. 배포판은 두 종류로 나뉩니다.

- **LTS(Long-Term Support, 장기 지원) 배포판**: 약 5년간 지원. 안정성이 중요할 때 선택.
- **일반 배포판**: 약 1.5년 지원. 최신 기능을 빨리 쓰고 싶을 때 선택.

Jazzy Jalisco

Jazzy Jalisco 는 2024년에 발표된 LTS 배포판으로, 2029년까지 지원됩니다. 짝을 이루는 운영체제는 같은 해 발표된 Ubuntu 24.04 LTS입니다. 새 LTS이면서 운영체제와 지원 기간이 길게 맞물려 있어, 지금 학습을 시작하는 책의 기준 환경으로 가장 적합합니다.

정의(3) LTS

장기 지원 버전. 자주 바뀌지 않고 오랜 기간 보안·버그 수정을 받기 때문에, 교육과 제품 개발의 안정적인 토대가 됩니다.

이 책의 기준 환경

이 책의 모든 예제는 다음 환경에서 작성하고 검증했습니다.

항목	버전
운영체제	Ubuntu 24.04 LTS (Noble Numbat)
ROS2 배포판	Jazzy Jalisco
기본 DDS	Fast DDS
Python	3.12

표 1-2. 이 책의 기준 개발 환경

버전이 다르면 명령이나 동작이 조금씩 달라질 수 있으니, 학습 단계에서는 같은 환경을 맞추기를 권합니다.

1.5 다섯 가지 통신 미리보기

본격적인 설치에 앞서 앞으로 배울 내용을 개략적으로 살펴봅니다. ROS2에서 노드들은 **다섯 가지 방식**으로 대화합니다. 각 방식은 2부(3~7장)에서 하나씩 자세히 다룹니다.

통신 방식	한 줄 설명	비유	학습
토픽(topic)	한 방향으로 계속 흘러보내기	라디오 방송	3장
서비스(service)	요청하고 답을 받기	질문과 대답	4장
액션(action)	오래 걸리는 작업 + 중간 보고	택배 배송 추적	5장
파라미터(parameter)	노드의 설정값 다루기	환경 설정	6장
런치(launch)	여러 노드를 한 번에 실행	시동 스크립트	7장

표 1-3. ROS2의 다섯 가지 통신 방식

각 방식은 쓰임이 다릅니다. 끊임없이 흐르는 센서 데이터는 토픽으로, 한 번 묻고 답을 받는 계산은 서비스로, 오래 걸리는 이동 명령은 진행 상황을 받아 가며 액션으로 보냅니다. 파라미터는 노드의 설정을 바꾸고, 런치는 이 모든 노드를 한 번에 띄웁니다. 여기서는 각 방식의 쓰임을 개략적으로 구분하는 정도면 충분합니다. 자세한 사용법은 해당 장에서 코드로 익힙니다.

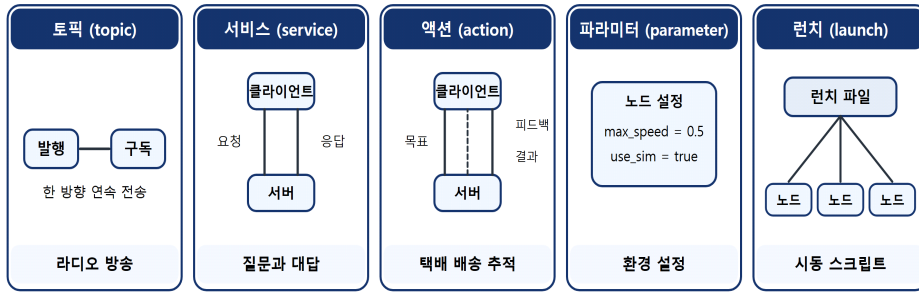


그림 1-5. ROS2의 다섯 가지 통신 방식 한눈에 보기

1.6 개발 환경 준비

Ubuntu 24.04 마련하기

ROS2 Jazzy는 Ubuntu 24.04를 기준으로 합니다. Ubuntu를 준비하는 방법은 여러 가지가 있습니다.

- **네이티브 설치:** PC에 Ubuntu만 설치. 성능이 가장 좋고 권장되는 방식입니다.
- **듀얼 부팅:** 윈도우와 Ubuntu를 함께 설치해 부팅 때 선택.
- **WSL 2:** 윈도우 안에서 리눅스를 실행. 가볍게 시작하기 좋습니다.
- **가상머신:** VirtualBox 등으로 윈도우/맥 위에서 실행. 간편하지만 속도가 느립니다.

처음 학습하는 단계라면 네이티브 설치나 듀얼 부팅을 권하지만, 윈도우를 그대로 쓰면서 시작하고 싶다면 WSL 2가 좋은 선택입니다.

WSL 2로 Ubuntu 설치하기 (윈도우 사용자)

윈도우 11이라면 WSL 2로 Ubuntu 24.04를 손쉽게 설치할 수 있습니다. 관리자 권한 PowerShell 을 열고 다음을 실행합니다.

```
wsl --install -d Ubuntu-24.04
```

설치가 끝나면 재부팅한 뒤 Ubuntu가 자동으로 실행되며, 사용자 이름과 비밀번호를 한 번 설정합니다. 이후에는 시작 메뉴에서 'Ubuntu 24.04'를 실행하면 리눅스 터미널이 열립니다. 윈도우 11의 WSL 2는 WSLg 기능을 내장하고 있어, RViz2나 rqt_graph 같은 그래픽 도구도 추가 설정 없이 창으로 뜹니다. 다만 그래

픽 가속이 필요한 무거운 시뮬레이션(2권)은 네이티브 환경을 권장합니다.

정의(4) WSL 2

Windows Subsystem for Linux 2. 윈도우 안에서 실제 리눅스 커널을 구동해 리눅스 프로그램을 그대로 실행하는 기능. 윈도우와 파일을 공유하면서 Ubuntu를 쓸 수 있습니다.

로케일 설정

설치에 앞서 시스템이 UTF-8 문자를 제대로 다루도록 로케일을 설정합니다. 터미널을 열어 다음을 실행합니다.

```
locale # 현재 설정 확인

sudo apt update && sudo apt install locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8 LANG=en_US.UTF-8
export LANG=en_US.UTF-8

locale # UTF-8 적용 확인
```

1.7 ROS2 Jazzy 설치

이제 본격적으로 ROS2를 설치합니다. 설치하는 ① 저장소 등록 → ② 패키지 설치 → ③ 개발 도구 설치의 순서로 진행됩니다.

apt 저장소 등록

ROS2는 Ubuntu의 패키지 관리자 **apt**로 설치합니다. 먼저 ROS2 패키지가 들어 있는 저장소를 시스템에 등록해야 합니다. Universe 저장소를 활성화한 뒤, ROS2 저장소 정보를 담은 패키지를 내려받아 설치합니다.

```
# Universe 저장소 활성화
sudo apt install software-properties-common
sudo add-apt-repository universe

# ROS2 저장소 등록 패키지 설치
sudo apt update && sudo apt install curl -y
export ROS_APT_SOURCE_VERSION=$(curl -s \
https://api.github.com/repos/ros-infrastructure/ros-apt-source/release
```

```
s/latest \  
| grep -F "tag_name" | awk -F\" '{print $4}')
```

```
curl -L -o /tmp/ros2-apt-source.deb \  
  
"https://github.com/ros-infrastructure/ros-apt-source/releases/download/${ROS_APT_SOURCE_VERSION}/ros2-apt-source_${ROS_APT_SOURCE_VERSION}.  
$(. /etc/os-release && echo $VERSION_CODENAME)_all.deb"  
  
sudo apt install /tmp/ros2-apt-source.deb
```

위 과정은 ROS2 저장소의 인증 키와 주소를 시스템에 자동으로 등록해 줍니다.

ROS2 설치

저장소가 등록되었으면 패키지 목록을 갱신하고 ROS2를 설치합니다. 설치 옵션은 여러 가지인데, 학습용으로는 시각화 도구까지 포함된 **desktop** 묶음을 권장합니다.

```
sudo apt update
sudo apt upgrade -y

# 데스크톱 묶음 설치 (RViz, 데모, 튜토리얼 포함)
sudo apt install ros-jazzy-desktop -y
```

`ros-jazzy-desktop`에는 핵심 라이브러리뿐 아니라 RViz2 같은 시각화 도구와 예제가 함께 들어 있어, 학습에 필요한 대부분이 한 번에 준비됩니다. 설치 용량이 부담된다면 핵심만 담긴 `ros-jazzy-ros-base`를 쓸 수도 있지만, 이 책은 `desktop`을 기준으로 합니다.

개발 도구 설치

패키지를 직접 빌드하려면 colcon을 비롯한 개발 도구가 필요합니다. 2장부터 사용하여 미리 설치해 둡니다.

```
sudo apt install ros-dev-tools -y
```

1.8 환경 설정

setup 파일 불러오기

ROS2 명령을 쓰려면, 현재 터미널에 ROS2 환경을 불러와야 합니다. 설치 경로
의 **setup.bash**를 실행(source)하면 됩니다.

```
source /opt/ros/jazzy/setup.bash
```

이제 이 터미널에서는 **ros2** 명령을 쓸 수 있습니다. 잘 되었는지 확인해 봅니다.

```
ros2 --help
```

.bashrc로 자동화하기

문제는 **source** 명령이 **현재 터미널에서만** 유효하다는 점입니다. 터미널을 새로 열 때마다 매번 입력하기는 번거롭습니다. 터미널이 열릴 때 자동으로 실행되도록 **.bashrc** 파일에 등록해 둡니다.

```
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc
```

이제부터는 새 터미널을 열면 ROS2 환경이 자동으로 준비됩니다.

정의(5) source

셸 스크립트를 현재 터미널 환경에 적용하는 명령. setup.bash를 source하면 ROS2 명령과 경로가 현재 터미널에 등록됩니다.

ROS_DOMAIN_ID - 통신 범위를 나눈다

같은 네트워크에 ROS2를 쓰는 여러 사람이 있다면, 서로의 데이터가 섞일 수 있습니다. 분산 발견이 같은 네트워크의 노드를 모두 찾아내기 때문입니다. 이를 막기 위해 **ROS_DOMAIN_ID** 라는 번호로 통신 그룹을 나눕니다. 같은 번호를 쓰는 노드끼리만 서로를 발견합니다.

```
export ROS_DOMAIN_ID=7
echo "export ROS_DOMAIN_ID=7" >> ~/.bashrc
```

0부터 101 사이의 값을 권장하며, 강의실이나 연구실에서는 사람마다 다른 번호를 쓰면 충돌을 피할 수 있습니다. 혼자 학습할 때는 통신 범위를 자기 컴퓨터 안으로 한정하는 설정도 유용합니다.

```
export ROS_AUTOMATIC_DISCOVERY_RANGE=LOCALHOST
```

RMW_IMPLEMENTATION - DDS 구현 선택

어떤 DDS 구현을 쓸지는 **RMW_IMPLEMENTATION** 환경 변수로 정합니다. 설정하지 않으면 Jazzy의 기본값인 Fast DDS(**rmw_fastrtps_cpp**)가 쓰입니다. 현재 어떤 구현이 동작 중인지는 다음으로 확인합니다.

```
ros2 doctor --report | grep middleware
```

DDS 구현을 직접 바꿔 보는 실습은 1.10절에서 진행합니다. 한 가지만 기억하세요, 서로 통신하는 노드는 같은 RMW 구현을 써야 합니다.

1.9 설치 확인 — 첫 통신

설치가 제대로 되었는지, ROS2가 기본으로 제공하는 예제로 확인해 봅니다. 한 노드가 메시지를 보내고(talker), 다른 노드가 받는(listener) 가장 단순한 통신입니다.

talker와 listener 실행

터미널 1 을 열고 메시지를 보내는 노드를 실행합니다.

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_cpp talker
```

다음과 같이 0.5초마다 메시지를 보내는 출력이 보이면 정상입니다.

```
[INFO] [talker]: Publishing: 'Hello World: 1'
[INFO] [talker]: Publishing: 'Hello World: 2'
[INFO] [talker]: Publishing: 'Hello World: 3'
```

이제 터미널 2 를 새로 열고 메시지를 받는 노드를 실행합니다.

```
source /opt/ros/jazzy/setup.bash
ros2 run demo_nodes_py listener
```

talker가 보낸 메시지를 listener가 받아 출력하면 통신이 성공한 것입니다.

```
[INFO] [listener]: I heard: [Hello World: 10]
[INFO] [listener]: I heard: [Hello World: 11]
```

여기서 talker는 C++(**demo_nodes_cpp**), listener는 Python(**demo_nodes_py**)으로 작성된 노드입니다. 서로 다른 언어로 만든 두 노드가 문제없이 통신합니다. 1.3 절에서 본 계층 구조에서 두 노드가 같은 **rcl** 위에 올라가 있기 때문입니다.

ros2 명령으로 들여다보기

통신이 오가는 동안, 터미널 3 을 열어 시스템을 들여다봅니다. 먼저 실행 중인 노드의 목록을 봅니다.

```
ros2 node list
```

```
/talker  
/listener
```

두 노드가 어떤 토픽으로 대화하는지도 확인할 수 있습니다.

```
ros2 topic list
```

```
/chatter  
/parameter_events  
/rosout
```

/chatter가 talker와 listener가 메시지를 주고받는 토픽입니다. 그 내용을 직접 엿볼 수도 있습니다.

```
ros2 topic echo /chatter
```

ros2 topic echo는 해당 토픽에 흐르는 데이터를 화면에 그대로 보여 줍니다. 우리가 만들지 않은 통신도 이렇게 들여다볼 수 있다는 것은, 디버깅에 매우 강력한 도구가 됩니다.

rqt_graph로 그림으로 보기

명령어 대신 그림으로 보고 싶다면 **rqt_graph**를 실행합니다.

```
rqt_graph
```

talker 노드에서 **/chatter** 토픽을 거쳐 listener 노드로 화살표가 이어진 그림이 나타납니다. 노드와 토픽의 관계를 한눈에 보여 주므로, 시스템이 복잡해질수록 유용합니다.

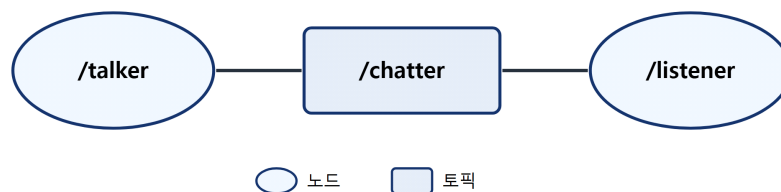


그림 1-6. rqt_graph로 본 talker → /chatter → listener 통신 구조

확인이 끝났으면 각 터미널에서 **Ctrl + C**를 눌러 노드를 종료합니다.

1.10 [미니 실습] DDS 구현 바꿔 보기

1.3절에서 ROS2는 RMW 계층 덕분에 DDS 구현을 갈아 끼울 수 있다고 했습니다. 정말 그런지 직접 확인해 봅시다. 기본 Fast DDS 대신 또 다른 대표 구현인 Cyclone DDS 로 바꿔도, 우리가 코드를 한 줄도 고치지 않고 talker/listener가 그대로 동작하는 것을 볼 것입니다.

1단계. Cyclone DDS 구현을 설치합니다.

```
sudo apt install ros-jazzy-rmw-cyclonedds-cpp -y
```

2단계. 두 개의 터미널 모두 에서 RMW 구현을 Cyclone DDS로 지정합니다. 앞서 강조했듯 통신하는 양쪽이 같은 구현을 써야 합니다.

```
# 터미널 1, 터미널 2 모두에서 실행
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
source /opt/ros/jazzy/setup.bash
```

3단계. 적용되었는지 확인합니다.

```
ros2 doctor --report | grep middleware
```

rmw_cyclonedds_cpp가 보이면 성공입니다.

4단계. 다시 talker와 listener를 실행합니다.

```
# 터미널 1
ros2 run demo_nodes_cpp talker
# 터미널 2
ros2 run demo_nodes_py listener
```

기본 Fast DDS일 때와 똑같이 통신됩니다. 같은 코드가 서로 다른 DDS 위에서 동일하게 동작한다는 것, 곧 RMW 계층이 제공하는 이식성을 직접 확인한 결과입니다. 실습을 마쳤으면 `export RMW_IMPLEMENTATION=` 로 변수를 비우거나 터미널을 새로 열어 기본값으로 돌아갑니다.

생각해 보기: 한쪽 터미널만 Cyclone DDS로 바꾸고 다른 쪽은 기본값(Fast DDS)으로 두면 어떻게 될까요? 직접 해 보고 결과를 관찰해 보세요.

1.11 자주 만나는 오류와 해결

설치와 첫 실행에서 자주 마주치는 문제와 해결책을 정리했습니다.

증상	원인	해결
ros2: command not found	setup.bash를 source하지 않음	source /opt/ros/jazzy/setup.bash 실행, .bashrc에 등록
listener가 메시지를 못 받음	두 터미널의 ROS_DOMAIN_ID가 다름	양쪽 ID를 같게 맞춤
RMW 바꾼 뒤 통신 안 됨	한쪽만 RMW_IMPLEMENTATION 변경	양쪽 터미널을 같은 구현으로 맞춤
다른 사람의 노드가 섞여 보임	같은 네트워크·같은 도메인 사용	ROS_DOMAIN_ID를 고유 번호로 변경
ros-jazzy-desktop 설치 실패	저장소 미등록 또는 apt 미갱신	저장소 등록 후 sudo apt update 재실행
add-apt-repository 명령 없음	software-properties-common 미설치	해당 패키지 먼저 설치
WSL에서 rqt 창이 안 뜸	구버전 WSL(WSLg 미지원)	wsl --update로 최신화 후 재시도

표 1-4. 설치·실행에서 자주 만나는 오류와 해결

대부분의 초기 문제는 **환경을 source하지 않았거나, ROS_DOMAIN_ID·RMW 구현이 양쪽에서 맞지 않아** 생깁니다. 통신이 안 될 때는 이 세 가지를 가장 먼저 의심하세요.

정리

- ROS는 로봇을 이루는 여러 노드가 표준 방식으로 통신하도록 돕는 미들웨어이자 도구 모음이며, 물류·제조·서비스·연구 등 현업 전반에서 쓰입니다.
- ROS2는 ROS1의 한계를 넘기 위해 산업 표준 DDS 위에서 다시 설계되었으며, 중앙 관리자 없이 분산 발전으로 동작합니다.
- ROS2는 응용 코드부터 DDS까지 여러 계층으로 이루어지며, 공통 계층 **rcl** 덕분에 파이썬·C++ 노드가 함께 통신하고, **rmw** 덕분에 DDS를 갈아 끼울 수 있습니다.
- 노드는 토픽·서비스·액션·파라미터·런치 다섯 가지 방식으로 대화하며, 이는 2부에서 차례로 배웁니다.
- 이 책은 LTS 배포판인 Jazzy와 Ubuntu 24.04를 기준으로 하며, 윈도우 사용자는 WSL 2로도 시작할 수 있습니다.
- 설치 후에는 setup.bash를 source하고 ROS_DOMAIN_ID를 설정해야 하며,

talker/listener 예제로 동작을 검증합니다.

핵심 용어

- ROS(Robot Operating System): 로봇 소프트웨어 노드 간 통신과 개발을 돕는 미들웨어·도구 모음
- 노드(node): 하나의 일을 맡아 실행되는 ROS2 프로그램의 기본 단위
- 미들웨어: 서로 다른 프로그램을 중간에서 연결해 주는 소프트웨어 계층
- DDS(Data Distribution Service): ROS2 통신의 기반이 되는 국제 표준 통신 규격
- 분산 발견: 중앙 관리자 없이 노드가 서로를 스스로 찾아 연결하는 방식
- QoS: 통신의 신뢰성·이력·지속성 등 전달 품질을 정하는 설정
- RMW: ROS2와 DDS 구현 사이를 잇는 추상화 계층
- rcl / rclpy / rclcpp: ROS2의 공통 핵심 계층(rcl)과 그 위의 파이썬·C++ 클라이언트 라이브러리
- LTS: 약 5년간 지원되는 장기 지원 배포판
- ROS_DOMAIN_ID: 같은 번호끼리만 통신하도록 그룹을 나누는 환경 변수
- RMW_IMPLEMENTATION: 사용할 DDS 구현을 지정하는 환경 변수

연습문제

1. ROS가 운영체제가 아니라 미들웨어라고 불리는 이유를 한 문장으로 설명하십시오.
2. ROS1의 **roscore**에 해당하는 중앙 관리자가 ROS2에는 없습니다. 이를 가능하게 한 DDS의 기능은 무엇인가?
3. 파이썬으로 만든 노드와 C++로 만든 노드가 서로 통신할 수 있는 이유를 ROS2 계층 구조(rcl)와 연결지어 설명하십시오.
4. 다음 상황에 가장 적합한 통신 방식(토픽/서비스/액션)을 고르고 그 이유를 서술하십시오. (가) 라이다 거리 데이터를 계속 받기 (나) 두 수를 더한 결과를 한

변 받기 (다) 목적지까지 이동시키며 남은 거리를 보고받기

4. 같은 네트워크의 동료와 노드가 섞이지 않게 하려면 어떤 환경 변수를 어떻게 설정해야 하는가?

5. (심화) 1.10절 실습에서 한쪽 터미널만 RMW 구현을 바꾸면 통신이 되지 않습니다. 그 이유를 RMW와 DDS 개념으로 설명하시오.

연습문제 해설

1. ROS는 하드웨어를 직접 관리하는 운영체제가 아니라, 리눅스 같은 운영체제 위에서 노드 실행과 통신, 개발 도구를 제공하는 소프트웨어 계층이기 때문입니다.

2. DDS의 자동 탐색(discovery)입니다. 노드들이 중앙 서버 없이 같은 도메인의 상대를 스스로 찾아 연결하므로 roscore 같은 중앙 관리자가 필요 없습니다.

3. 두 노드 모두 언어별 라이브러리(rclpy·rclcpp) 아래의 공통 계층 rcl과 DDS를 거쳐 같은 규격의 메시지를 주고받기 때문입니다. 언어가 달라도 전송되는 메시지 형식은 같습니다.

4. ROS_DOMAIN_ID 를 서로 다른 값으로 설정합니다. 도메인이 다르면 DDS 탐색 범위가 갈라져 상대의 노드가 보이지 않습니다.

5. RMW는 ROS2와 특정 DDS 구현을 잇는 어댑터입니다. 양쪽이 서로 다른 DDS 구현을 쓰면 탐색·전송 규약의 호환이 보장되지 않아 통신이 실패할 수 있습니다. RMW_IMPLEMENTATION 을 같은 값으로 맞추면 다시 연결됩니다.

마무리 점검

☐ ROS2가 라이브러리·통신·도구를 묶은 로봇 소프트웨어 플랫폼임을 설명할 수 있습니다.

☐ ROS1과 ROS2의 차이, 그리고 DDS가 맡는 역할을 설명할 수 있습니다.

☐ 다섯 가지 통신 방식의 쓰임을 한 줄씩 구분할 수 있습니다.

☐ Ubuntu 24.04에 ROS2 Jazzy를 설치하고 환경 설정을 마쳤습니다.

☐ talker/listener 예제로 설치를 검증했습니다.

□ 설치·실행 오류가 나면 표 1-4에서 증상부터 찾을 수 있습니다.

다음 장 예고

– 2장에서는 내 코드를 담은 그릇을 만듭니다. 워크스페이스를 만들고 colcon으로 빌드해, 3장부터 작성할 노드가 들어갈 `my_py_pkg` 와 `my_cpp_pkg` 를 준비합니다.

02

워크스페이스와 빌드 시스템

colcon과 ament로 패키지를 빌드한다.
파이썬과 C++ 패키지를 만들고
오버레이 구조를 익힌다.

이 장에서 다루는 내용

- 워크스페이스 폴더 구조와 src/build/install/log의 역할
- colcon과 ament의 역할 분리 및 빌드·소싱 흐름
- Python·C++ 패키지 생성과 디렉터리 구조
- 오버레이/언더레이 계층으로 내 코드가 인식되는 원리

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 워크스페이스(workspace)가 무엇이고 왜 필요한지 설명할 수 있습니다.
2. colcon과 ament의 역할, 그리고 빌드와 소싱(sourcing)의 흐름을 설명할 수 있습니다.
3. 내 손으로 워크스페이스를 만들고 colcon build 할 수 있습니다.
4. Python 패키지(ament_python)와 C++ 패키지(ament_cmake)를 각각 생성하고 구조를 읽을 수 있습니다.
5. 오버레이/언더레이(overlay/underlay) 개념으로 내 코드가 왜 인식되는지 설명할 수 있습니다.

2.1 워크스페이스란 무엇인가

워크스페이스(workspace) 는 내가 만든 ROS 2 패키지들을 한곳에 모아 함께 빌드하는

작업 폴더입니다. ROS 2 개발은 거의 항상 이 구조 위에서 이뤄집니다.

워크스페이스 이름은 관례상 `ros2_ws` 를 많이 쓰지만, 사용자가 임의로 정의할 수 있습니다.

이 책에서는 홈 디렉터리 아래 `ros2_ws` 를 기준으로 설명합니다.

```
mkdir -p ~/ros2_ws/src
```

핵심 규칙은 단 하나입니다.

모든 소스 패키지는 워크스페이스의 `src` 폴더 안에 둡니다.

`src` 에 패키지를 넣고 워크스페이스 최상위에서 빌드하면, ROS 2는 자동으로 `src` 아래를

훑어 패키지들을 찾아 빌드합니다. 빌드 결과는 워크스페이스 안의 네 폴더에 나뉘어 생깁니다.

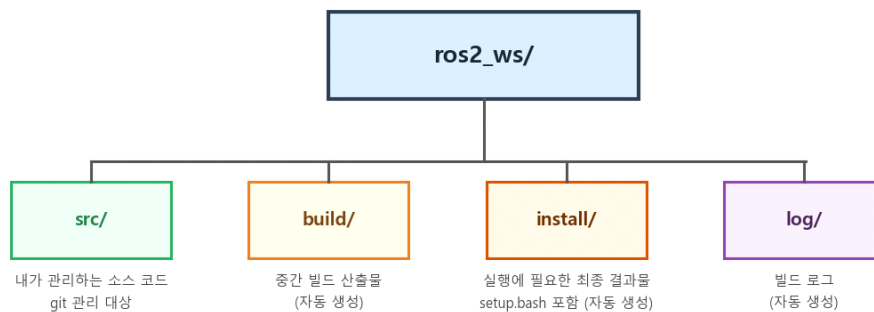


그림 2-1. 워크스페이스 폴더 구조

폴더	생성 시점	내용
src/	내가 직접 생성	소스 코드(패키지들). 유일하게 사람이 손대는 곳
build/	빌드 시 자동	중간 빌드 산출물(오브젝트 파일 등)
install/	빌드 시 자동	설치 결과 – 실행에 필요한 것이 여기 모입니다
log/	빌드 시 자동	빌드 로그

표 2-1. 워크스페이스 네 폴더의 생성 시점과 역할

build, install, log 는 언제든지 지우고 다시 빌드해도 됩니다. 버전 관리(git)에는 src 만 올리고 나머지 셋은 .gitignore 로 제외하는 것이 표준입니다.

클론한 사람은 자기 환경에서 colcon build 를 실행하면 install/ 이 새로 생성됩니다.

워크스페이스 최상위(~/.ros2_ws)에 아래 내용으로 .gitignore 를 만들면 됩니다.

```
build/  
install/  
log/
```

2.2 colcon과 ament — 빌드 시스템의 두 축

ROS 1을 써 봤다면 catkin_make 를 기억할 것입니다. ROS 2에서는 이것이 둘로 나뉘었습니다.

- **colcon** — Collective Construction 의 약자로, 워크스페이스 전체를 빌드하는 빌드 도구(명령)입니다.

src 폴더 아래 패키지들을 의존성 순서대로 모아서 빌드하는 것을 담당합니다.

(1장에서 ros-dev-tools 로 함께 설치했다)

- **ament** — 패키지 하나하나가 어떻게 빌드되는지를 정의하는 빌드 시스템으로 언어에 따라

두 종류를 씁니다.

- ament_python — Python 패키지용. setup.py/setup.cfg 로 설정.

- ament_cmake — C++ 패키지용. CMakeLists.txt 로 설정.

ament 는 패키지 내부를 구성하고 빌드하는 명세(API)고, colcon 은 이 ament 패키지들을

전체적으로 모아서(Collection) 의존성 순서대로 빌드해 주는 빌드 도구(Build Tool)입니다.

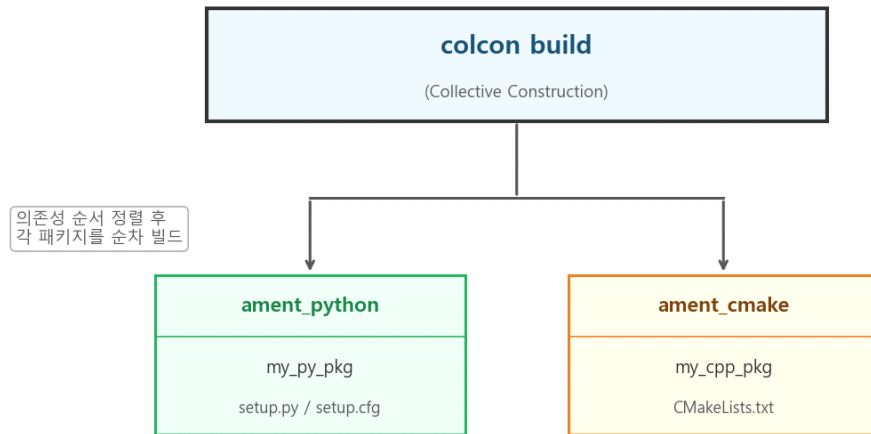


그림 2-2. colcon과 ament의 역할 분리

Foxy → Jazzy 차이

빌드 도구와 시스템의 큰 틀(colcon + ament)은 Foxy와 동일합니다. 다만 Jazzy의 기반인

Ubuntu 24.04 는 Python 3.12 를 쓰는데, 이 버전에서 ament_python 패키지를 빌드하면

setup.py 방식에 대한 경고(SetuptoolsDeprecationWarning)가 보일 수 있습니다. 빌드는 정상적으로 끝나며 동작에 문제없습니다. 경고와 오류를 구분하는 눈이 필요합니다(2.8절).

2.3 [따라하기] 워크스페이스 만들기

직접 만들어 보겠습니다. 터미널을 엽니다. (1장에서 ~/.bashrc 에 source /opt/ros/jazzy/setup.bash 를 넣었다고 가정합니다)

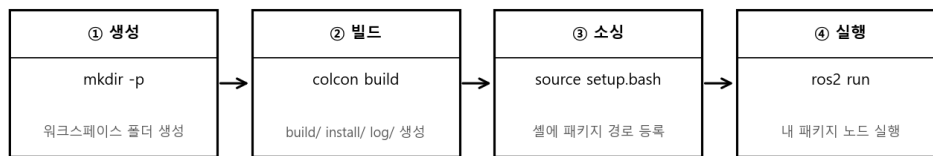


그림 2-3. 워크스페이스 생성부터 노드 실행까지의 흐름

① 워크스페이스 폴더와 src 만들기

```
mkdir -p ~/ros2_ws/src
cd ~/ros2_ws
```

-p 옵션은 중간 경로가 없어도 한 번에 생성하고, 이미 존재해도 오류를 내지 않습니다.

~/ros2_ws/src 처럼 여러 단계를 한 번에 만들 때 유용합니다.

② 비어 있는 상태에서 환경 체크

colcon build 는 반드시 워크스페이스 최상위 폴더(~/ros2_ws)에서 실행해야 합니다.

src 안이나 다른 위치에서 실행하면 빌드 결과가 엉뚱한 곳에 생기거나 오류가 발생합니다.

```
colcon build
```

src 가 비어 있어도 빌드는 성공하며, build install log 세 폴더가 생깁니다.

```
ls
# build install log src
```

각 폴더 안을 들여다보면 다음과 같은 파일이 생성돼 있습니다.

```
./build:
COLCON_IGNORE

./install:
COLCON_IGNORE local_setup.bash local_setup.sh local_setup.zsh
setup.bash setup.sh setup.zsh ...
```

```
./log:
build_2026-06-14_15-49-02 COLCON_IGNORE latest latest_build
```

폴더	역할
build/	각 패키지의 중간 빌드 산출물. CMake 캐시, 오브젝트 파일 등이 여기 쌓입니다.
install/	빌드 완료 후 실행에 필요한 파일이 모입니다. source 명령으로 이 폴더를 셸에 알립니다.
log/	빌드 로그, 빌드 오류 추적에 씁니다.

표 2-2. 빈 빌드가 만드는 세 폴더

환경이 정상인 것을 확인했으면 세 폴더를 삭제합니다. 실제 패키지를 추가한 뒤 다시 빌드할 것입니다.

```
rm -rf build install log
```

③ 워크스페이스 소싱(sourcing)

빌드한 결과를 셸에 알려 줘야 `ros2 run` 으로 내 패키지를 실행할 수 있습니다.

지금 `install/` 폴더를 삭제했으므로 2.4절에서 패키지를 빌드한 뒤 아래 명령을 실행합니다.

```
source ~/ros2_ws/install/setup.bash
```

이 줄이 바로 2.7절에서 설명할 오버레이를 활성화하는 명령입니다. 새 터미널을 열 때마다

워크스페이스를 쓰려면 다시 실행해야 합니다. 자주 쓰면 `~/.bashrc` 에 추가해도 되지 만,

워크스페이스가 여러 개라면 헷갈리니 처음엔 손으로 치며 흐름을 익히길 권합니다.

`~/.bashrc` 에 넣을 때는 반드시 절대 경로로 써야 합니다.

```
source ~/ros2_ws/install/setup.bash # 올바른 (절대 경로)
source install/setup.bash          # ~/.bashrc 에서는 잘못된 - 셸 시작
시 현재 폴더가 ~ 이므로 실패
```

`~/ros2_ws` 안에서 직접 실행할 때는 상대 경로도 동작합니다. 문제는 `~/.bashrc` 에 넣는 경우뿐입니다.

2.4 [따라하기] Python 패키지 만들기

이제 src 안에 첫 패키지를 만듭니다. ROS 2는 패키지 뼈대를 자동 생성해 주는 명령을 줍니다.

```
cd ~/ros2_ws/src
ros2 pkg create my_py_pkg --build-type ament_python --dependencies
rcldpy
```

- my_py_pkg — 패키지 이름
- --build-type ament_python — Python 패키지로 만듭니다
- --dependencies rcldpy — 의존성으로 rcldpy(ROS 2 Python 클라이언트 라이브러리) 추가

생성된 구조:

```
my_py_pkg/
├── my_py_pkg/          ← 실제 파이썬 모듈 (노드 .py 를 여기에 둔다)
│   └── __init__.py
├── resource/my_py_pkg
├── test/               ← 자동 생성된 테스트 (스타일 검사 등)
├── package.xml         ← 패키지 메타정보와 의존성
├── setup.py            ← 빌드·설치 설정 및 노드 실행 명령 등록
└── setup.cfg           ← 스크립트 설치 경로 지정 (최소 설정)
```

Jazzy(Python 3.12) 기준으로 패키지의 실질적인 설정은 setup.py 에 담깁니다.

노드를 ros2 run 으로 실행하려면 setup.py 의 entry_points 안 console_scripts 에 등록해야 합니다.

setup.py 의 console_scripts 부분:

```
entry_points={
    'console_scripts': [
        # 'my_node = my_py_pkg.my_node:main',    # 3장에서 채운다
    ],
},
```

setup.cfg 는 스크립트 설치 경로만 담는 최소 파일입니다. 직접 편집할 일은 거의 없습니다.

```
[develop]
```

```
script_dir=$base/lib/my_py_pkg
[install]
install_scripts=$base/lib/my_py_pkg
```

빌드하고 소싱합니다. 반드시 워크스페이스 최상위에서 빌드합니다.

```
cd ~/ros2_ws
colcon build
source ~/ros2_ws/install/setup.bash
```

`source ~/ros2_ws/install/setup.bash` 는 내 워크스페이스를 셸에 알리는 오버레이 소싱입니다.

`/opt/ros/jazzy/setup.bash` 가 ROS 2 본체(시스템 설치 패키지들)를 셸에 알리는 것이라면,

`~/ros2_ws/install/setup.bash` 는 그 위에 내가 만든 패키지를 얹는 것입니다.

자세한 내용은 2.7절에서 다룹니다.

Python 코드는 수정할 때마다 다시 빌드하기 번거롭습니다. 기본 빌드는 .py 파일을

`install/` 폴더에 복사하지만, `--symlink-install` 을 쓰면 복사 대신 `src/` 원본을

가리키는 심볼릭 링크를 `install/` 에 만듭니다. 원본을 고치면 즉시 반영되어

재빌드가 필요 없습니다. C++ 은 어차피 컴파일이 필요하므로 이 옵션의 효과가 없습니다.

```
colcon build --symlink-install
```

2.5 [따라하기] C++ 패키지 만들기

같은 워크스페이스에 C++ 패키지도 만들어 둡니다. (이 책은 Python/C++ 병행이 원칙입니다)

```
cd ~/ros2_ws/src
ros2 pkg create my_cpp_pkg --build-type ament_cmake --dependencies
rclcpp
```

- `--build-type ament_cmake` - C++ 패키지
- `--dependencies rclcpp` - C++ 클라이언트 라이브러리

생성된 구조:

```

my_cpp_pkg/
├─ include/my_cpp_pkg/ ← 헤더 (.hpp)
├─ src/                  ← 소스 (.cpp) 를 여기에 둔다
├─ package.xml           ← 메타정보와 의존성 (Python 과 동일 형식)
└─ CMakeLists.txt       ← 빌드 설정 (ament_cmake)

```

다시 워크스페이스 최상위에서 빌드합니다.

```

cd ~/ros2_ws
colcon build
source ~/ros2_ws/install/setup.bash

```

이제 워크스페이스에 패키지 두 개(my_py_pkg, my_cpp_pkg)가 함께 빌드됩니다. 확인:

```

ros2 pkg list | grep my_
# my_cpp_pkg
# my_py_pkg

```

2.6 패키지 구조 해부 — 무엇이 무엇을 정의하나

패키지를 처음 만들면 파일이 여럿 생겨 막막합니다. 딱 두 종류만 이해하면 됩니다.

package.xml — 이 패키지는 무엇이고 무엇에 의존하는가

Python/C++ 공통입니다. 패키지 이름, 버전, 설명, 라이선스, 그리고 의존성을 적습니다.

Jazzy 에서 ros2 pkg create 가 생성하는 package.xml 은 format 3 을 사용합니다.

```

<?xml version="1.0"?>
<?xml-model href="http://download.ros.org/schema/package_format3.xsd"
schematypens="http://www.w3.org/2001/XMLSchema"?>
<package format="3">
  <name>my_py_pkg</name>
  <version>0.0.0</version>

```

```

<description>TODO: Package description</description>
<maintainer email="user@todo.todo">user</maintainer>
<license>TODO: License declaration</license>

<depend>rclpy</depend>

<test_depend>ament_copyright</test_depend>
<test_depend>ament_flake8</test_depend>
<test_depend>ament_pep257</test_depend>
<test_depend>python3-pytest</test_depend>

<export>
  <build_type>ament_python</build_type>
</export>
</package>

```

colcon 은 이 의존성을 읽어 빌드 순서를 정합니다. A가 B에 의존하면 B를 먼저 빌드합니다.

setup.py (Python) / CMakeLists.txt (C++) — 무엇을 빌드·설치하는가

특히 Python 에서 노드를 실행 명령으로 등록하는 곳이 setup.py 의 console_scripts 다.

3장에서 노드를 만들면 여기에 등록해야 ros2 run 으로 실행됩니다. 지금은 위치만 기억합니다.

자주 하는 실수: 노드 .py 를 만들어도 setup.py 의 console_scripts 에 등록하지 않으면 ros2 run 이 못 찾습니다. 코드는 맞는데 실행이 안 될 때 1순위로 의심할 곳입니다.

2.7 오버레이와 언더레이 — 내 코드가 인식되는 원리

소싱을 두 번 한다는 점이 처음엔 헷갈립니다. 구조를 그림으로 보겠습니다.

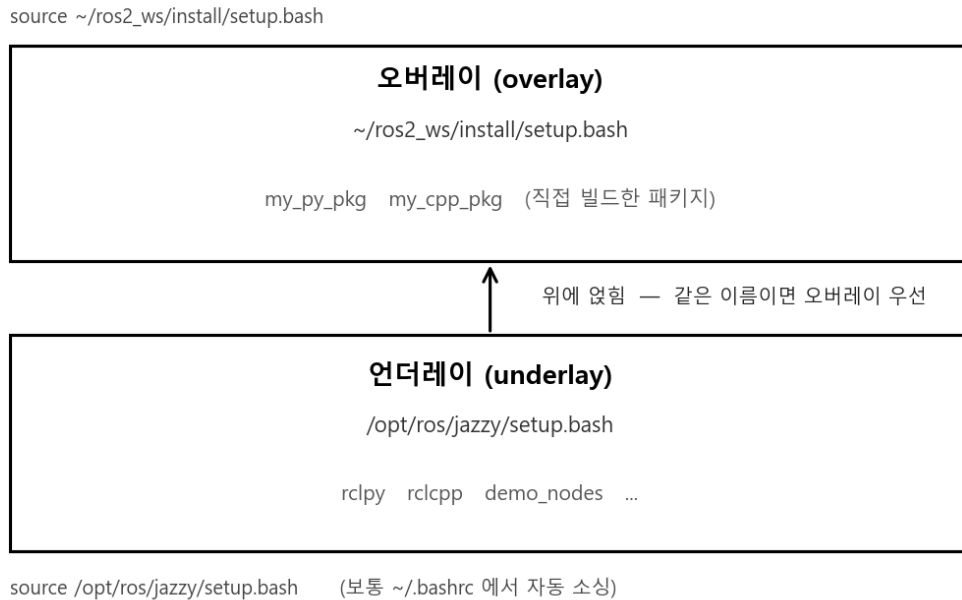


그림 2-4. 언더레이와 오버레이의 계층 구조

- **언더레이**: ROS 2 배포판 자체. 보통 ~/.bashrc 에서 자동 소싱(1장).
- **오버레이**: 내 워크스페이스. 언더레이 위에 얹혀, 같은 이름이 있으면 오버레이가 우선.

즉 source install/setup.bash 는 내가 방금 빌드한 패키지들을 이 셸이 알게 하라는 뜻입니다.

이걸 안 하면 ros2 run my_py_pkg ... 가 패키지를 못 찾음 오류를 냅니다.

setup.bash 와 local_setup.bash 의 차이

install 폴더에는 파일이 두 개 생깁니다.

- setup.bash — 언더레이 포함 전체 소싱. 일반적으로 이것을 씁니다.
- local_setup.bash — 이 워크스페이스만 소싱. 워크스페이스를 여러 겹 쌓는 고급 용도. 처음에는 setup.bash 만 쓰면 됩니다.

흔한 함정: 빌드한 터미널과 실행하는 터미널이 다른 경우. 새 터미널에는 오버레이가 안 걸려 있습니다. 새 터미널에서 실행하려면 그 터미널에서 다시

source ~/ros2_ws/install/setup.bash 를 실행합니다.

코드 해설 · 실행 확인 · 버전 체크

- **코드 해설 포인트:** package.xml, setup.py, CMakeLists.txt 가 빌드와 실행에 어떻게 관여하는지 해설합니다.
- **실행 확인 포인트:** colcon build, source install/setup.bash, ros2 pkg list | grep my_ 로 빌드 결과를 확인합니다.
- **버전/환경 체크:** Ubuntu 24.04 의 Python 3.12 와 setuptools 경고, Jazzy 의 colcon 사용 흐름을 기준으로 정리합니다.

2.8 트러블슈팅 — 막혔을 때

증상	원인	해결
colcon: command not found	dev-tools 미설치	sudo apt install ros-dev-tools (1장)
ros2 run 이 패키지를 못 찾음	오버레이 미소싱	source ~/ros2_ws/install/setup.bash
src 비었는데 빌드해도 install 없음	워크스페이스 최상위 아님	pwd 로 위치 확인 후 cd ~/ros2_ws
src 안에서 colcon build 를 쳤다	잘못된 위치	cd ~/ros2_ws 로 이동 후 재실행
SetuptoolsDeprecationWarning 노란 경고	Ubuntu 24.04 Python 3.12	경고일 뿐, 무시 가능. 빌드 성공은 마지막 Summary 출로 판단
패키지 하나만 다시 빌드하고 싶다	전체 재빌드는 느낌	colcon build --packages-select my_py_pkg
빌드가 꼬였다 (이상한 캐시)	build/install 오염	rm -rf build install log 후 재빌드
새 터미널에서 내 패키지가 안 보입니다	오버레이 소싱 안 함	새 터미널마다 source ~/ros2_ws/install/setup.bash

표 2-3. 워크스페이스에서 자주 만나는 오류와 해결

경고는 오류가 아닙니다. colcon build 마지막의 Summary: N packages finished 가 보이면 성공입니다. 빨간 Failed 가 없으면 노란 경고는 넘어가도 됩니다.

2.9 [워크드 예제] 빈 워크스페이스부터 첫 패키지까지 한 흐름

아래를 그대로 따라 하면 2장 전체가 손에 익습니다. 각 명령의 결과를 확인하며 진행합니다.

```
# 1. 워크스페이스 골격
mkdir -p ~/ros2_ws/src && cd ~/ros2_ws
colcon build                # build/ install/ log/ 생성
ls                          # build install log src

# 2. Python 패키지 생성
cd src
ros2 pkg create demo_py --build-type ament_python --dependencies rclpy
ls demo_py                  # demo_py/ package.xml setup.py
setup.cfg ...

# 3. 빌드 + 오버레이 소싱
cd ~/ros2_ws
colcon build --packages-select demo_py
source ~/ros2_ws/install/setup.bash

# 4. 인식 확인
ros2 pkg list | grep demo_py    # demo_py 가 보이면 성공
ros2 pkg executables demo_py    # (아직 노드 없음 → 비어 있음, 정상)
```

마지막에 demo_py 가 목록에 보이면, 빈 그릇에 첫 패키지를 담아 빌드/인식까지 시킨 것입니다.

3장에서 이 패키지 안에 노드를 넣으면 4번의 executables 에 노드 이름이 등장합니다.

2.10 colcon 자주 쓰는 옵션 정리

매번 전체 빌드는 느립니다. 실무에서 쓰는 옵션을 외워 두면 개발 속도가 크게 빨라집니다.

옵션	효과	언제
<code>--packages-select A B</code>	A, B 패키지만 빌드	특정 패키지만 수정했을 때
<code>--packages-up-to A</code>	A와 A가 의존하는 것까지	의존 패키지가 바뀌었을 때
<code>--symlink-install</code>	.py 를 install/ 에 복사 대신 심볼릭 링크로 - 원본 수정 시 재빌드 불필요 (C++ 에는 효과 없음)	Python 코드 수정했을 때
<code>--event-handlers console_direct+</code>	빌드 로그 실시간 출력	빌드가 멈춘 듯할 때 원인 파악
<code>rm -rf build install log</code>	캐시 초기화	빌드가 꼬였을 때

표 2-4. colcon build의 자주 쓰는 옵션

정리

- ROS2 워크스페이스는 src(소스)·build(중간 산출물)·install(실행 대상)·log(빌드 기록) 네 폴더로 이루어지며, 개발자가 직접 관리하는 것은 src뿐입니다.
- colcon은 여러 패키지를 한 번에 빌드하는 빌드 도구이고, ament는 그 아래에서 실제 빌드를 담당하는 빌드 시스템입니다. Python 패키지는 ament_python, C++ 패키지는 ament_cmake로 만듭니다.
- `ros2 pkg create`로 패키지 골격을 만들고, package.xml에 의존성을, setup.py 또는 CMakeLists.txt에 실행할 노드를 등록합니다.
- colcon build 다음에는 `source install/setup.bash`로 오버레이를 소싱해야 방금 빌드한 패키지가 인식됩니다. 새 터미널을 열 때마다 다시 소싱합니다.
- `--packages-select`, `--symlink-install` 같은 옵션으로 빌드 속도를 높일 수 있고, 캐시가 꼬이면 build/install/log를 지우고 다시 빌드합니다.

핵심 용어

- **워크스페이스(workspace)**: ROS2 패키지를 모아 빌드하는 작업 폴더입니다. src/build/install/log로 구성됩니다.
- **colcon**: 여러 패키지를 한 번에 빌드하는 빌드 도구입니다.

- **ament**: ROS2의 빌드 시스템입니다. Python용 `ament_python`과 C++용 `ament_cmake`가 있습니다.
- **패키지(package)**: 노드·설정·의존성을 담은 배포 단위입니다. `package.xml`로 메타데이터와 의존성을 정의합니다.
- **오버레이(overlay)**: 기반 환경(underlay) 위에 내 워크스페이스를 얹어, 내가 빌드한 패키지를 우선 인식하게 하는 구조입니다.

연습문제

1. `~/ros2_ws`에 `my_first_pkg`라는 Python 패키지를 `rcipy` 의존성으로 추가 생성하고 빌드하시오. `ros2 pkg list`에서 보이는지 확인하시오.
2. `build install log`를 모두 지운 뒤 다시 `colcon build`하시오. 무엇이 다시 생기는가?
3. 새 터미널을 열어 소싱 없이 `ros2 pkg list | grep my_`를 실행해 보라. 왜 안 보이는가?
보이게 하려면 무엇을 해야 하는가?
4. `package.xml`을 열어 `depend` 줄을 찾아라. `my_cpp_pkg`는 무엇에 의존하는가?
5. (생각해보기) 왜 `src`만 `git`에 올리고 `build/install/log`는 올리지 않을까?

연습문제 해설

- **1번** `ros2 pkg create my_first_pkg --build-type ament_python --dependencies rcipy` 후
워크스페이스 최상위에서 `colcon build` → `source ~/ros2_ws/install/setup.bash`
→ `ros2 pkg list`.
- **2번** 세 폴더를 지워도 `src`만 있으면 `colcon build`가 `build/install/log`를 다시

생성합니다. 소스가 src 에 있기 때문입니다.

- 3번 새 터미널은 오버레이가 안 걸려 있어 안 보입니다. 그 터미널에서 `source ~/ros2_ws/install/setup.bash` 를 하면 보입니다 (2.7절 오버레이).
- 4번 `my_cpp_pkg` 는 생성 시 `--dependencies rclcpp` 를 주었으므로 `rclcpp` 에 의존합니다.
- 5번 `build/install/log` 는 재생성 가능한 산출물이라 `git` 에 올리면 용량만 키우고 충돌을 부릅니다. 소스(src)만 버전 관리하는 것이 원칙입니다.

참고 자료

- ROS 2 Jazzy — Creating a workspace:
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-A-Workspace/Creating-A-Workspace.html>
- ROS 2 Jazzy — Creating a package:
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Creating-Your-First-ROS2-Package.html>
- colcon 문서: <https://colcon.readthedocs.io/>
- 표윤석/임태훈, ROS 2로 시작하는 로봇 프로그래밍, 루비페이퍼 — 빌드 시스템 장

마무리 점검

- ☐ 워크스페이스의 `src/build/install/log` 역할을 각각 한 줄로 말할 수 있습니다.
- ☐ `colcon`(빌드 도구)과 `ament`(빌드 시스템) 차이를 설명할 수 있습니다.
- ☐ Python(`ament_python`), C++(`ament_cmake`) 패키지를 각각 만들어 빌드했습니다.
- ☐ 오버레이 소싱(`source install/setup.bash`)이 왜 필요한지 압니다.

- `setup.bash` 와 `local_setup.bash` 의 차이를 압니다.
- 경고와 오류를 구분해 빌드 성공 여부를 판단할 수 있습니다.

다음 장 예고 – 3장에서는 방금 만든 `my_py_pkg/my_cpp_pkg` 안에 첫 노드를 작성하고, 토픽(topic)으로 메시지를 발행/구독합니다. `setup.py` 의 `console_scripts` 가 드디어 채워집니다.

03

토픽

노드가 메시지를 주고받는 기본 방식.
퍼블리셔와 서브스크라이버를 구현하고
QoS로 통신 품질을 맞춘다.

이 장에서 다루는 내용

- 토픽의 발행/구독(publish/subscribe) 통신 모델과 특징
- 퍼블리셔(publisher)와 서브스크라이버(subscriber) 노드를 Python으로 작성하는 방법
- C++로 동일한 토픽 통신 구현하기
- QoS 설정의 기본 개념과 깊이(depth) 값의 의미
- ros2 topic 명령어를 활용한 토픽 디버깅 방법

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 토픽(topic)의 단방향 발행/구독 모델을 서비스와 구분하여 설명할 수 있습니다.
2. rclpy/rclcpp로 퍼블리셔·서브스크라이버 노드를 직접 작성하고 실행할 수 있습니다.
3. setup.py와 CMakeLists.txt에 노드를 등록하고 빌드할 수 있습니다.
4. QoS 깊이 값 10이 무엇을 의미하는지 설명할 수 있습니다.
5. ros2 topic CLI 명령어로 토픽의 상태를 확인하고 문제를 진단할 수 있습니다.

이번 장에서 만드는 것

- /number 토픽으로 숫자를 발행하는 Python·C++ 퍼블리셔 노드
- /number 토픽을 구독해 값을 로그로 출력하는 Python·C++ 서브스크라이버 노드
- 받은 값을 2배로 만들어 /number_x2로 다시 발행하는 워크드 예제 노드
- ros2 topic 명령과 rqt_graph로 확인한 토픽 연결 상태

공통 학습 흐름: 개념 → 따라하기 → 코드 해설 → 실행 확인 → 버전/환경 체크 → 트러블슈팅 → 연습문제 → 마무리 점검

2장에서 ~/ros2_ws 워크스페이스에 Python 패키지 my_py_pkg와 C++ 패키지 my_cpp_pkg를 만들어 빌드까지 마쳤습니다. 하지만 두 패키지는 아직 비어 있는

그릇입니다. 이 장에서는 그 그릇에 첫 노드를 담습니다. 1장에서 talker/listener 예제로 구경만 했던 토픽 통신을 이번에는 직접 만들어 봅니다. 토픽은 ROS 2의 다섯 가지 통신 방식 중 가장 기본이고 가장 많이 쓰이는 방식이므로, 이 장의 내용은 이후 모든 장의 토대가 됩니다.

3.1 토픽 통신 모델

토픽이란: 이름이 붙은 단방향 데이터 통로

토픽은 노드들이 메시지를 주고받는 이름이 붙은 통로입니다. 데이터를 내보내는 노드를 퍼블리셔(publisher, 발행자), 받아 보는 노드를 서브스크라이버(subscriber, 구독자) 라고 부릅니다. 퍼블리셔가 토픽에 메시지를 내보내면, 그 토픽을 구독하고 있는 모든 서브스크라이버가 같은 메시지를 받습니다.

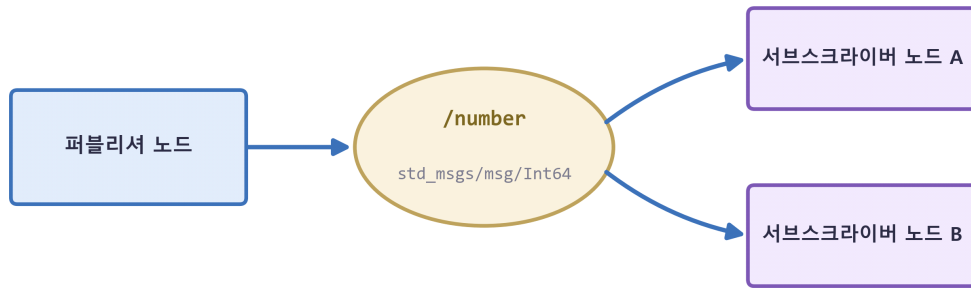
라디오 방송에 비유하면 이해가 쉽습니다. 방송국(퍼블리셔)은 주파수(토픽 이름)에 맞춰 전파를 내보낼 뿐, 누가 듣고 있는지 알지 못하고 알 필요도 없습니다. 청취자(서브스크라이버)는 주파수만 맞추면 방송을 들을 수 있고, 청취자가 몇 명이든 방송 내용은 같습니다.

비유를 기술 용어로 옮기면 이렇습니다. 토픽 통신은 단방향(one-way)입니다. 퍼블리셔는 메시지를 보낼 뿐 응답을 기다리지 않습니다. 또한 비동기(asynchronous)입니다. 보내는 쪽과 받는 쪽이 서로의 존재나 상태를 확인하지 않고 각자의 주기로 동작합니다. 마지막으로 익명(anonymous)입니다. 퍼블리셔와 서브스크라이버는 서로가 누구인지 모른 채 토픽 이름만으로 연결됩니다.

정의(1) 토픽(topic)

ROS 2에서 퍼블리셔 노드가 발행한 메시지를 해당 토픽을 구독하는 모든 서브스크라이버 노드에 단방향·비동기로 전달하는 통신 방식입니다.

다대다 연결과 느슨한 결합



하나의 토픽에는 퍼블리셔와 서브스크라이버가 여럿 연결될 수 있습니다. 그림 3-1의 구조를 봅시다.

그림 3-1. 하나의 퍼블리셔가 /number 토픽에 발행한 메시지를 두 서브스크라이버가 동시에 받는 구조

퍼블리셔가 /number 토픽에 숫자를 발행하면, 이 토픽을 구독하는 노드 A와 노드 B가 모두 같은 값을 받습니다. 반대로 여러 퍼블리셔가 하나의 토픽에 발행할 수도 있습니다. 이런 다대다(N:M) 연결이 가능한 이유는 노드들이 서로를 직접 참조하지 않고 토픽이라는 중간 통로만 바라보기 때문입니다.

이 성질을 **느슨한 결합(loose coupling)** 이라고 부릅니다. 서브스크라이버를 하나 추가하거나 제거해도 퍼블리셔의 코드는 한 줄도 바뀌지 않습니다. 카메라 노드가 영상을 발행하고 있을 때, 나중에 영상 저장 노드나 인공지능 인식 노드를 붙이고 싶으면 그저 같은 토픽을 구독하는 노드를 새로 실행하기만 하면 됩니다. 로봇 소프트웨어가 부품을 조립하듯 만들어질 수 있는 것은 바로 이 느슨한 결합 덕분입니다.

이름과 타입: 통신이 성립하는 두 가지 조건

토픽 통신이 성립하려면 두 가지가 정확히 일치해야 합니다.

- **토픽 이름:** 퍼블리셔와 서브스크라이버가 같은 이름(예: /number)을 사용해야 합니다. 이름이 한 글자라도 다르면 ROS 2는 오류를 내지 않고 그냥 서로 다른 두 개의 토픽을 만듭니다. 통신이 조용히 실패하는 가장 흔한 원인입니다.
- **메시지 타입:** 토픽마다 오가는 데이터의 형식이 정해져 있습니다. 이 장에서

사용할 `std_msgs/msg/Int64`는 64비트 정수 하나를 담는 표준 메시지 타입입니다. 퍼블리셔와 서브스크라이버의 타입이 다르면 통신이 이루어지지 않습니다.

메시지 타입의 구조는 언제든지 터미널에서 확인할 수 있습니다.

```
ros2 interface show std_msgs/msg/Int64
```

출력 앞부분에는 이 타입이 예제용 표준 메시지라는 안내 주석이 몇 줄 나오고, 마지막 줄에 실제 필드 정의가 있습니다.

```
int64 data
```

`Int64` 타입은 `data`라는 이름의 정수 필드 하나로 이루어져 있습니다. 코드에서 `msg.data = 5`처럼 이 필드에 값을 넣어 발행하게 됩니다.

어디에 쓰나: 끊이지 않고 이어지는 데이터

토픽은 지속적으로 이어지는 데이터에 적합합니다. 실제 로봇에서 토픽이 쓰이는 대표 사례는 다음과 같습니다.

- **센서 데이터:** 라이다는 초당 수십 번 거리 데이터를 `/scan` 토픽으로 발행하고, 카메라는 영상 프레임을 계속 발행합니다.
- **로봇 상태:** 배터리 잔량, 현재 위치, 관절 각도처럼 끊임없이 갱신되는 상태 정보를 발행합니다.
- **속도 명령:** 주행 제어 노드는 `/cmd_vel` 토픽으로 바퀴 속도 명령을 계속 내보냅니다.

공통점은 응답이 필요 없는 데이터가 계속 이어진다는 것입니다. 반대로 두 수를 더해 결과를 돌려받는 것처럼 요청 한 번에 응답 한 번이 필요한 일에는 4장에서 배울 서비스가, 목적지까지 이동하라는 명령처럼 오래 걸리고 중간 경과를 보고받아야 하는 일에는 5장에서 배울 액션이 적합합니다. 다섯 가지 통신 방식의 전체 구분은 1장의 표 1-3을 참고하세요.

3.2 퍼블리셔 만들기

이제 직접 만들어 봅시다. 1초마다 0, 1, 2 순서로 커지는 숫자를 `/number` 토픽

으로 발행하는 노드입니다. 2장에서 만든 my_py_pkg 패키지에 추가합니다.

Python 퍼블리셔 구현하기

my_py_pkg/my_py_pkg/number_publisher.py 파일을 생성하고 아래 코드를 작성합니다.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int64

class NumberPublisher(Node):
    def __init__(self):
        super().__init__("number_publisher")
        # 퍼블리셔 생성: 메시지 타입, 토픽명, QoS 깊이 지정
        self.publisher_ = self.create_publisher(Int64, "number", 10)
        # 1.0초마다 publish_number 콜백을 실행하는 타이머
        self.timer_ = self.create_timer(1.0, self.publish_number)
        self.counter_ = 0
        self.get_logger().info("숫자 퍼블리셔 시작")

    def publish_number(self):
        msg = Int64()
        msg.data = self.counter_
        self.publisher_.publish(msg)
        self.get_logger().info(f"발행: {msg.data}")
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = NumberPublisher()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
```

코드 해설: rclpy 노드의 뼈대

이 짧은 코드에 rclpy 프로그래밍의 뼈대가 모두 들어 있습니다. 구간별로 살펴봅시다.

초기화와 종료: main() 함수는 모든 rclpy 노드가 공유하는 공통 골격입니다.

- rclpy.init(): ROS 2 통신 기능을 초기화합니다. 어떤 노드보다 먼저 호출해야 합니다.

- rclpy.spin(node): 노드를 살아 있는 상태로 유지하면서, 등록된 콜백(타이머, 구독 등)이 호출되기를 기다렸다가 실행하는 이벤트 루프입니다. 이 함수는 Ctrl + C로 중단하기 전까지 반환되지 않습니다. spin이 없으면 노드는 만들어지자마자 프로그램이 끝나 버립니다.

- node.destroy_node()와 rclpy.shutdown(): 노드가 쓰던 자원을 정리하고 ROS 2 통신을 종료합니다.

- 마지막의 if __name__ 블록은 이 파일을 python3로 직접 실행할 때도 main()이 호출되게 하는 Python 관례입니다.

노드 클래스: NumberPublisher는 Node 클래스를 상속합니다. super().__init__("number_publisher")에서 지정한 문자열이 ROS 2 네트워크에서 이 노드를 식별하는 노드 이름이 됩니다. ros2 node list에 /number_publisher로 표시됩니다.

퍼블리셔 생성: create_publisher(Int64, "number", 10)은 세 가지를 지정합니다. 메시지 타입(Int64), 토픽 이름("number"), 그리고 QoS 깊이(10)입니다. 마지막 숫자 10은 KEEP_LAST 정책에서 최근 메시지 샘플을 최대 10개까지 대기열에 보관한다는 뜻인데, 자세한 의미는 3.5절에서 다룹니다. 지금은 특별한 이유가 없으면 10을 쓰는 것이 관례라고 기억하면 됩니다.

타이머와 콜백: create_timer(1.0, self.publish_number)는 1.0초마다 publish_number 메서드를 호출하는 타이머를 만듭니다. 이렇게 특정 사건이 일어

나면 호출하도록 등록해 두는 함수를 콜백(callback) 함수라고 합니다. 타이머 콜백 안에서 Int64 메시지 객체를 만들고 data 필드에 값을 채운 뒤 publish()로 발행합니다.

정의(2) 콜백(callback)

특정 사건(타이머 만료, 메시지 수신 등)이 발생했을 때 실행되도록 미리 등록해 두는 함수. rclpy에서는 spin이 사건을 감지해 등록된 콜백을 호출합니다.

setup.py 등록과 package.xml 확인

이 노드를 ros2 run으로 실행하려면 setup.py 파일의 console_scripts에 등록해야 합니다. 2장에서 위치만 봐 두었던 바로 그곳입니다.

```
entry_points={
    'console_scripts': [
        'number_publisher = my_py_pkg.number_publisher:main',
    ],
}
```

실행명 = 패키지.모듈:함수 형식입니다. 등호 왼쪽 number_publisher가 ros2 run에서 쓸 실행 이름, 오른쪽은 실행할 main 함수의 위치입니다.

코드에서 std_msgs를 import했으므로, package.xml에 의존성이 선언되어 있는지도 확인합니다. 2장에서 패키지를 만들 때 rclpy만 지정했으므로 아래 한 줄을 추가합니다.

```
<depend>std_msgs</depend>
```

빌드와 실행

워크스페이스 최상위에서 빌드하고, 환경을 다시 불러옵니다.

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
```

빌드 후 source install/setup.bash를 잊으면 방금 등록한 실행 파일을 셸이 찾지 못합니다. 이제 노드를 실행합니다.

```
ros2 run my_py_pkg number_publisher
```

다음과 같이 1초마다 로그가 출력되면 정상입니다.

```
[INFO] [1751690000.123456789] [number_publisher]: 숫자 퍼블리셔 시작
[INFO] [1751690001.123456789] [number_publisher]: 발행: 0
[INFO] [1751690002.123456789] [number_publisher]: 발행: 1
[INFO] [1751690003.123456789] [number_publisher]: 발행: 2
```

지금은 발행만 하고 있을 뿐 받는 노드가 없습니다. 그래도 퍼블리셔는 아무 불평 없이 계속 발행합니다. 단방향 통신이기 때문입니다.

3.3 서브스크라이버 만들기

이번에는 /number 토픽을 구독해서, 값이 도착할 때마다 로그로 출력하는 노드를 만듭니다.

Python 서브스크라이버 구현하기

my_py_pkg/my_py_pkg/number_subscriber.py 파일을 생성하고 아래 코드를 작성합니다.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int64

class NumberSubscriber(Node):
    def __init__(self):
        super().__init__("number_subscriber")
        # 서브스크라이버 생성: 타입, 토픽명, 콜백 함수, QoS 깊이 지정
        self.subscriber_ = self.create_subscription(
            Int64, "number", self.callback_number, 10)
        self.get_logger().info("숫자 서브스크라이버 시작")

    def callback_number(self, msg):
        self.get_logger().info(f"받음: {msg.data}")
```

```
def main(args=None):
    rclpy.init(args=args)
    node = NumberSubscriber()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()
```

코드 해설: 구독은 콜백으로 받습니다

구조는 퍼블리셔와 거의 같고, 딱 한 곳이 다릅니다. `create_subscription(Int64, "number", self.callback_number, 10)`은 네 가지를 지정합니다. 메시지 타입, 토픽 이름, 콜백 함수, QoS 깊이입니다. 타입과 토픽 이름이 퍼블리셔와 정확히 같다는 점에 주목하세요. 이 두 가지가 일치해야 통신이 성립합니다.

퍼블리셔의 타이머 콜백이 1초라는 시간 사건에 반응했다면, 서브스크라이버의 `callback_number`는 메시지 도착이라는 사건에 반응합니다. 메시지가 도착할 때마다 `spin`이 콜백을 호출하고, 수신된 메시지가 `msg` 인자로 전달됩니다. 서브스크라이버는 언제 메시지가 올지 모르므로, 능동적으로 확인하는 대신 콜백을 등록해 두고 기다리는 것입니다.

등록, 빌드, 실행

`setup.py`의 `console_scripts`에 한 줄을 추가합니다.

```
'number_subscriber = my_py_pkg.number_subscriber:main',
```

빌드하고 환경을 다시 불러옵니다.

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
```

이제 두 개의 터미널로 통신을 확인합니다. 터미널 1에서 퍼블리셔를 실행합니다.

```
ros2 run my_py_pkg number_publisher
```

터미널 2를 새로 열고 서브스크라이버를 실행합니다. 새 터미널에서는 `source ~/ros2_ws/install/setup.bash`를 잊지 않습니다(1장에서 `.bashrc`에 언더레이만 등록했다면 오버레이는 직접 소싱해야 합니다).

```
source ~/ros2_ws/install/setup.bash
ros2 run my_py_pkg number_subscriber
```

터미널 2에 다음과 같은 출력이 이어지면 첫 토픽 통신에 성공한 것입니다.

```
[INFO] [1751690101.234567890] [number_subscriber]: 숫자 서브스크라이버 시작
[INFO] [1751690102.234567890] [number_subscriber]: 받음: 7
[INFO] [1751690103.234567890] [number_subscriber]: 받음: 8
[INFO] [1751690104.234567890] [number_subscriber]: 받음: 9
```

받기 시작한 값이 0이 아니라는 점이 눈에 띕니다. 퍼블리셔가 먼저 실행되어 이미 발행한 값들은 지나갔고, 서브스크라이버는 구독을 시작한 시점 이후의 메시지만 받습니다. 라디오를 중간부터 듣는 것과 같습니다. 이것이 토픽의 기본 동작입니다.

터미널 3에서 `rqt_graph`를 실행하면 연결 구조를 그림으로 확인할 수 있습니다.

```
rqt_graph
```

`number_publisher` 노드에서 `/number` 토픽을 거쳐 `number_subscriber` 노드로 화살표가 이어져 있으면 정상입니다.

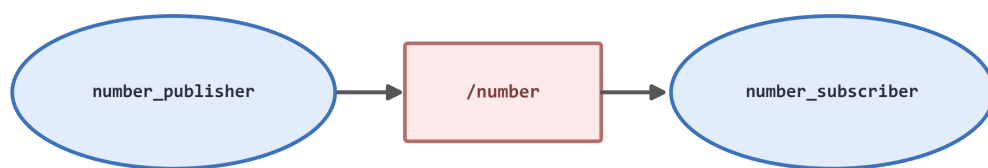


그림 3-2. `rqt_graph`로 확인하는 노드와 토픽의 연결 구조

3.4 C++로 구현하기

동일한 토픽 통신을 C++ 언어 기반의 `rclcpp` 라이브러리로 구현해 보겠습니다.

2장에서 만든 my_cpp_pkg 패키지를 사용합니다. Python 버전과 기능이 완전히 같으므로, 언어가 달라져도 ROS 2 API의 골격이 그대로 유지된다는 점을 확인하는데 집중하세요.

C++ 퍼블리셔 작성

my_cpp_pkg/src/number_publisher.cpp 파일을 만들고 아래 코드를 작성합니다.

```
#include <chrono>
#include <cstdint>
#include <functional>
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/int64.hpp"

class NumberPublisher : public rclcpp::Node {
public:
    NumberPublisher() : Node("number_publisher"), counter_(0) {
        // 퍼블리셔 생성: 템플릿 인자로 타입, 인자로 토픽명과 QoS 깊이 지정
        publisher_ =
this->create_publisher<std_msgs::msg::Int64>("number", 10);
        // 1초마다 publishNumber 콜백을 실행하는 타이머
        timer_ = this->create_wall_timer(
            std::chrono::seconds(1),
            std::bind(&NumberPublisher::publishNumber, this));
        RCLCPP_INFO(this->get_logger(), "C++ 숫자 퍼블리셔 시작");
    }

private:
    void publishNumber() {
        auto msg = std_msgs::msg::Int64();
        msg.data = counter_;
        publisher_->publish(msg);
        RCLCPP_INFO(this->get_logger(), "발행: %ld", msg.data);
        counter_++;
    }

    rclcpp::Publisher<std_msgs::msg::Int64>::SharedPtr publisher_;
```

```

    rclcpp::TimerBase::SharedPtr timer_;
    int64_t counter_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<NumberPublisher>());
    rclcpp::shutdown();
    return 0;
}

```

C++ 서브스크라이버 작성

my_cpp_pkg/src/number_subscriber.cpp 파일을 만들고 아래 코드를 작성합니다.

```

#include <functional>
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/int64.hpp"
using std::placeholders::_1;

class NumberSubscriber : public rclcpp::Node {
public:
    NumberSubscriber() : Node("number_subscriber") {
        // 서브스크라이버 생성: 토픽명, QoS 깊이, 콜백 함수(바인딩) 지정
        subscriber_ = this->create_subscription<std_msgs::msg::Int64>(
            "number", 10,
            std::bind(&NumberSubscriber::callbackNumber, this, _1));
        RCLCPP_INFO(this->get_logger(), "C++ 숫자 서브스크라이버 시작");
    }

private:
    void callbackNumber(const std_msgs::msg::Int64::SharedPtr msg) {
        RCLCPP_INFO(this->get_logger(), "받음: %ld", msg->data);
    }

    rclcpp::Subscription<std_msgs::msg::Int64>::SharedPtr subscriber_;
};

```

```
int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<NumberSubscriber>());
    rclcpp::shutdown();
    return 0;
}
```

코드 해설: Python과의 대응 관계

C++ 문법이 낯설어도 걱정할 필요 없습니다. 구조를 Python과 나란히 놓고 보면 하는 일이 정확히 같습니다.

개념	Python(rclpy)	C++(rclcpp)
초기화	rclpy.init(args=args)	rclcpp::init(argc, argv)
노드 클래스	class X(Node)	class X : public rclcpp::Node
퍼블리셔 생성	create_publisher(Int64, "number", 10)	create_publisher<std_msgs::msg::Int64>("number", 10)
서브스크라이버 생성	create_subscription(타입, 이름, 콜백, 10)	create_subscription<타입>(이름, 10, 콜백)
타이머 생성	create_timer(1.0, 콜백)	create_wall_timer(std::chrono::seconds(1), 콜백)
콜백 연결	메서드를 그대로 전달	std::bind(&클래스::메서드, this, ...)
이벤트 루프	rclpy.spin(node)	rclcpp::spin(노드 포인터)
로그 출력	self.get_logger().info(...)	RCLCPP_INFO(this->get_logger(), ...)
종료	rclpy.shutdown()	rclcpp::shutdown()

표 3-1. 토픽 통신 API의 Python·C++ 대응 관계

언어 문법 차이에서 오는 특징만 짚어 둡니다. 첫째, C++은 메시지 타입을 `create_publisher<std_msgs::msg::Int64>`처럼 템플릿 인자로 지정합니다. 둘째, 클래스 메서드를 콜백으로 넘길 때 `std::bind(&클래스::메서드, this)`로 이 객체의 이 메서드임을 명시해야 하며, 콜백이 인자를 받으면 `_1` 같은 자리 표시자를 함께 씁니다. 셋째, 퍼블리셔·타이머 등은 `SharedPtr`(공유 포인터) 멤버 변수로 보관합니다. 보관하지 않으면 생성 직후 객체가 소멸되어 통신이 되지 않습니다.

CMakeLists.txt와 package.xml 설정

Python의 `setup.py` 등록에 해당하는 작업을 C++에서는 `CMakeLists.txt`가 담당

합니다. my_cpp_pkg/CMakeLists.txt에 아래 내용을 추가합니다.

```
find_package(rclcpp REQUIRED)
find_package(std_msgs REQUIRED)

add_executable(number_publisher src/number_publisher.cpp)
ament_target_dependencies(number_publisher rclcpp std_msgs)

add_executable(number_subscriber src/number_subscriber.cpp)
ament_target_dependencies(number_subscriber rclcpp std_msgs)

install(TARGETS number_publisher number_subscriber
        DESTINATION lib/${PROJECT_NAME})
```

세 단계로 이루어집니다. find_package로 의존 패키지를 찾고, add_executable로 소스 파일을 실행 파일로 빌드하도록 지정하고, ament_target_dependencies로 실행 파일에 의존성을 연결합니다. 마지막 install(TARGETS ...)은 빌드된 실행 파일을 ros2 run이 찾는 위치(lib/패키지명)에 설치하라는 규칙입니다. 이 규칙이 빠지면 빌드는 성공해도 ros2 run이 실행 파일을 찾지 못합니다.

package.xml의 의존성 영역에도 아래 한 줄을 추가합니다(rclcpp는 2장에서 패키지 생성 시 이미 등록되었습니다).

```
<depend>std_msgs</depend>
```

빌드와 실행: 언어를 넘나드는 통신

빌드하고 환경을 다시 불러옵니다.

```
cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
source install/setup.bash
```

이왕이면 언어를 섞어서 실행해 봅시다. 터미널 1에서 C++ 퍼블리셔를, 터미널 2에서 Python 서브스크라이버를 실행합니다.

```
# 터미널 1
ros2 run my_cpp_pkg number_publisher

# 터미널 2
```

```
ros2 run my_py_pkg number_subscriber
```

Python 서브스크라이버가 C++ 퍼블리셔의 숫자를 그대로 받아 출력합니다. 1장에서 talker(C++)와 listener(Python)로 확인했던 언어 간 통신을, 이번에는 직접 만든 노드로 재현한 것입니다. 토픽 이름과 메시지 타입만 일치하면 구현 언어는 통신에 아무 영향을 주지 않습니다.

주의: 같은 이름의 노드를 동시에 띄우지 않기

Python판과 C++판 퍼블리셔는 노드 이름이 number_publisher로 같습니다. 두 노드를 동시에 실행하면 이름 충돌 경고가 뜨고 디버깅 도구가 혼란스러워집니다. 이 장의 실습에서는 한 번에 한 쪽만 실행합니다.

3.5 QoS 설정

깊이 10의 의미

지금까지 create_publisher와 create_subscription의 마지막 인자로 아무 설명 없이 10을 넘겼습니다. 이 숫자의 정체를 알아보시다.

이 값은 **QoS 깊이(depth)**로, 전달을 기다리는 메시지를 보관하는 대기열(queue)의 크기입니다. 수신 측이 메시지를 처리하는 속도보다 발행 속도가 빠르면 메시지가 대기열에 쌓이는데, 깊이가 10이면 최근 10개까지만 보관하고 그보다 오래된 것은 버립니다. 1장에서 소개한 QoS의 이력(history) 설정 중 최근 N개 유지(KEEP_LAST) 정책에 해당합니다.

깊이 10은 ROS 2 예제 전반에서 쓰이는 무난한 기본값입니다. 초당 수백 건씩 발행되는 고속 센서라면 더 크게, 최신 값 하나만 의미 있는 데이터라면 1로 줄이는 식으로 조절할 수 있습니다.

신뢰성과 지속성: 프로파일로 고른다

1장에서 QoS는 통신 품질을 고르는 설정이라고 소개했습니다. 깊이 외에 실무에서 자주 만나는 두 가지만 짚어 둡니다.

설정	선택지	의미
신뢰성(reliability)	RELIABLE / BEST_EFFORT	유실 시 재전송해 반드시 전달 / 유실을 감수하고 빠르게 전달
지속성(durability)	VOLATILE / TRANSIENT_LOCAL	지금 오가는 것만 전달 / 늦게 구독한 쪽에도 마지막 메시지 전달

표 3-2. 자주 쓰는 QoS 설정 두 가지

기본값은 RELIABLE과 VOLATILE입니다. 숫자 10만 넘겼을 때는 깊이 10에 기본 프로파일이 적용된 것입니다. 세부 설정이 필요하다면 QoSProfile 객체를 만들어 넘깁니다. 다음은 신뢰성을 BEST_EFFORT로 바꾼 서브스크라이버의 핵심 부분입니다.

```
# 핵심 부분만 발췌: QoSProfile로 세부 설정 지정
from rclpy.qos import QoSProfile, ReliabilityPolicy

qos = QoSProfile(depth=10, reliability=ReliabilityPolicy.BEST_EFFORT)
self.subscriber_ = self.create_subscription(
    Int64, "number", self.callback_number, qos)
```

주의할 점은 퍼블리셔와 서브스크라이버의 QoS가 호환되어야 통신이 성립한다는 것입니다. 대표적인 비호환 조합은 BEST_EFFORT 퍼블리셔에 RELIABLE 서브스크라이버를 연결하는 경우입니다. 구독자는 반드시 전달하라고 요구하는데 발행자가 이를 보장하지 않기 때문입니다. 이때 노드는 오류로 멈추는 대신 incompatible QoS 경고 로그를 남기고 메시지를 받지 못합니다. 통신이 안 될 때 점검할 항목이 토픽 이름, 타입에 이어 하나 더 생긴 셈입니다.

라이다·카메라 같은 센서 데이터에는 BEST_EFFORT 기반의 센서 데이터 프로파일(rclpy.qos.qos_profile_sensor_data)이 관례적으로 쓰입니다. 실제 센서 토픽을 구독할 2권 프로젝트에서 다시 만나게 됩니다. QoS의 나머지 세부 설정은 입문 범위를 넘으므로, 이 장에서는 깊이의 의미와 호환 규칙까지만 이해하면 충분합니다.

3.6 ros2 topic 명령으로 살펴보기

코드를 한 줄도 쓰지 않고 토픽을 들여다보는 CLI 명령어들입니다. 토픽 문제의 대부분은 이 명령들로 원인을 찾을 수 있습니다. 터미널 1에서 ros2 run

my_py_pkg number_publisher를 실행해 둔 상태에서, 터미널 2로 하나씩 확인해 봅시다.

토픽 목록 확인

현재 활성화된 모든 토픽 목록을 보여줍니다.

```
ros2 topic list  
  
/number  
/parameter_events  
/rosout
```

/number가 보이면 퍼블리셔가 정상적으로 토픽을 만든 것입니다. /parameter_events와 /rosout은 모든 노드가 자동으로 만드는 시스템 토픽이니 신경 쓰지 않아도 됩니다. -t 옵션을 붙이면 각 토픽의 타입을 함께 보여줍니다.

```
ros2 topic list -t  
  
/number [std_msgs/msg/Int64]  
/parameter_events [rcl_interfaces/msg/ParameterEvent]  
/rosout [rcl_interfaces/msg/Log]
```

토픽 타입 확인

특정 토픽의 메시지 타입만 확인합니다.

```
ros2 topic type /number  
  
std_msgs/msg/Int64
```

퍼블리셔와 서브스크라이버 양쪽에서 이 명령을 확인하면 타입 불일치를 바로 찾아낼 수 있습니다.

토픽 상세 정보 확인

토픽의 타입과 함께 연결된 퍼블리셔·서브스크라이버 수를 보여줍니다.

```
ros2 topic info /number  
  
Type: std_msgs/msg/Int64
```

```
Publisher count: 1
Subscription count: 0
```

Subscription count: 0은 아직 구독자가 없다는 뜻입니다. 통신이 안 될 때 가장 먼저 볼 명령입니다. 발행자 수가 0이면 퍼블리셔 쪽 문제, 구독자 수가 0이면 서브스크라이버 쪽 문제로 범위를 좁힐 수 있습니다.

토픽 내용 엿보기

토픽에 발행되는 메시지를 화면에 그대로 출력합니다.

```
ros2 topic echo /number
data: 14
---
data: 15
---
```

서브스크라이버 노드를 만들지 않고도 토픽 내용을 확인할 수 있으므로, 퍼블리셔가 제대로 발행하고 있는지 검증하는 데 가장 자주 쓰입니다.

발행 주기 측정

토픽의 발행 주기를 헤르츠(Hz) 단위로 측정합니다.

```
ros2 topic hz /number
average rate: 1.000
  min: 0.999s max: 1.001s std dev: 0.00055s window: 10
```

이 퍼블리셔는 1초마다 발행하므로 약 1.0 Hz가 나오면 정상입니다. 센서가 기대한 주기로 데이터를 내보내는지 점검할 때 유용합니다.

CLI로 직접 발행하기

코드 없이 터미널에서 직접 메시지를 발행할 수도 있습니다. 타입 뒤에 메시지 내용을 YAML 형식 문자열로 전달합니다.

```
ros2 topic pub /number std_msgs/msg/Int64 "{data: 42}"
```

이 명령은 기본적으로 1 Hz로 반복 발행합니다. 한 번만 발행하려면 `--once` 옵션

션을 붙입니다. 서브스크라이버만 만들어 둔 상태에서 퍼블리셔 없이 동작을 시험할 때 매우 유용합니다. 터미널 1에서 `ros2 run my_py_pkg number_subscriber`를 실행해 두고 터미널 2에서 위 명령을 실행하면, 서브스크라이버가 받음: 42를 출력합니다. 직접 만든 구독자 코드가 CLI 퍼블리셔와도 아무 문제 없이 연결되는 것입니다.

그래프로 시각화

`rqt_graph`는 노드와 토픽의 연결 관계를 그래프로 보여줍니다. 시스템이 복잡해질수록 이름 오타나 끊어진 연결을 눈으로 찾는 데 가장 빠른 도구입니다. 화면 좌측 상단의 새로 고침 버튼을 누르면 최신 상태가 반영됩니다.

워크드 예제: 구독하며 다시 발행하는 노드

실전에서 토픽은 수신 → 가공 → 재발행으로 이어지는 중간 처리 노드에 가장 많이 쓰입니다. `/number`를 구독해 2배로 만들어 `/number_x2`로 발행하고, 값이 문턱값을 넘으면 경고를 출력하는 노드를 만들어 봅시다.

`my_py_pkg/my_py_pkg/number_doubler.py`를 만들고 아래와 같이 구현합니다.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int64

WARN_THRESHOLD = 20 # 이 값 이상이면 경고 출력

class NumberDoubler(Node):
    def __init__(self):
        super().__init__("number_doubler")
        self.publisher_ = self.create_publisher(Int64, "number_x2", 10)
        self.subscriber_ = self.create_subscription(
            Int64, "number", self.callback_number, 10)
        self.get_logger().info("숫자 2배 노드 시작")

    def callback_number(self, msg):
```

```

        out = Int64()
        out.data = msg.data * 2
        self.publisher_.publish(out)
        if out.data >= WARN_THRESHOLD:
            self.get_logger().warn(f"{msg.data} → {out.data} (문턱값 초과)")
        else:
            self.get_logger().info(f"{msg.data} → {out.data}")

def main(args=None):
    rclpy.init(args=args)
    node = NumberDoubler()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

setup.py에 'number_doubler = my_py_pkg.number_doubler:main'을 등록하고 빌드한 뒤, 세 개의 터미널에서 실행합니다.

```

# 터미널 1: 원본 숫자 발행
ros2 run my_py_pkg number_publisher

# 터미널 2: 구독 + 가공 + 재발행
ros2 run my_py_pkg number_doubler

# 터미널 3: 결과 확인
ros2 topic echo /number_x2

```

터미널 3에 2배 값이 이어지고, 터미널 2에서는 값이 20 이상이 되는 순간부터 [WARN] 로그가 노란색으로 출력됩니다. 한 노드가 서브스크라이버이면서 동시에 퍼블리셔인 이 구조가 실제 로봇 소프트웨어의 기본 패턴입니다. 센서 값을 받아 판단한 뒤 명령을 내보내는 모든 노드가 이 모양을 하고 있습니다.

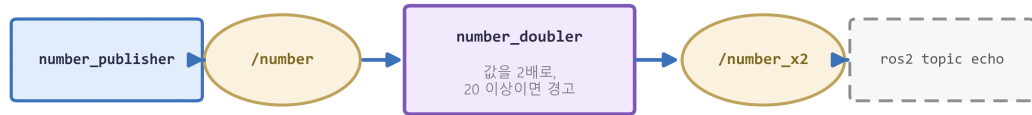


그림 3-3. number_publisher에서 number_doubler를 거쳐 /number_x2로 이어지는 수신·가공·재발행 구조

실제 로봇에서의 토픽: /scan과 /cmd_vel

이 장에서 익힌 패턴이 실제 로봇에서 어떻게 확장되는지 잠깐 살펴봅니다. 이동 로봇의 자율주행 뼈대는 토픽 두 개로 요약됩니다.

- /scan(sensor_msgs/msg/LaserScan): 라이다가 발행하는 거리 데이터입니다. 로봇 주변 여러 방향까지의 거리가 배열로 담겨 초당 수 회에서 수십 회 발행됩니다.
- /cmd_vel(geometry_msgs/msg/Twist): 로봇의 목표 속도 명령입니다. 직진 속도와 회전 속도를 담아 주행 제어기로 보냅니다.

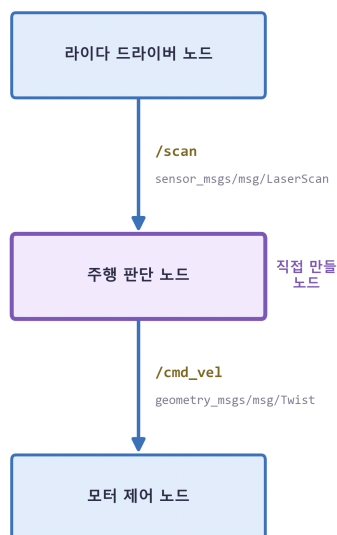


그림 3-4. 라이다 드라이버에서 주행 판단 노드를 거쳐 모터 제어로 이어지는 토픽 연결

가운데의 주행 판단 노드를 보세요. /scan을 구독하고 /cmd_vel을 발행합니다. 방금 워크드 예제에서 만든 number_doubler와 구조가 정확히 같습니다. 다른 점은 Int64 대신 필드가 여러 개인 메시지 타입을 쓴다는 것입니다. 타입 구조가 궁금하다면 지금 바로 확인해 볼 수 있습니다.

```
ros2 interface show geometry_msgs/msg/Twist
```

이 구조를 실제 로봇과 시뮬레이터에서 완성하는 것은 2권의 프로젝트에서 다룹니다. 지금 단계에서는 이 장의 발행/구독 패턴이 규모만 커질 뿐 그대로 쓰인다는 것을 확인하는 것으로 충분합니다.

코드 해설 · 실행 확인 · 버전 체크

- **코드 해설 포인트:** create_publisher/create_subscription의 인자 구성(타입·토픽명·콜백·QoS 깊이), create_timer와 콜백의 관계, rclpy.spin이 콜백을 구동하는 구조, Python의 console_scripts와 C++의 add_executable + install(TARGETS ...) 등록 흐름을 이해합니다.

- **실행 확인 포인트:** 터미널 두 개로 퍼블리셔·서브스크라이버를 실행해 숫자가 전달되는지 확인하고, ros2 topic list/info/echo/hz로 토픽 상태를 검증합니다. ros2 topic info /number의 Publisher/Subscription count가 기대와 일치하는지, rqt_graph에 노드-토픽 연결이 그려지는지 확인합니다.

- **버전/환경 체크:** ROS 2 Jazzy(Ubuntu 24.04, Python 3.12) 기준입니다. 토픽 생성 API(create_publisher, create_subscription)는 Foxy·Humble 등 이전 배포판과 사실상 동일하므로 인터넷의 이전 배포판 예제도 대부분 그대로 통용됩니다. 다만 빌드 시 Python 3.12의 setuptools 경고가 출력되는 점(2장 참조)과, Jazzy의 기본 RMW가 Fast DDS라는 점을 기억하세요.

트러블슈팅

증상	원인	확인	해결
ros2 run에서 No executable found	setup.py의 console_scripts 미등록 또는 재빌드 안 함	ros2 pkg executables my_py_pkg	등록 후 colcon build + source install/setup.bash
Package 'my_py_pkg' not found	오버레이 미소싱	ros2 pkg list 출력에서 my_ 확인	해당 터미널에서 source ~/ros2_ws/install/setup.bash
코드 수정했는데 동작이 그대로	빌드하지 않고 실행	ls -l install/ 시각 확인	colcon build 후 다시 소싱·실행
토픽 목록에 /number 가 없음	퍼블리셔 노드가 실행되지 않음	ros2 node list	퍼블리셔 실행 여부·오류 로그 확인
퍼블리셔는 도는데 구독자가 못 받음	토픽 이름 불일치(오타)	ros2 topic list로 유사 이름 두 개가 있는지 확인	양쪽 코드의 토픽 이름을 통일
이름은 같은데 여전히 못 받음	메시지 타입 불일치	ros2 topic info /number	양쪽 코드의 타입을 통일
타입·이름 모두 같은데 못 받음	QoS 비호환(BEST_EFFORT 발행 + RELIABLE 구독)	실행 로그에서 incompatible QoS 경고 검색	양쪽 QoS 프로파일을 호환 조합으로 조정
다른 PC(또는 동료)와 통신 안 됨	ROS_DOMAIN_ID 불일치	양쪽에서 echo \$ROS_DOMAIN_ID	같은 번호로 통일(1장 참조)
No module named 'std_msgs'	의존성 누락	package.xml 열어 확인	<depend>std_msgs</depend> 추가 후 재빌드
C++ 빌드는 성공했는데 ros2 run이 못 찾음	CMakeLists.txt의 install(TARGETS ...) 누락	ls install/my_cpp_pkg/lib/my_cpp_pkg/	install 규칙 추가 후 재빌드
C++ 링크 오류(undefined reference)	ament_target_dependencies 누락	CMakeLists.txt 확인	실행 파일에 rclcpp std_msgs 의존성 연결

표 3-3. 토픽 구현 시 자주 발생하는 오류와 해결 방법

통신이 안 될 때는 다음 순서로 점검합니다.

1. ros2 node list로 노드가 살아 있는지 확인합니다.
2. ros2 topic list로 토픽 이름이 하나로 모였는지 확인합니다.
3. ros2 topic info로 타입과 연결 수가 맞는지 확인합니다.
4. 실행 로그에 QoS 경고가 없는지 확인합니다.
5. ROS_DOMAIN_ID가 서로 같은지 확인합니다.

이 순서로 확인하면 토픽 문제의 대부분이 해결됩니다.

정리

- 토픽은 이름 붙은 통로를 통해 퍼블리셔가 발행한 메시지를 모든 서브스크라이버에게 전달하는 단방향·비동기 통신 방식이며, 센서 데이터나 속도 명령처럼 지속적으로 이어지는 데이터에 적합합니다.

- 토픽 통신이 성립하려면 토픽 이름과 메시지 타입이 양쪽에서 정확히 일치해야 하고, QoS 설정이 호환되어야 합니다.
- rclpy 노드는 init → 노드 생성 → spin → shutdown의 공통 골격을 가지며, 퍼블리셔는 타이머 콜백에서 publish()를 호출하고 서브스크라이버는 수신 콜백으로 메시지를 받습니다.
- Python과 C++의 토픽 API는 일대일로 대응하며, 노드 등록 위치(setup.py vs CMakeLists.txt)만 다릅니다.
- ros2 topic CLI와 rqt_graph를 이용하면 코드 수정 없이 토픽의 이름·타입·연결 상태·발행 주기를 진단할 수 있습니다.

핵심 용어

- **토픽(topic)**: 퍼블리셔가 발행한 메시지를 구독자 모두에게 단방향으로 전달하는 이름 붙은 통신 통로
- **퍼블리셔(publisher)**: 토픽에 메시지를 발행하는 주체
- **서브스크라이버(subscriber)**: 토픽을 구독해 메시지를 수신하는 주체
- **메시지(message)**: 토픽으로 주고받는 데이터의 단위. .msg 파일로 형식이 정의됩니다
- **콜백(callback)**: 타이머 만료나 메시지 수신 같은 사건이 발생했을 때 실행되도록 등록해 두는 함수
- **spin**: 노드를 살아 있게 유지하며 콜백을 구동하는 이벤트 루프
- **QoS 깊이(depth)**: 전달 대기 중인 메시지를 보관하는 대기열의 크기
- **느슨한 결합(loose coupling)**: 노드들이 서로를 직접 참조하지 않고 토픽 이름만으로 연결되는 성질

연습문제

1. [개념 확인] 토픽 통신에서 퍼블리셔가 서브스크라이버의 존재를 몰라도 되는 이유를 느슨한 결합이라는 용어를 사용해 설명하시오.

2. [개념 확인] `create_publisher(Int64, "number", 10)`에서 세 번째 인자 10의 의미를 한 문장으로 설명하시오.
3. [코드 수정] `number_publisher`의 발행 주기를 0.5초로 바꾸고, `ros2 topic hz /number`로 약 2.0 Hz가 측정되는지 확인하시오.
4. [코드 수정] `number_publisher`가 숫자를 1씩이 아니라 5씩 증가시키며 발행하도록 수정하시오.
5. [실행 결과 예상] 퍼블리셔를 30초쯤 먼저 실행한 뒤 서브스크라이버를 실행하면, 서브스크라이버가 처음 받는 값은 0인가 아닌가? 그 이유는 무엇인가?
6. [CLI 실습] 서브스크라이버만 실행한 상태에서 `ros2 topic pub`를 이용해 `/number`에 100을 한 번만 발행하고, 구독자가 받는지 확인하시오.
7. [오류 원인 찾기] 퍼블리셔는 "number", 서브스크라이버는 "Number" 토픽을 사용하도록 작성했습니다. 실행하면 오류가 발생하는가? 통신은 되는가? 어떤 명령으로 이 문제를 발견할 수 있는가?
8. [Python·C++ 비교] Python의 `console_scripts` 등록에 대응하는 C++ 쪽 설정 두 가지(`CMakeLists.txt` 기준)를 쓰시오.
9. [통신 방식 선택] 다음 상황에 토픽이 적합한지 판단하고 근거를 쓰시오. (가) 카메라 영상 프레임 전달 (나) 지도를 파일로 저장하라는 일회성 요청 (다) 로봇 배터리 잔량의 주기적 보고
10. [응용 구현] `/number`를 구독해 지금까지 받은 값들의 합계를 `/number_sum` 토픽으로 발행하는 노드를 작성하시오.

연습문제 해설

- 1번 퍼블리셔는 토픽에 발행할 뿐 수신자를 참조하지 않으므로, 구독자를 추가·제거해도 퍼블리셔 코드가 바뀌지 않습니다. 이렇게 상대를 직접 알지 못한 채 토픽 이름으로만 연결되는 성질이 느슨한 결합입니다.
- 2번 전달 대기 메시지를 최대 10개까지 보관하는 QoS 대기열의 깊이입니다.
- 3번 `create_timer(0.5, self.publish_number)`로 수정 후 재빌드. `ros2 topic hz`

/number가 약 2.0을 보이면 성공.

- **4번** `self.counter_ += 1`을 `self.counter_ += 5`로 수정합니다.
 - **5번** 0이 아닙니다. 토픽은 구독 시작 이후 발행된 메시지만 전달하므로, 먼저 발행된 값들은 받을 수 없습니다(기본 QoS의 VOLATILE 지속성).
 - **6번** `ros2 topic pub --once /number std_msgs/msg/Int64 "{data: 100}"` 실행
→ 구독자가 받음: 100을 출력합니다.
 - **7번** 오류는 발생하지 않고 /number와 /Number 두 개의 토픽이 따로 만들어져 통신만 조용히 실패합니다. `ros2 topic list`에서 비슷한 이름 두 개를 발견하거나, `ros2 topic info`에서 한쪽 count가 0인 것으로 확인할 수 있습니다.
 - **8번** `add_executable`(실행명 소스.cpp)로 실행 파일을 정의하고, `install(TARGETS 실행명 DESTINATION lib/${PROJECT_NAME})`로 `ros2 run`이 찾는 위치에 설치합니다.
 - **9번** (가) 토픽. 연속으로 이어지는 스트림입니다. (나) 부적합. 일회성 요청·응답이므로 서비스(4장)가 맞다. (다) 토픽. 주기적으로 갱신되는 상태 보고다.
 - **10번** 워크드 예제의 NumberDoubler 구조에서 콜백을 누적 합산으로 바꾼다. 멤버 변수 `self.sum_ = 0`을 두고 콜백에서 `self.sum_ += msg.data` 후 발행합니다.
- 다음 장 예고** 토픽은 한 방향으로 보낼 뿐 응답을 받을 수 없습니다. 4장에서는 질문하고 답을 받는 양방향 통신, 서비스를 다룹니다. 두 수를 더해 주는 서버를 만들며 요청-응답 모델을 익힙니다.

참고 자료

- ROS 2 Jazzy 공식 문서, Understanding topics:
<https://docs.ros.org/en/jazzy/Concepts/Basic/About-Topics.html>
- ROS 2 Jazzy 공식 문서, Writing a simple publisher and subscriber
(Python):
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Py-Publisher-And-Subscriber.html>

- ROS 2 Jazzy 공식 문서, Writing a simple publisher and subscriber (C++):
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Writing-A-Simple-Cpp-Publisher-And-Subscriber.html>
- ROS 2 Jazzy 공식 문서, About QoS settings:
<https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html>
- 표윤석·임태훈, 『ROS 2로 시작하는 로봇 프로그래밍』, 루비페이퍼

마무리 점검

- ☐ 토픽의 단방향·비동기·다대다 특성을 설명할 수 있습니다.
- ☐ Python으로 퍼블리셔와 서브스크라이버를 작성하고 두 터미널에서 통신을 확인했습니다.
- ☐ C++로 같은 기능을 구현하고, Python 노드와 언어를 넘나드는 통신을 확인했습니다.
- ☐ console_scripts(Python)와 add_executable + install(C++)의 역할을 구분해 말할 수 있습니다.
- ☐ QoS 깊이 값의 의미와 QoS 비호환이 통신에 미치는 영향을 설명할 수 있습니다.
- ☐ `ros2 topic list/type/info/echo/hz/pub` 명령으로 토픽을 진단할 수 있습니다.
- ☐ 통신이 안 될 때의 점검 순서(노드 → 이름 → 타입 → QoS → 도메인)를 압니다.

04

서비스

요청과 응답으로 동작하는 통신.
서버와 클라이언트를 만들고
ros2 service 명령으로 확인한다.

이 장에서 다루는 내용

- * 서비스와 토픽의 통신 방식 차이 및 서비스의 특징
- * 서비스 서버(Service Server)와 서비스 클라이언트(Service Client)의 역할과 구현 방법
- * C++로 서비스 통신 구현하기
- * `ros2 service` 명령어를 활용한 서비스 디버깅 방법

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 서비스(service)의 요청-응답(request/response) 모델을 토픽과 구분하여 설명할 수 있습니다.
2. `rclpy/rclcpp`로 서비스 서버·클라이언트를 직접 작성할 수 있습니다.
3. 비동기 호출(`call_async`)의 흐름과 콜백 처리 방식을 이해합니다.
4. `ros2 service` CLI 명령어로 서비스를 호출하고 점검할 수 있습니다.

4.1 서비스 통신 모델

서비스란 — 한 번의 질문과 답

토픽이 라디오 방송(broadcast)처럼 정보를 일방적으로 내보내는 단방향 스트림 방식이라면, 서비스는 일대일로 통화하며 질문과 대답을 나누는 양방향 통신 방식입니다.

서비스 통신 모델은 정보를 제공하는 **서버(server)**와 정보를 요청하는 **클라이언트(client)**로 나뉩니다. 클라이언트가 특정 연산이나 처리를 요청(request)하면, 서버는 그 요청을 처리한 뒤 결과를 응답(response)으로 돌려줍니다. 이러한 구조를 요청-응답 모델이라고 부르며, 요청 1회당 응답 1회가 매칭됩니다.

```
[클라이언트] --요청(request)--> [서버]
[클라이언트] <---응답(response)-- [서버]
```

토픽과 서비스는 다음과 같은 명확한 차이점을 가집니다.

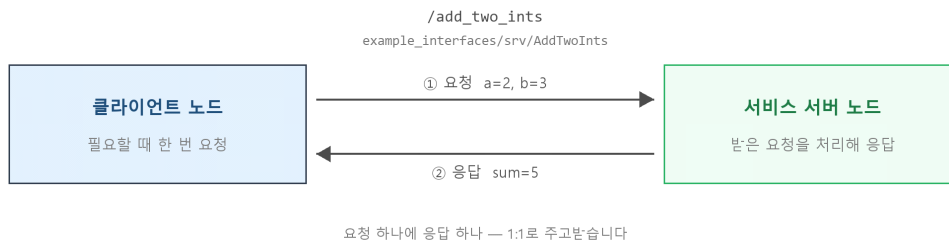


그림 4-1. 클라이언트가 요청하고 서버가 응답하는 서비스 통신 구조

구분	토픽(topic)	서비스(service)
방향	단방향	양방향(요청·응답)
횟수	지속적인 데이터 흐름	호출할 때 1회
용도	센서 데이터 스트림, 속도 명령 등 지속 흐름	연산 요청, 상태 변경, 일회성 설정 등

표 4-1. 토픽과 서비스의 비교

언제 서비스를 쓰나

서비스는 “두 값을 더해줘”, “현재 로봇의 위치가 표시된 지도를 저장해줘”, “로

봇의 LED를 켜거나 꺼줘"와 같이 요청한 일의 수행 여부나 결과값을 즉각적으로 확인해야 할 때 사용합니다.

다만, 로봇에게 "저 방으로 이동해라"와 같이 작업 수행에 오랜 시간이 걸리고 중간 과정을 모니터링해야 하는 경우에는 5장에서 배울 액션(action) 을 사용하는 것이 적합합니다.

서비스 타입 - .srv 파일

서비스 통신을 하기 위해서는 어떤 데이터를 요청하고 응답할지 형식을 정의해야 합니다. 이 형식은 .srv 파일에 선언합니다. .srv 파일은 세 개의 하이픈(---)을 구분선으로 사용하여 윗부분은 요청(request) 필드, 아랫부분은 응답(response) 필드를 정의합니다.

이번 장 실습에서 사용할 기본 예제 인터페이스인 example_interfaces/srv/AddTwoInts는 다음과 같이 구성되어 있습니다.

```
int64 a          # 요청(request) 데이터 1
int64 b          # 요청(request) 데이터 2
---
int64 sum        # 응답(response) 데이터
```

이 인터페이스 구조는 터미널 창에 다음 명령어를 입력하여 언제든지 확인할 수 있습니다.

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

정의(1) 서비스(service)

ROS 2에서 클라이언트 노드가 요청을 보내고 서버 노드가 이에 대한 응답을 반환하는 일대일 양방향 통신 방식입니다.

4.2 서비스 서버와 클라이언트

파이썬(Python)의 `rclpy` 라이브러리를 사용해 두 정수를 더해주는 서비스 서버와 클라이언트를 구현해 보겠습니다.

Python 서비스 서버 구현하기

`my_py_pkg/my_py_pkg/add_two_ints_server.py` 파일을 생성하고 아래 코드를 작성합니다.

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsServer(Node):
    def __init__(self):
        super().__init__("add_two_ints_server")
        # 서비스 서버 생성: 타입, 서비스명, 콜백 함수 지정
        self.server_ = self.create_service(
            AddTwoInts, "add_two_ints", self.callback_add)
        self.get_logger().info("서비스 서버 준비 완료")

    def callback_add(self, request, response):
        response.sum = request.a + request.b
        self.get_logger().info(f"{request.a} + {request.b} = {response.sum}")
        return response

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

이 노드를 실행하기 위해 `setup.py` 파일의 `console_scripts`에 아래와 같이 등록합니다.

```
entry_points={
    'console_scripts': [
        'add_two_ints_server = my_py_pkg.add_two_ints_server:main',
    ],
}
```

```
},
```

이후 패키지를 빌드하고 실행해 둡니다.

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
ros2 run my_py_pkg add_two_ints_server
```

Python 서비스 클라이언트 구현하기

서버가 준비되면 요청을 보낼 클라이언트 노드가 필요합니다.

`my_py_pkg/my_py_pkg/add_two_ints_client.py` 파일을 생성하고 작성합니다.

```
import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

class AddTwoIntsClient(Node):
    def __init__(self):
        super().__init__("add_two_ints_client")
        # 서비스 클라이언트 생성: 타입, 서비스명 지정
        self.client_ = self.create_client(AddTwoInts, "add_two_ints")
        # 서비스 서버가 활성화될 때까지 1초 간격으로 대기
        while not self.client_.wait_for_service(timeout_sec=1.0):
            self.get_logger().warn("서버 대기 중...")
        self.call_add(3, 5)

    def call_add(self, a, b):
        request = AddTwoInts.Request()
        request.a = a
        request.b = b
        # 비동기 방식으로 서비스 호출
        future = self.client_.call_async(request)
        future.add_done_callback(self.callback_response)

    def callback_response(self, future):
        try:
            response = future.result()
```

```

        self.get_logger().info(f"결과: {response.sum}")
    except Exception as e:
        self.get_logger().error(f"서비스 호출 실패: {e}")

def main(args=None):
    rclpy.init(args=args)
    node = AddTwoIntsClient()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

서버와 마찬가지로 `setup.py`에 클라이언트 노드 실행 항목을 등록합니다.

```
'add_two_ints_client = my_py_pkg.add_two_ints_client:main',
```

빌드 후 새로운 터미널에서 클라이언트를 실행하면 로그에 **결과: 8**이 출력되는 것을 볼 수 있습니다.

```
ros2 run my_py_pkg add_two_ints_client
```

△ 동기 호출의 함정 (Deadlock)

`rclpy`에서 동기식 호출 메서드(`client.call(request)`)를 사용하거나 `rclpy.spin_until_future_complete`를 단일 스레드 콜백 내부에서 중첩으로 실행하면 프로그램이 정지해버리는 교착 상태(deadlock)가 발생할 가능성이 높습니다. 입문 단계에서는 항상 위 예제와 같이 `call_async` 메서드와 완료 콜백(`done_callback`)을 조합하는 비동기 패턴을 권장합니다.

[워크드 예제] 곱셈 서비스 직접 만들기

`AddTwoInts` 타입을 그대로 재활용하되, 곱셈 연산을 수행하는 서버를 만들어 보겠습니다.

`my_py_pkg/my_py_pkg/multiply_server.py`를 만들고 아래와 같이 구현합니다.

```

import rclpy
from rclpy.node import Node
from example_interfaces.srv import AddTwoInts

```

```

class MultiplyServer(Node):
    def __init__(self):
        super().__init__("multiply_server")
        # 서비스명을 "multiply"로 지정
        self.create_service(AddTwoInts, "multiply", self.cb)
        self.get_logger().info("곱셈 서비스 서버 준비 완료")

    def cb(self, request, response):
        response.sum = request.a * request.b      # 더하기 대신 곱하기 적용
        self.get_logger().info(f"{request.a} * {request.b} = {response.sum}")
        return response

def main(args=None):
    rclpy.init(args=args)
    node = MultiplyServer()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()

```

`setup.py`에 등록하고 빌드한 뒤 아래 명령어로 터미널에서 직접 호출해 봅니다.

```

ros2 service call /multiply example_interfaces/srv/AddTwoInts "{a: 6, b: 7}"

```

결과로 **sum=42**를 반환받으면 정상입니다. 동일한 데이터 타입(**AddTwoInts**)도 내부 서비스 이름과 콜백 로직에 따라 유연하게 재사용할 수 있습니다.

4.3 C++로 구현하기

동일한 서비스 통신을 C++ 언어 기반의 **rclcpp** 라이브러리로 구현해 보겠습니다.

C++ 서비스 서버 작성

`my_cpp_pkg/src/add_two_ints_server.cpp` 파일을 만들고 아래 코드를 작성합니

다.

```
#include <functional>
#include <memory>
#include "rclcpp/rclcpp.hpp"
#include "example_interfaces/srv/add_two_ints.hpp"
using std::placeholders::_1;
using std::placeholders::_2;

class AddTwoIntsServer : public rclcpp::Node {
public:
    AddTwoIntsServer() : Node("add_two_ints_server") {
        // 서비스 서버 생성: 타입, 서비스명, 콜백 함수(바인딩) 지정
        server_ =
this->create_service<example_interfaces::srv::AddTwoInts>(
        "add_two_ints",
        std::bind(&AddTwoIntsServer::callbackAdd, this, _1, _2));
        RCLCPP_INFO(this->get_logger(), "C++ 서비스 서버 준비 완료");
    }

private:
    void callbackAdd(
        const example_interfaces::srv::AddTwoInts::Request::SharedPtr
req,
        const example_interfaces::srv::AddTwoInts::Response::SharedPtr
res) {
        res->sum = req->a + req->b;
        RCLCPP_INFO(this->get_logger(), "%ld + %ld = %ld", req->a,
req->b, res->sum);
    }
    rclcpp::Service<example_interfaces::srv::AddTwoInts>::SharedPtr
server_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<AddTwoIntsServer>());
    rclcpp::shutdown();
    return 0;
}
```

CMakeLists.txt & package.xml 빌드 설정

C++ 빌드 관리를 위해 의존성을 추가해 주어야 합니다.

CMakeLists.txt 파일에 아래 내용을 추가합니다.

```
find_package(rclcpp REQUIRED)
find_package(example_interfaces REQUIRED)

add_executable(add_two_ints_server src/add_two_ints_server.cpp)
ament_target_dependencies(add_two_ints_server rclcpp
example_interfaces)

install(TARGETS add_two_ints_server
DESTINATION lib/${PROJECT_NAME})
```

package.xml 파일의 의존성 영역에 아래 두 줄을 등록합니다.

```
<depend>rclcpp</depend>
<depend>example_interfaces</depend>
```

작성이 완료되면 패키지를 빌드하고 실행합니다.

```
cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
source install/setup.bash
ros2 run my_cpp_pkg add_two_ints_server
```

정의(2) RMW

ROS 2와 DDS 사이를 연결하는 추상화 계층입니다. 이 덕분에 C++ 노드와 Python 노드처럼 서로 다른 언어로 구현되더라도 RMW 계층 위에서 규격화된 메시지를 주고받으며 원활하게 소통할 수 있습니다.

4.4 ros2 service 명령으로 살펴보기

개발 도중 노드가 올바르게 서비스를 제공하고 통신을 맺고 있는지 터미널 상에서 확인할 수 있는 유용한 디버깅 명령어들입니다.

서비스 목록 확인

현재 활성화되어 있는 모든 서비스 목록을 보여줍니다.

```
ros2 service list
```

서비스 타입 확인

특정 서비스의 타입을 상세히 보여줍니다.

```
ros2 service type /add_two_ints
```

서비스 수동 호출 (중요)

코드 작성 전에 서비스 서버가 정상 동작하는지 테스트할 수 있습니다. 타입 뒤에 요청부 데이터를 JSON 형식의 문자열로 인자 전달합니다.

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts "{a: 10, b: 20}"
```

서비스 인터페이스 구조 확인

서비스가 정의하고 있는 요청/응답 변수 규격을 확인합니다.

```
ros2 interface show example_interfaces/srv/AddTwoInts
```

코드 해설 · 실행 확인 · 버전 체크

- **코드 해설 포인트:** `.srv` 서비스 파일의 요청부(Request)와 응답부(Response) 분리 구조를 바탕으로, 서버의 `create_service`, 클라이언트의 `create_client` 호출 및 비동기 호출 처리를 위한 `call_async`와 `done_callback` 결합 구조를 상세히 이해합니다.

- **실행 확인 포인트:** `ros2 service call`을 이용하여 Python 서버와 C++ 서버 각각에 요청을 보냈을 때 연산 결과가 터미널에 반환되는지 확인합니다. 또한 `ros2 service list`에 생성한 서비스명이 잡히는지 검증합니다.

· 버전/환경 체크: ROS 2 Jazzy 기준의 비동기 Future 객체 반환 및 RMW Fast DDS 미들웨어 계층의 동작 확인 흐름을 기준으로 검토합니다.

트러블슈팅

증상	원인	해결
클라이언트가 켜진 채 영원히 대기함	서비스 서버 노드가 켜지지 않았거나 서비스명이 다름	ros2 service list로 이름 대조 및 서버 기동 여부 확인
call_async 호출 후 응답 콜백이 반응 없음	실행 노드 루프에 spin을 걸지 않았거나 콜백 누락	rclpy.spin(node) 또는 rclcpp::spin이 돌고 있는지 점검
서비스 호출 도중 타입 에러가 발생함	요청 인자(JSON 구조 등) 포맷 및 필드 오타	ros2 interface show로 필드명(a, b)을 재확인하고 규격을 맞춰 입력
통신 도중 노드가 멈춤 (데드락)	콜백 함수 내부에서 다른 동기 통신을 중첩 대기함	비동기 호출(call_async)과 완료 콜백 함수 등록 방식으로 구조 개선

표 4-2. 서비스 구현 시 자주 발생하는 오류와 해결 방법

정리

* 서비스는 클라이언트가 요청하고 서버가 이에 상응하는 결과를 반환하는 일대일 양방향 요청-응답 통신 모델입니다.

* 서비스 타입(.srv 파일)은 --- 기호를 사용해 위칸의 요청(Request) 데이터 필드와 아래칸의 응답(Response) 데이터 필드를 나눕니다.

* 서비스 노드를 설계할 때 동기식 대기 루프는 교착 상태(deadlock)를 일으키기 쉬우므로, 비동기 호출(call_async)과 완료 콜백 함수 구조를 결합해 구현하는 것이 중요합니다.

* ros2 service call 등 전용 디버깅 CLI 명령을 활용하면 노드 생성 전이나 개발 과정에서 손쉽게 연산 검증이 가능합니다.

핵심 용어

- * 서비스(service): 요청과 응답 구조를 갖는 일대일 양방향 통신 방식
- * DDS(Data Distribution Service): 마스터 서버 없이 분산 디스커버리를 제공하는 분산 통신 규격
- * RMW(ROS Middleware): ROS 2와 하부 DDS 구현체 간 통신을 중개하여 상호 이식성을 높이는 미들웨어 추상화 계층
- * 비동기 호출(call_async): 요청을 보낸 뒤 응답이 올 때까지 프로그램 실행을 멈추지 않고, 응답 수신 시 비동기로 콜백을 수행하는 통신 패턴
- * LTS(Long-Term Support): 5년의 장기 기술 지원을 보장하는 ROS 배포판 버전 규격 (예: Jazzy Jalisco)

연습문제

1. AddTwoInts 타입을 그대로 사용하되, 두 수의 곱을 응답하는 곱셈 서비스를 구현하고 수동 호출 테스트를 실행하시오.
2. 클라이언트 노드가 실행될 때 고정 인자(3, 5) 대신 프로그램 실행 인수(sys.argv)로 두 숫자를 입력받아 서비스 요청을 수행하도록 코드를 고쳐 보시오.
3. ros2 service call 명령어를 통해 곱셈 서비스 서버를 호출해 결과를 확인하시오.
4. 아래 상황에 가장 적합한 통신 방식(토픽/서비스/액션)을 짚고 그 근거를 적으시오.
 - * (가) 라이다(LiDAR) 거리 스캔 데이터 수신
 - * (나) 로봇의 주행 속도 모터 제어 명령
 - * (다) 로봇 제어 시스템 내 누적 연산치 데이터 리셋 요청
 - * (라) 로봇 카메라 센서 데이터 실시간 확인
 - * (마) 시작 지점에서 10m 떨어진 거실 중앙까지 이동 주행 명령

연습문제 해설

- * 1번 문제: MultiplyServer를 생성하여 콜백 로직에 `response.sum = request.a`
* `request.b`를 설정합니다.
- * 2번 문제: Python의 `sys` 패키지를 가져와 `main` 혹은 초기화 함수 내에서 `a = int(sys.argv[1])`, `b = int(sys.argv[2])` 형태로 입력을 받아 처리합니다. (`ros2 run my_py_pkg client_node 4 9` 방식으로 기동)
- * 3번 문제: `ros2 service call /multiply example_interfaces/srv/AddTwoInts "{a: 5, b: 6}"` 명령을 사용해 `sum=30` 결과를 검증합니다.
- * 4번 문제: (가) 계속 흐르므로 토픽, (나) 속도 명령 지속 전송이므로 토픽, (다) 1회성 변경 요구이므로 서비스, (라) 이미지 데이터 스트리밍이므로 토픽, (마) 오래 걸리는 연속 작업과 중간 모니터링이 필요하므로 액션.

참고 자료

- ROS 2 Jazzy 공식 문서 – Writing a simple service and client (Python/C++)
- gcamp_ros2_basic 서비스 패키지 리포지토리 (`py_service_pkg`, `cpp_service_pkg`)
- 표윤석·임태훈, 『ROS 2로 시작하는 로봇 프로그래밍』, 루비페이퍼

마무리 점검

- ☐ 서비스가 필요한 상황(요청 하나에 응답 하나)을 토픽과 대비해 설명할 수 있습니다.
- ☐ .srv 파일의 요청/응답 2구역 구조를 압니다.
- ☐ Python으로 서비스 서버와 클라이언트를 작성하고 응답을 확인했습니다.
- ☐ C++로 같은 기능을 구현하고 빌드·실행했습니다.
- ☐ 서버가 준비될 때까지 클라이언트가 기다리는 동작(`wait_for_service`)을 압니다.

다.

□ `ros2 service list/type/call` 명령으로 서비스를 진단할 수 있습니다.

다음 장 예고

– 5장에서는 요청 한 번으로 끝나지 않는 긴 작업을 다룹니다. 목표를 보내고 피드백을 받아 가며, 필요하면 중간에 취소하는 액션 통신입니다.

05

액션

긴 작업을 목표·피드백·결과로 다룬다.
서버와 클라이언트를 구현하고
목표 취소까지 익힌다.

이 장에서 다루는 내용

- 오래 걸리는 작업에 서비스가 부족한 이유와 액션의 통신 모델
- 목표(goal)·피드백(feedback)·결과(result)·취소(cancel)의 전체 흐름
- Python과 C++로 액션 서버·클라이언트 구현하기
- 실행 중인 목표를 실제로 취소하는 실습
- ros2 action 명령어를 활용한 액션 디버깅 방법

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 액션(action)이 토픽·서비스로 부족한 상황을 메우는 이유를 설명할 수 있습니다.
2. 목표·피드백·결과·취소로 이어지는 액션 통신의 흐름을 그림으로 설명할 수 있습니다.
3. rclpy/rclcpp로 액션 서버와 클라이언트를 직접 작성할 수 있습니다.
4. 실행 중인 목표에 취소를 요청하고, 서버가 이를 처리하는 구조를 이해합니다.
5. ros2 action CLI 명령어로 목표를 전송하고 피드백을 관찰할 수 있습니다.

이번 장에서 만드는 것

- 피보나치 수열을 계산하며 중간 피드백을 발행하는 Python·C++ 액션 서버
- 목표를 보내고 피드백·결과를 수신하는 Python·C++ 액션 클라이언트
- 목표 전송 3초 뒤 자동으로 취소를 요청하는 실습 클라이언트
- ros2 action send_goal --feedback으로 확인한 목표 수락 → 피드백 → 결과의 전체 흐름

공통 학습 흐름: 개념 → 따라하기 → 코드 해설 → 실행 확인 → 버전/환경 체크 → 트러블슈팅 → 연습문제 → 마무리 점검

4장에서는 서비스로 질문 한 번에 대답 한 번인 통신을 만들었습니다. 두 수를 더하는 계산은 순식간에 끝나므로 이 모델로 충분했습니다. 그런데 로봇에게 시키는 일 중에는 순식간에 끝나지 않는 것이 많습니다. 저 방까지 이동하라거나 로봇

팔로 부품을 집어 올리라는 일은 수십 초가 걸리고, 그동안 잘 되고 있는지 확인하고 싶고, 상황이 바뀌면 중단시키고 싶습니다. 이런 요구를 위해 준비된 세 번째 통신 방식이 **액션(action)** 입니다. 실습은 이번에도 2장에서 만든 `my_py_pkg`와 `my_cpp_pkg`에 이어서 진행합니다.

5.1 액션 통신 모델

서비스로는 부족할 때

4장의 서비스 모델로 10m 전진하라는 명령을 보낸다고 상상해 봅시다. 클라이언트는 요청을 보내고 응답을 기다립니다. 그런데 로봇이 도착하기까지 30초가 걸린다면 문제가 하나둘이 아닙니다.

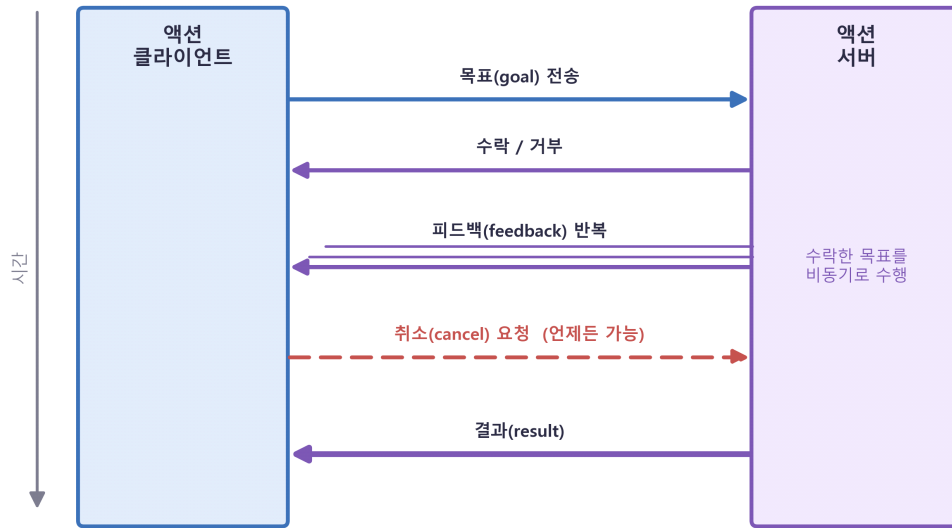
- **중간 상황을 알 수 없습니다.** 응답은 완료 시점에 한 번뿐이므로, 그 사이 로봇이 어디까지 갔는지, 잘 가고 있기는 한지 알 방법이 없습니다.
- **중단시킬 수 없습니다.** 이동 도중 사람이 나타나 멈춰야 해도, 이미 보낸 요청을 취소하는 수단이 서비스에는 없습니다.
- **기다림이 깁니다.** 서비스는 짧은 처리를 전제로 설계된 모델이라, 수십 초짜리 작업을 담기에는 구조 자체가 어색합니다.

액션은 이 세 가지 문제를 정면으로 해결합니다. 클라이언트가 **목표(goal)** 를 보내면 서버는 이를 수락하거나 거부하고, 수락한 목표를 비동기로 수행하면서 **피드백(feedback)** 을 주기적으로 보내고, 끝나면 **결과(result)** 를 돌려줍니다. 수행 도중 클라이언트는 언제든지 **취소(cancel)** 를 요청할 수 있습니다.

정의(1) 액션(action)

클라이언트가 보낸 목표를 서버가 수락해 오랜 시간에 걸쳐 수행하면서, 진행 상황을 피드백으로 보고하고 완료 시 결과를 반환하며, 도중 취소가 가능한 ROS 2의 통신 방식입니다.

목표·피드백·결과·취소: 통신의 네 가지 축



액션 통신의 전체 흐름을 그림 5-1의 구조도로 봅시다.

그림 5-1. 액션 통신의 전체 흐름. 목표 전송과 수락, 반복되는 피드백, 최종 결과, 그리고 취소 요청

택배 배송에 비유해 봅시다. 주문(목표)을 넣으면 판매자가 주문을 접수(수락)하고, 배송이 진행되는 동안 상품 준비 중 → 배송 중 → 배달 출발로 이어지는 배송 조회(피드백)를 볼 수 있으며, 물건이 도착하면 배송 완료(결과) 알림이 옵니다. 마음이 바뀌면 배송 전에 주문을 취소(취소 요청)할 수도 있습니다.

비유를 기술 용어로 정리하면 이렇습니다. 목표를 받아 수행하는 쪽이 **액션 서버(action server)**, 목표를 보내고 경과를 지켜보는 쪽이 **액션 클라이언트(action client)** 입니다. 서버는 받은 목표를 무조건 수행하는 것이 아니라 **수락(accept)** 또는 **거부(reject)** 를 먼저 결정합니다. 잘못된 목표(예: 음수 거리 이동)를 걸러 내는 검사 단계입니다. 수락된 목표는 서버 안에서 비동기로 실행되므로, 클라이언트는 응답에 묶여 기다리지 않고 자기 일을 계속하면서 피드백과 결과를 콜백으로 받습니다.

토픽·서비스·액션: 역할 분담

세 통신 방식은 경쟁 관계가 아니라 분업 관계입니다. 데이터가 계속 이어지면

토픽, 짧은 요청에 즉시 답하면 서비스, 오래 걸리고 경과 보고와 취소가 필요하면 액션입니다. 실제 로봇에서는 세 방식이 한 시스템 안에 공존합니다. 라이다 데이터는 토픽으로 전달되고, 지도 저장은 서비스로 요청하고, 목적지 이동은 액션으로 명령하는 식입니다. 세 방식의 전체 비교표는 이 장의 정리에서 다시 확인합니다.

참고로 액션은 내부적으로 서비스와 토픽을 조합해 구현됩니다. 목표 전송·결과 요청·취소는 서비스로, 피드백과 상태 알림은 토픽으로 처리됩니다. 그래서 액션을 서비스와 토픽의 합성이라고 요약하기도 합니다. 다만 rclpy/rclcpp의 액션 API가 이 내부 구조를 모두 감싸 주므로, 목표·피드백·결과·취소라는 개념만으로 액션을 다루면 됩니다. 내부 구현을 파고들 필요는 없습니다.

액션 타입: .action 파일

서비스의 형식을 .srv 파일에 정의했듯, 액션의 형식은 .action 파일에 정의합니다. .action 파일은 세 개의 하이픈(---) 두 줄로 세 영역을 나눕니다.

```
목표(goal) 필드
---
결과(result) 필드
---
피드백(feedback) 필드
```

이번 장 실습에서 사용할 표준 예제 인터페이스는 `example_interfaces/action/Fibonacci`입니다. 피보나치 수열(앞의 두 수를 더해 다음 수를 만드는 수열: 0, 1, 1, 2, 3, 5, ...)을 계산하는 액션으로, 구조는 다음 명령으로 확인합니다.

```
ros2 interface show example_interfaces/action/Fibonacci
int32 order
---
int32[] sequence
---
int32[] sequence
```

세 영역을 위에서부터 읽으면 이렇습니다. 목표는 order(수열을 몇 개까지 만들지), 결과는 완성된 수열 sequence, 피드백은 계산 중간까지 만들어진 수열

sequence입니다. 결과와 피드백의 필드 이름이 같지만 서로 다른 영역에 속한 별개의 필드입니다. 수열 계산은 한 단계씩 진행되므로 오래 걸리는 작업과 중간 경과 보고를 함께 연습하기에 알맞은 예제입니다.

버전 확인: 이전 배포판과 Jazzy의 차이

인터넷의 액션 예제 상당수는 튜토리얼용 커스텀 패키지 (action_tutorials_interfaces 등)의 Fibonacci를 사용하며, 그 피드백 필드 이름은 partial_sequence입니다. 이 장에서 사용하는 example_interfaces/action/Fibonacci는 피드백 필드 이름이 sequence입니다. 다른 자료의 코드를 가져올 때 피드백 필드 이름이 달라 오류가 나는 경우가 흔하니, ros2 interface show로 실제 필드 이름을 확인하는 습관을 들이세요.

나만의 .action 파일을 만들어 빌드하는 방법은 6장 커스텀 인터페이스에서 다룹니다. 이 장에서는 제공되는 표준 타입으로 액션의 사용법과 통신 흐름에 집중합니다.

5.2 액션 서버와 클라이언트

파이썬의 rclpy 라이브러리로 피보나치 액션 서버와 클라이언트를 구현해 보겠습니다. 서버는 목표로 받은 개수만큼 수열을 만들되, 한 단계마다 1초씩 걸리는 느린 작업을 흉내 내면서 중간 피드백을 발행합니다.

Python 액션 서버 구현하기

my_py_pkg/my_py_pkg/fibonacci_server.py 파일을 생성하고 아래 코드를 작성합니다.

```
import time

import rclpy
from rclpy.action import ActionServer, CancelResponse, GoalResponse
from rclpy.callback_groups import ReentrantCallbackGroup
from rclpy.executors import MultiThreadedExecutor
```

```

from rclpy.node import Node

from example_interfaces.action import Fibonacci

class FibonacciServer(Node):
    def __init__(self):
        super().__init__("fibonacci_server")
        # 액션 서버 생성: 타입, 액션명, 실행·수락·취소 콜백 지정
        self.server_ = ActionServer(
            self,
            Fibonacci,
            "fibonacci",
            execute_callback=self.execute_callback,
            goal_callback=self.goal_callback,
            cancel_callback=self.cancel_callback,
            callback_group=ReentrantCallbackGroup())
        self.get_logger().info("피보나치 액션 서버 준비 완료")

    def goal_callback(self, goal_request):
        # 목표 수락/거부 판단: 수열은 [0, 1]에서 시작하므로 2개 미만은 거부
        if goal_request.order < 2:
            self.get_logger().warn(f"목표 거부:
order={goal_request.order}")
            return GoalResponse.REJECT
        self.get_logger().info(f"목표 수락: order={goal_request.order}")
        return GoalResponse.ACCEPT

    def cancel_callback(self, goal_handle):
        # 취소 요청을 받아들일지 판단
        self.get_logger().info("취소 요청 수신: 수락")
        return CancelResponse.ACCEPT

    def execute_callback(self, goal_handle):
        order = goal_handle.request.order
        sequence = [0, 1]
        feedback = Fibonacci.Feedback()
        for i in range(2, order):
            # 취소 요청이 들어왔는지 매 단계 검사

```

```

        if goal_handle.is_cancel_requested:
            goal_handle.canceled()
            self.get_logger().warn("목표가 취소되었습니다")
            result = Fibonacci.Result()
            result.sequence = sequence
            return result

        sequence.append(sequence[i - 1] + sequence[i - 2])
        feedback.sequence = sequence
        goal_handle.publish_feedback(feedback)
        self.get_logger().info(f"피드백 발행: {sequence}")
        time.sleep(1.0) # 오래 걸리는 작업 흉내
        goal_handle.succeed()
        result = Fibonacci.Result()
        result.sequence = sequence
        self.get_logger().info(f"목표 완료: {sequence}")
        return result

def main(args=None):
    rclpy.init(args=args)
    node = FibonacciServer()
    executor = MultiThreadedExecutor()
    rclpy.spin(node, executor=executor)
    node.destroy_node()
    rclpy.shutdown()

if __name__ == "__main__":
    main()

```

서버 코드 해설: 세 개의 콜백과 실행기

액션 서버의 핵심은 역할이 나뉜 세 개의 콜백입니다.

- **goal_callback:** 목표가 도착하면 가장 먼저 호출됩니다. 목표 내용 (goal_request)을 보고 GoalResponse.ACCEPT 또는 REJECT를 반환합니다. 여기서 거부된 목표는 실행되지 않습니다.
- **cancel_callback:** 클라이언트가 취소를 요청하면 호출됩니다.

CancelResponse.ACCEPT를 반환해야 취소 절차가 진행됩니다. 주의할 점이 있습니다. rclpy의 기본 동작은 취소 거부이므로, 이 콜백을 등록하지 않으면 클라이언트가 아무리 취소를 요청해도 받아들여지지 않습니다.

- **execute_callback**: 수락된 목표를 실제로 수행합니다. goal_handle을 통해 목표 데이터를 읽고(goal_handle.request.order), 피드백을 발행하고(publish_feedback), 마지막으로 목표의 최종 상태를 확정합니다. 성공이면 goal_handle.succeed()를 호출한 뒤 결과 객체를 반환하고, 취소면 goal_handle.canceled()를 호출한 뒤 반환합니다.

실행 루프의 매 단계에서 goal_handle.is_cancel_requested를 검사하는 부분이 취소 처리의 핵심입니다. 취소는 서버가 강제로 중단당하는 것이 아니라, 실행 콜백이 스스로 확인하고 멈추는 방식으로 동작합니다. 로봇이 이동 도중 멈추려면 감속·정지 같은 마무리가 필요하듯, 안전하게 멈추는 방법은 작업을 수행하는 쪽만 알기 때문입니다.

마지막으로 main()을 보세요. 지금까지처럼 rclpy.spin(node)가 아니라 **MultiThreadedExecutor**를 만들어 넘기고, 액션 서버에는 **ReentrantCallbackGroup**을 지정했습니다. 이것이 이 장에서 가장 중요한 구조적 선택입니다.

이유는 이렇습니다. 기본 실행기는 단일 스레드로 콜백을 한 번에 하나씩 실행합니다. 그런데 이 서버의 execute_callback은 time.sleep을 반복하며 수십 초를 점유합니다. 단일 스레드라면 그동안 다른 콜백이 전혀 실행되지 못하므로, 클라이언트가 보낸 취소 요청을 처리할 콜백도 실행 콜백이 끝날 때까지 밀려 버립니다. 취소 검사 코드를 아무리 잘 써 두어도 is_cancel_requested가 참이 될 기회 자체가 없는 것입니다. MultiThreadedExecutor는 콜백을 여러 스레드에서 동시에 실행할 수 있게 하고, ReentrantCallbackGroup은 액션 서버의 콜백들이 서로를 기다리지 않고 병행 실행되도록 허용합니다. 이 두 가지 덕분에 실행 콜백이 돌아가는 도중에도 취소 요청이 처리됩니다.

정의(2) 실행기(executor)

노드에 등록된 콜백들을 어떤 스레드에서 어떤 순서로 실행할지 관리하는 구성 요소. 단일 스레드 실행기는 콜백을 하나씩, 멀티 스레드 실행기는 여러 콜백을 동시에 실행할 수 있습니다.

등록과 빌드, CLI로 첫 테스트

setup.py의 console_scripts에 등록합니다.

```
'fibonacci_server = my_py_pkg.fibonacci_server:main',
```

package.xml에는 example_interfaces 의존성이 필요합니다. 4장 실습을 따라 했다면 이미 추가되어 있을 것이므로 확인만 하고, 없다면 추가합니다.

```
<depend>example_interfaces</depend>
```

빌드하고 실행합니다.

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
```

터미널 1에서 서버를 실행해 둡니다.

```
ros2 run my_py_pkg fibonacci_server
```

클라이언트를 만들기 전에, CLI로 서버가 동작하는지 먼저 확인해 봅시다. 터미널 2에서 목표를 전송합니다. --feedback 옵션을 붙이면 피드백도 함께 출력됩니다.

```
ros2 action send_goal /fibonacci example_interfaces/action/Fibonacci
"{order: 6}" --feedback
```

다음과 비슷한 흐름이 출력되면 정상입니다(예시이며, 출력 형식은 환경에 따라 다소 다를 수 있습니다).

```
Waiting for an action server to become available...
Sending goal:
  order: 6

Goal accepted with ID: a1b2c3d4e5f6...

Feedback:
```

```

sequence:
- 0
- 1
- 1

Feedback:
sequence:
- 0
- 1
- 1
- 2

(중간 피드백 계속)

Result:
sequence:
- 0
- 1
- 1
- 2
- 3
- 5

Goal finished with status: SUCCEEDED

```

목표 수락 → 피드백 여러 번 → 최종 결과의 순서가 그대로 보입니다. 터미널 1의 서버 로그에서도 목표 수락과 피드백 발행 기록을 확인할 수 있습니다.

Python 액션 클라이언트 구현하기

이제 코드로 목표를 보내는 클라이언트를 만듭니다.
my_py_pkg/my_py_pkg/fibonacci_client.py 파일을 생성하고 작성합니다.

```

import rclpy
from rclpy.action import ActionClient
from rclpy.node import Node

from example_interfaces.action import Fibonacci

```

```

class FibonacciClient(Node):
    def __init__(self):
        super().__init__("fibonacci_client")
        # 액션 클라이언트 생성: 타입, 액션명 지정
        self.client_ = ActionClient(self, Fibonacci, "fibonacci")

    def send_goal(self, order):
        self.get_logger().info("액션 서버 대기 중...")
        self.client_.wait_for_server()
        goal = Fibonacci.Goal()
        goal.order = order
        self.get_logger().info(f"목표 전송: order={order}")
        # 비동기 목표 전송: goal future를 돌려받는다
        send_goal_future = self.client_.send_goal_async(
            goal, feedback_callback=self.feedback_callback)
        send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        # goal future 완료 -> 목표 핸들 획득
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().warn("목표가 거부되었습니다")
            rclpy.shutdown()
            return
        self.get_logger().info("목표가 수락되었습니다")
        # 결과 future 등록: 이걸 빠뜨리면 결과를 영영 받지 못한다
        result_future = goal_handle.get_result_async()
        result_future.add_done_callback(self.result_callback)

    def feedback_callback(self, feedback_msg):
        partial = list(feedback_msg.feedback.sequence)
        self.get_logger().info(f"피드백 수신: {partial}")

    def result_callback(self, future):
        result = future.result().result
        self.get_logger().info(f"최종 결과: {list(result.sequence)}")
        rclpy.shutdown() # 결과를 받았으므로 spin을 끝낸다

```

```
def main(args=None):
    rclpy.init(args=args)
    node = FibonacciClient()
    node.send_goal(8)
    rclpy.spin(node)

if __name__ == "__main__":
    main()
```

클라이언트 코드 해설: future와 goal handle 구분하기

클라이언트의 흐름은 서버 대기 → 목표 생성 → 비동기 전송 → 수락 확인 → 피드백 수신 → 결과 요청 → 결과 출력의 일곱 단계입니다. 코드에 등장하는 비슷해 보이는 개념들을 정확히 구분해 봅시다.

- **send_goal_async()**와 goal future: 목표를 보내고 즉시 반환됩니다. 반환값인 goal future는 서버가 목표를 수락했는지 아닌지가 나중에 담기는 상자입니다. 4장의 **call_async()**가 응답을 담은 future를 돌려주던 것과 같은 패턴입니다.

- goal handle: goal future가 완료되면 **future.result()**로 **목표 핸들(goal handle)**을 얻습니다. 목표 핸들은 서버에 접수된 내 목표를 가리키는 손잡이로, **accepted** 필드로 수락 여부를 확인하고, 결과를 요청하거나(**get_result_async**) 취소를 요청하는(**cancel_goal_async**) 창구가 됩니다.

- **get_result_async()**와 result future: 목표가 수락되었다고 결과가 저절로 오지는 않습니다. 끝나면 결과를 달라고 result future를 등록해야 하고, 서버의 실행이 끝나면 이 future가 완료되어 **result_callback**이 호출됩니다. 입문자가 가장 자주 빠뜨리는 단계입니다. 이것 빠뜨리면 서버는 일을 다 끝냈는데 클라이언트는 아무것도 받지 못한 채 조용히 기다리게 됩니다.

- **feedback callback**: **send_goal_async**에 함께 등록하며, 서버가 피드백을 발행할 때마다 호출됩니다. 피드백 데이터는 **feedback_msg.feedback.sequence**처럼 **.feedback**을 한 번 거쳐 접근합니다.

목표 거부와 수락이 갈리는 지점도 눈여겨보세요. goal_handle.accepted가 거짓이면 서버의 goal_callback이 거부한 것입니다. 거부된 목표에는 피드백도 결과도 없으므로 그대로 반환합니다.

클라이언트 실행

setup.py에 등록하고 빌드합니다.

```
'fibonacci_client = my_py_pkg.fibonacci_client:main',  
  
cd ~/ros2_ws  
colcon build --packages-select my_py_pkg  
source install/setup.bash
```

터미널 1에 서버가 실행 중인 상태에서, 터미널 2에서 클라이언트를 실행합니다.

```
ros2 run my_py_pkg fibonacci_client
```

터미널 2에 다음과 같은 흐름이 출력됩니다(예시).

```
[INFO] [fibonacci_client]: 액션 서버 대기 중...  
[INFO] [fibonacci_client]: 목표 전송: order=8  
[INFO] [fibonacci_client]: 목표가 수락되었습니다  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1]  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1, 2]  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1, 2, 3]  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1, 2, 3, 5]  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1, 2, 3, 5, 8]  
[INFO] [fibonacci_client]: 피드백 수신: [0, 1, 1, 2, 3, 5, 8, 13]  
[INFO] [fibonacci_client]: 최종 결과: [0, 1, 1, 2, 3, 5, 8, 13]
```

1초 간격으로 피드백이 쌓이다가 마지막에 결과가 도착하고 클라이언트가 종료됩니다. 목표 → 피드백 → 결과의 전체 사이클을 코드로 완주한 것입니다.

C++로 구현하기

동일한 액션 서버와 클라이언트를 C++로 구현합니다. C++에서는 액션 전용 라이브러리 rclcpp_action을 사용합니다.

C++ 액션 서버 작성

my_cpp_pkg/src/fibonacci_server.cpp 파일을 만들고 아래 코드를 작성합니다.

```
#include <functional>
#include <memory>
#include <thread>
#include "example_interfaces/action/fibonacci.hpp"
#include "rclcpp/rclcpp.hpp"
#include "rclcpp_action/rclcpp_action.hpp"
using Fibonacci = example_interfaces::action::Fibonacci;
using GoalHandleFibonacci =
rclcpp_action::ServerGoalHandle<Fibonacci>;
using namespace std::placeholders;

class FibonacciServer : public rclcpp::Node {
public:
    FibonacciServer() : Node("fibonacci_server") {
        // 액션 서버 생성: goal/cancel/accepted 세 콜백을 바인딩
        server_ = rclcpp_action::create_server<Fibonacci>(
            this, "fibonacci",
            std::bind(&FibonacciServer::handleGoal, this, _1, _2),
            std::bind(&FibonacciServer::handleCancel, this, _1),
            std::bind(&FibonacciServer::handleAccepted, this, _1));
        RCLCPP_INFO(this->get_logger(), "C++ 피보나치 액션 서버 준비 완료");
    }

private:
    rclcpp_action::GoalResponse handleGoal(
        const rclcpp_action::GoalUUID & uuid,
        std::shared_ptr<const Fibonacci::Goal> goal) {
        (void)uuid;
        if (goal->order < 2) {
            RCLCPP_WARN(this->get_logger(), "목표 거부: order=%d",
goal->order);
            return rclcpp_action::GoalResponse::REJECT;
        }
        RCLCPP_INFO(this->get_logger(), "목표 수락: order=%d",
goal->order);
        return rclcpp_action::GoalResponse::ACCEPT_AND_EXECUTE;
    }
}
```

```

rclcpp_action::CancelResponse handleCancel(
    const std::shared_ptr<GoalHandleFibonacci> goal_handle) {
    (void)goal_handle;
    RCLCPP_INFO(this->get_logger(), "취소 요청 수신: 수락");
    return rclcpp_action::CancelResponse::ACCEPT;
}

void handleAccepted(const std::shared_ptr<GoalHandleFibonacci>
goal_handle) {
    // 실행이 오래 걸리므로 spin을 막지 않도록 별도 스레드에서 수행
    std::thread{std::bind(&FibonacciServer::execute, this, _1),
goal_handle}
        .detach();
}

void execute(const std::shared_ptr<GoalHandleFibonacci>
goal_handle) {
    rclcpp::Rate loop_rate(1); // 1초에 한 단계
    const auto goal = goal_handle->get_goal();
    auto feedback = std::make_shared<Fibonacci::Feedback>();
    auto result = std::make_shared<Fibonacci::Result>();
    auto & sequence = feedback->sequence;
    sequence.push_back(0);
    sequence.push_back(1);
    for (int i = 2; (i < goal->order) && rclcpp::ok(); ++i) {
        // 취소 요청이 들어왔는지 매 단계 검사
        if (goal_handle->is_canceling()) {
            result->sequence = sequence;
            goal_handle->canceled(result);
            RCLCPP_WARN(this->get_logger(), "목표가 취소되었습니다");
            return;
        }
        sequence.push_back(sequence[i - 1] + sequence[i - 2]);
        goal_handle->publish_feedback(feedback);
        RCLCPP_INFO(this->get_logger(), "피드백 발행: %d개",
(int)sequence.size());
        loop_rate.sleep();
    }
    if (rclcpp::ok()) {

```

```

        result->sequence = sequence;
        goal_handle->succeed(result);
        RCLCPP_INFO(this->get_logger(), "목표 완료");
    }
}

rclcpp_action::Server<Fibonacci>::SharedPtr server_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<FibonacciServer>());
    rclcpp::shutdown();
    return 0;
}

```

C++ 서버의 구조는 Python과 같은 논리를 다르게 배치한 것입니다. 콜백이 goal·cancel·accepted 셋으로 나뉘는데, Python의 execute_callback에 해당하는 실행 함수는 accepted callback에서 시작됩니다. 그리고 Python에서 MultiThreadedExecutor로 해결했던 긴 실행이 spin을 막는 문제를, C++ 예제에서는 handleAccepted가 실행 함수를 별도 스레드(std::thread)로 띄우는 방식으로 해결합니다.

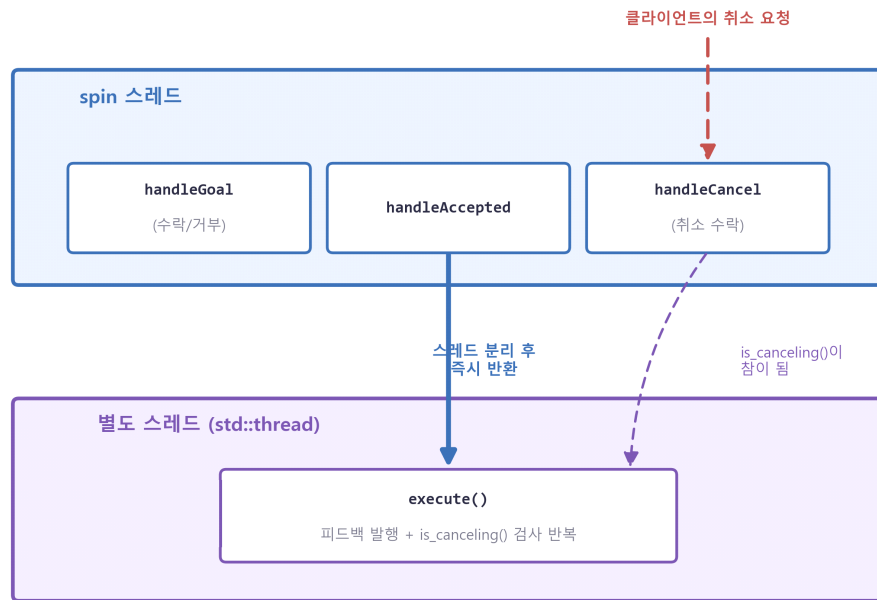


그림 5-2. C++ 액션 서버의 콜백 구조 - 실행 함수를 별도 스레드로 분리해 spin 스레드가 취소 요청을 받는 형태

실행이 다른 스레드에서 돌아가므로 spin 스레드는 자유롭게 취소 요청을 받을 수 있고, `is_canceling()` 검사가 실제로 동작합니다. 수단은 다르지만 목적은 같습니다. 오래 걸리는 실행이 콜백 처리를 독점하지 못하게 하는 것입니다.

또 하나, Python의 수락 응답은 `ACCEPT` 하나였지만 C++은 `ACCEPT_AND_EXECUTE`(수락 즉시 실행)를 사용합니다. 수락만 해 두고 실행 시점을 미루는 `ACCEPT_AND_DEFER`도 있지만 입문 범위를 넘으므로, 즉시 실행형만 기억하면 됩니다.

C++ 액션 클라이언트 작성

`my_cpp_pkg/src/fibonacci_client.cpp` 파일을 만들고 아래 코드를 작성합니다.

```
#include <chrono>
#include <functional>
#include <memory>
#include <string>
#include "example_interfaces/action/fibonacci.hpp"
#include "rclcpp/rclcpp.hpp"
```

```

#include "rclcpp_action/rclcpp_action.hpp"
using Fibonacci = example_interfaces::action::Fibonacci;
using GoalHandleFibonacci =
rclcpp_action::ClientGoalHandle<Fibonacci>;
using namespace std::placeholders;

class FibonacciClient : public rclcpp::Node {
public:
    FibonacciClient() : Node("fibonacci_client") {
        client_ = rclcpp_action::create_client<Fibonacci>(this,
"fibonacci");
        // 생성 직후 한 번만 목표를 보내도록 타이머 사용
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(500),
            std::bind(&FibonacciClient::sendGoal, this));
    }

private:
    void sendGoal() {
        timer_>cancel(); // 한 번만 실행
        if (!client_>wait_for_action_server(std::chrono::seconds(5)))
        {
            RCLCPP_ERROR(this->get_logger(), "액션 서버를 찾을 수 없습니다
");
            rclcpp::shutdown();
            return;
        }
        auto goal = Fibonacci::Goal();
        goal.order = 8;
        // goal options에 세 개의 콜백을 등록
        auto options =
rclcpp_action::Client<Fibonacci>::SendGoalOptions();
        options.goal_response_callback =
            std::bind(&FibonacciClient::goalResponseCallback, this,
_1);
        options.feedback_callback =
            std::bind(&FibonacciClient::feedbackCallback, this, _1,
_2);
        options.result_callback =
            std::bind(&FibonacciClient::resultCallback, this, _1);
    }

```

```

        RCLCPP_INFO(this->get_logger(), "목표 전송: order=%d",
goal.order);
        client_->async_send_goal(goal, options);
    }

    void goalResponseCallback(const GoalHandleFibonacci::SharedPtr &
goal_handle) {
        if (!goal_handle) {
            RCLCPP_WARN(this->get_logger(), "목표가 거부되었습니다");
        } else {
            RCLCPP_INFO(this->get_logger(), "목표가 수락되었습니다");
        }
    }

    void feedbackCallback(
        GoalHandleFibonacci::SharedPtr,
        const std::shared_ptr<const Fibonacci::Feedback> feedback) {
        std::string text;
        for (auto n : feedback->sequence) {
            text += std::to_string(n) + " ";
        }
        RCLCPP_INFO(this->get_logger(), "피드백 수신: %s",
text.c_str());
    }

    void resultCallback(const GoalHandleFibonacci::WrappedResult &
result) {
        if (result.code == rclcpp_action::ResultCode::CANCELED) {
            RCLCPP_WARN(this->get_logger(), "목표가 취소되었습니다");
            rclcpp::shutdown();
            return;
        }
        if (result.code != rclcpp_action::ResultCode::SUCCEEDED) {
            RCLCPP_ERROR(this->get_logger(), "목표가 실패했습니다");
            rclcpp::shutdown();
            return;
        }
        std::string text;
        for (auto n : result.result->sequence) {
            text += std::to_string(n) + " ";

```

```

    }
    RCLCPP_INFO(this->get_logger(), "최종 결과: %s", text.c_str());
    rclcpp::shutdown();
}

rclcpp_action::Client<Fibonacci>::SharedPtr client_;
rclcpp::TimerBase::SharedPtr timer_;
};

int main(int argc, char **argv) {
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<FibonacciClient>());
    rclcpp::shutdown();
    return 0;
}

```

C++ 클라이언트에서 눈에 띄는 것은 **SendGoalOptions** 입니다. Python에서는 피드백 콜백을 `send_goal_async`의 인자로, 결과 콜백을 `get_result_async` 이후에 각각 등록했지만, C++에서는 goal options라는 하나의 묶음에 목표 응답·피드백·결과 콜백 세 가지를 모두 등록한 뒤 `async_send_goal`에 함께 넘깁니다. 결과 future를 따로 등록하는 단계가 없는 대신, options에 `result_callback`을 지정하는 것이 그 역할을 합니다. 결과 콜백에서는 `result.code`로 성공(SUCCEEDED)·취소(CANCELED)·실패를 구분한 뒤 결과 데이터에 접근합니다.

CMakeLists.txt와 package.xml 설정

`my_cpp_pkg/CMakeLists.txt`에 아래 내용을 추가합니다. 3장·4장에서 이미 추가한 `find_package` 줄은 중복 작성하지 않습니다.

```

find_package(rclcpp_action REQUIRED)
find_package(example_interfaces REQUIRED)

add_executable(fibonacci_server src/fibonacci_server.cpp)
ament_target_dependencies(fibonacci_server rclcpp rclcpp_action
example_interfaces)

add_executable(fibonacci_client src/fibonacci_client.cpp)

```

```
ament_target_dependencies(fibonacci_client rclcpp rclcpp_action
example_interfaces)
```

```
install(TARGETS fibonacci_server fibonacci_client
DESTINATION lib/${PROJECT_NAME})
```

package.xml의 의존성 영역에 아래를 등록합니다(example_interfaces는 4장에서 추가했다면 확인만 합니다).

```
<depend>rclcpp_action</depend>
<depend>example_interfaces</depend>
```

빌드와 실행:

```
cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
source install/setup.bash
```

터미널 1에서 C++ 서버를, 터미널 2에서 C++ 클라이언트를 실행합니다.

```
# 터미널 1
ros2 run my_cpp_pkg fibonacci_server

# 터미널 2
ros2 run my_cpp_pkg fibonacci_client
```

Python 버전과 같은 흐름으로 피드백이 쌓이고 결과가 출력됩니다. 액션 이름과 타입이 같으므로 Python 서버에 C++ 클라이언트를 연결하는 조합도 물론 동작합니다(단, 같은 이름의 노드를 동시에 띄우지 않도록 서버는 한 쪽만 실행합니다).

Python과 C++ 대응 관계

개념	Python(rclpy)	C++(rclcpp_action)
서버 생성	ActionServer(self, 타입, 이름, ...)	rclcpp_action::create_server<타입>(this, 이름, ...)
목표 수락/거부	goal_callback → GoalResponse.ACCEPT	goal_callback → GoalResponse::ACCEPT_AND_EXECUTE
취소 수락	cancel_callback → CancelResponse.ACCEPT	cancel_callback → CancelResponse::ACCEPT
실행	execute_callback	accepted callback에서 별도 스레드로 execute 실행
취소 검사	goal_handle.is_cancel_requested	goal_handle->is_canceling()
피드백 발행	goal_handle.publish_feedback(fb)	goal_handle->publish_feedback(fb)
성공 처리	goal_handle.succeed() 후 결과 반환	goal_handle->succeed(result)
긴 실행 대책	MultiThreadedExecutor + ReentrantCallbackGroup	실행을 std::thread로 분리
클라이언트 생성	ActionClient(self, 타입, 이름)	rclcpp_action::create_client<타입>(this, 이름)
목표 전송	send_goal_async(goal, feedback_callback =...)	async_send_goal(goal, SendGoalOptions)
결과 수신	get_result_async() + done callback	options의 result_callback

표 5-1. 액션 API의 Python·C++ 대응 관계

5.3 피드백과 목표 취소

취소가 필요한 순간

액션이 토픽·서비스와 결정적으로 다른 능력이 취소입니다. 이동 중인 로봇 앞에 사람이 나타났을 때, 더 급한 작업이 생겨 현재 작업을 중단해야 할 때, 취소는 안전과 직결되는 기능입니다. 5.2절의 서버에는 이미 취소를 처리하는 코드(cancel_callback, is_cancel_requested 검사)가 들어 있습니다. 이번 절에서는 클라이언트 쪽에서 실제로 취소를 요청해, 취소의 전체 흐름을 눈으로 확인합니다.

실습 방법을 분명히 해 둡니다. 키보드 인터럽트(Ctrl + C)로 클라이언트를 강제 종료하는 것이 아니라, 클라이언트 코드가 목표 수락 3초 뒤에 스스로 취소를 요청하는 방식입니다. 취소 요청 → 서버의 취소 수락 → 실행 중단 → 취소 상태의 결과 수신까지가 모두 코드로 처리됩니다.

[따라하기] 3초 후 자동 취소하는 클라이언트

충분히 오래 걸리는 목표(order=20, 완료까지 약 18초)를 보내 놓고, 3초 뒤 취소
요청하는 클라이언트를 만듭니다.
my_py_pkg/my_py_pkg/timed_cancel_client.py 파일을 생성하고 작성합니다.

```
import rclpy
from action_msgs.msg import GoalStatus
from rclpy.action import ActionClient
from rclpy.node import Node

from example_interfaces.action import Fibonacci

class TimedCancelClient(Node):
    def __init__(self):
        super().__init__("timed_cancel_client")
        self.client_ = ActionClient(self, Fibonacci, "fibonacci")
        self.goal_handle_ = None

    def send_goal(self, order):
        self.get_logger().info("액션 서버 대기 중...")
        self.client_.wait_for_server()
        goal = Fibonacci.Goal()
        goal.order = order
        self.get_logger().info(f"목표 전송: order={order}")
        send_goal_future = self.client_.send_goal_async(
            goal, feedback_callback=self.feedback_callback)
        send_goal_future.add_done_callback(self.goal_response_callback)

    def goal_response_callback(self, future):
        goal_handle = future.result()
        if not goal_handle.accepted:
            self.get_logger().warn("목표가 거부되었습니다")
            rclpy.shutdown()
            return
        self.get_logger().info("목표가 수락되었습니다")
        self.goal_handle_ = goal_handle

        goal_handle.get_result_async().add_done_callback(self.result_callback)
        # 목표 수락 3초 뒤에 취소를 요청하는 타이머
```

```

        self.cancel_timer_ = self.create_timer(3.0, self.cancel_goal)

def cancel_goal(self):
    self.cancel_timer_.cancel() # 타이머는 한 번만 실행
    if self.goal_handle_ is None:
        return
    self.get_logger().warn("3초 경과: 취소 요청 전송")
    cancel_future = self.goal_handle_.cancel_goal_async()
    cancel_future.add_done_callback(self.cancel_response_callback)

def cancel_response_callback(self, future):
    response = future.result()
    if len(response.goals_canceling) > 0:
        self.get_logger().info("서버가 취소 요청을 접수했습니다")
    else:
        self.get_logger().warn("취소할 목표가 없습니다(이미 완료됨)")

def feedback_callback(self, feedback_msg):
    partial = list(feedback_msg.feedback.sequence)
    self.get_logger().info(f"피드백 수신: {partial}")

def result_callback(self, future):
    status = future.result().status
    if status == GoalStatus.STATUS_CANCELED:
        partial = list(future.result().result.sequence)
        self.get_logger().info(f"목표가 취소로 종료. 중단 시점 수열: {partial}")
    elif status == GoalStatus.STATUS_SUCCEEDED:
        self.get_logger().info(
            f"결과: {list(future.result().result.sequence)}")
    rclpy.shutdown()

def main(args=None):
    rclpy.init(args=args)
    node = TimedCancelClient()
    node.send_goal(20)
    rclpy.spin(node)

```

```
if __name__ == "__main__":
    main()
```

코드에서 새로 등장한 것은 세 가지입니다. 첫째, `cancel_goal_async()`는 목표 핸들을 통해 취소를 요청하고 `cancel future`를 돌려줍니다. 이 `future`가 완료되면 서버가 취소 요청을 접수했는지(`goals_canceling` 목록)를 알 수 있습니다. 둘째, 취소 요청의 접수와 목표의 종료는 별개입니다. 실제 종료는 서버의 실행 콜백이 `canceled()`를 호출한 뒤, 이미 등록해 둔 `result future`를 통해 전달됩니다. 셋째, **결과**의 **상태(status)**입니다. 같은 `result future`라도 성공이면 `STATUS_SUCCEEDED`, 취소면 `STATUS_CANCELED` 상태로 도착하므로, `action_msgs`의 `GoalStatus` 상수와 비교해 구분합니다. `action_msgs`를 import했으므로 `package.xml`에 의존성도 추가합니다.

```
<depend>action_msgs</depend>
```

실행과 관찰

`setup.py`에 `'timed_cancel_client = my_py_pkg.timed_cancel_client:main'`,을 등록하고 빌드합니다.

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source install/setup.bash
```

터미널 1에서 Python 액션 서버를 실행하고, 터미널 2에서 취소 클라이언트를 실행합니다.

```
# 터미널 1
ros2 run my_py_pkg fibonacci_server

# 터미널 2
ros2 run my_py_pkg timed_cancel_client
```

터미널 2의 출력 흐름은 다음과 같습니다(예시).

```
[INFO] [timed_cancel_client]: 액션 서버 대기 중...
[INFO] [timed_cancel_client]: 목표 전송: order=20
```

```
[INFO] [timed_cancel_client]: 목표가 수락되었습니다
[INFO] [timed_cancel_client]: 피드백 수신: [0, 1, 1]
[INFO] [timed_cancel_client]: 피드백 수신: [0, 1, 1, 2]
[INFO] [timed_cancel_client]: 피드백 수신: [0, 1, 1, 2, 3]
[WARN] [timed_cancel_client]: 3초 경과: 취소 요청 전송
[INFO] [timed_cancel_client]: 서버가 취소 요청을 접수했습니다
[INFO] [timed_cancel_client]: 목표가 취소로 종료. 중단 시점 수열: [0, 1, 1, 2, 3]
```

같은 시각 터미널 1의 서버 로그에는 취소 요청 수신: 수락과 목표가 취소되었습니다가 찍힙니다. 피드백이 3초 동안 쌓이다가, 취소 요청이 접수되고, 서버가 실행을 멈추고, 클라이언트가 취소 상태의 결과를 받는 전체 사이클이 완성되었습니다.

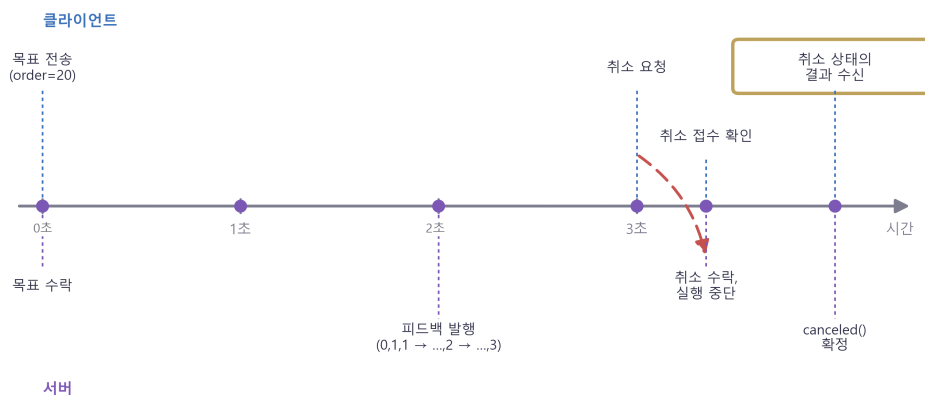


그림 5-3. 3초 후 자동 취소 실습의 시간 흐름 — 취소 요청 접수와 취소 상태의 결과 수신은 별개의 단계

이 실습이 동작하는 데 필요했던 조건을 되짚어 봅시다.

1. 서버에 `cancel_callback`이 등록되어 ACCEPT를 반환할 것(기본값은 거부)
2. 실행 콜백이 `is_cancel_requested`를 주기적으로 검사할 것
3. 서버가 `MultiThreadedExecutor`로 돌고 있어 실행 중에도 취소 요청 콜백이 처리될 것

셋 중 하나라도 빠지면 취소는 동작하지 않습니다. 트러블슈팅 표에서 각 경우의 증상을 확인할 수 있습니다.

참고: CLI에서도 취소를 볼 수 있습니다

ros2 action send_goal ... --feedback 실행 도중 Ctrl + C를 누르면, CLI 도구가 종료되기 전에 취소 요청을 서버로 보냅니다. 서버 로그에 취소 처리 기록이 남는 것을 확인해 보세요. 코드 없이 서버의 취소 처리를 시험하는 가장 빠른 방법입니다.

5.4 ros2 action 명령으로 살펴보기

서비스의 ros2 service처럼, 액션에도 전용 CLI가 있습니다. 터미널 1에 액션 서버를 실행해 둔 상태에서 하나씩 확인해 봅시다.

액션 목록 확인

현재 활성화된 모든 액션 목록을 보여줍니다.

```
ros2 action list  
/fibonacci
```

-t 옵션을 붙이면 액션 타입을 함께 보여줍니다.

```
ros2 action list -t  
/fibonacci [example_interfaces/action/Fibonacci]
```

액션 타입 확인

특정 액션의 타입만 확인합니다.

```
ros2 action type /fibonacci  
example_interfaces/action/Fibonacci
```

액션 상세 정보 확인

액션에 연결된 서버와 클라이언트의 수, 노드 이름을 보여줍니다.

```
ros2 action info /fibonacci  
  
Action: /fibonacci  
Action clients: 0  
Action servers: 1  
    /fibonacci_server
```

Action servers: 0이면 서버가 떠 있지 않다는 뜻이므로, 클라이언트가 무한 대기하는 문제의 원인을 바로 찾을 수 있습니다.

액션 인터페이스 구조 확인

액션이 정의하는 목표/결과/피드백 필드 규격을 확인합니다.

```
ros2 interface show example_interfaces/action/Fibonacci
```

목표 수동 전송

코드 없이 목표를 보내 서버를 시험합니다. 목표부 데이터를 YAML 형식 문자열로 전달합니다.

```
ros2 action send_goal /fibonacci example_interfaces/action/Fibonacci  
"{order: 10}"
```

이 형태는 목표 수락과 최종 결과만 출력합니다. 진행 중인 피드백까지 보려면 `--feedback` 옵션을 붙입니다.

```
ros2 action send_goal /fibonacci example_interfaces/action/Fibonacci  
"{order: 10}" --feedback
```

정상이라면 5.2절에서 본 것처럼 목표 수락(Goal accepted) → 피드백(Feedback) 여러 번 → 결과(Result) → 종료 상태(SUCCEEDED)의 순서로 출력됩니다. 이 순서 자체가 액션 통신 모델의 요약입니다.

실제 로봇에서의 액션: Nav2 내비게이션

이 장에서 만든 구조가 실제 로봇에서 어떻게 쓰이는지 확인해 봅시다. ROS 2의 표준 내비게이션 프레임워크 Nav2는 지정한 좌표까지 이동하라는 명령을 `NavigateToPose`라는 액션으로 제공합니다. 피보나치 예제와 나란히 놓으면 구조가 정확히 겹칩니다.

흐름	피보나치 예제	Nav2 내비게이션
목표	수열을 20개까지 계산하라	좌표 (x, y)까지 이동하라
수락/거부	order가 2 미만이면 거부	목표의 유효성을 확인하여 수락 또는 거부
피드백	지금까지 계산된 수열	현재 위치, 남은 거리, 예상 시간
취소	3초 뒤 클라이언트가 취소	장애물·위험 감지 시 이동 중단
결과	완성된 수열	도착 성공 또는 실패

표 5-2. 피보나치 예제와 Nav2 액션의 구조 대응

로봇이 목적지로 이동하는 수십 초 동안 상위 제어 노드는 피드백으로 진행률을 감시하다가, 위험 상황이면 목표를 취소하고, 완료되면 성공/실패 결과에 따라 다음 행동을 결정합니다. 이 장에서 작성한 클라이언트 코드의 골격(목표 전송, 수락 확인, 피드백 콜백, 결과 상태 분기, 취소 요청)이 그대로 Nav2 클라이언트의 골격이 됩니다. Nav2를 실제로 구동하는 내용은 2권에서 다루므로, 여기서는 액션 패턴이 실전으로 이어지는 연결 고리만 확인해 둡니다.

코드 해설 · 실행 확인 · 버전 체크

- **코드 해설 포인트:** .action 파일의 목표/결과/피드백 3구역 구조를 바탕으로, 서버의 세 콜백(goal/cancel/execute)의 역할 분담과 goal_handle을 통한 피드백 발행·상태 확정(succeed/canceled), 클라이언트의 goal future → goal handle → result future로 이어지는 2단계 비동기 흐름을 이해합니다. 긴 실행 콜백이 취소 처리를 막지 않도록 Python은 MultiThreadedExecutor + ReentrantCallbackGroup, C++은 실행 스레드 분리를 사용한 이유를 설명할 수 있어야 합니다.

- **실행 확인 포인트:** ros2 action send_goal --feedback으로 수락 → 피드백 → 결과의 순서를 확인하고, ros2 action list/info로 서버 존재를 검증합니다. timed_cancel_client 실행 시 서버와 클라이언트 양쪽 로그에 취소 접수·취소 종료 가 기록되는지 확인합니다.

- **버전/환경 체크:** ROS 2 Jazzy 기준으로 example_interfaces/action/Fibonacci의 피드백 필드는 sequence입니다(튜토리얼 커스텀 타입의 partial_sequence와 다

름). rclpy의 취소 기본 동작이 거부하는 점, ros2 action type 명령이 제공된다는 점도 Jazzy 기준으로 확인했습니다.

트러블슈팅

증상	원인	확인	해결
클라이언트가 wait_for_server에서 무한 대기	액션 서버 미실행 또는 액션 이름 불일치	ros2 action list, ros2 action info /fibonacci	서버 실행, 양쪽 액션 이름 통일
목표를 보냈는데 아무 반응 없음	액션 타입 불일치	ros2 action list -t로 타입 확인	양쪽 코드의 타입을 통일
목표가 계속 대기 상태에 머물	서버의 실행 콜백이 이전 목표에 묶여 있음	서버 로그에서 이전 목표 진행 확인	실행 구조 점검(멀티스레드 실행기·스레드 분리)
목표는 수락되는데 피드백이 안 오	feedback_callback 미등록	클라이언트 코드의 send_goal_async 인자 확인	send_goal_async(goal, feedback_callback=...)로 등록
피드백 콜백에서 AttributeError: partial_sequence	다른 자료의 필드 이름을 그대로 사용	ros2 interface show example_interfaces/action/Fibonacci	필드 이름을 sequence로 수정
서버는 완료했는데 결과를 못 받음	result future 미등록	클라이언트 코드에 get_result_async() 호출 여부 확인	수락 확인 후 get_result_async().add_done_callback(...) 등록
취소 요청이 항상 거부됨	cancel_callback 미등록(rclpy 기본값은 거부)	서버 코드의 ActionServer 인자 확인	cancel_callback을 등록하고 CancelResponse.ACCEPT 반환
취소를 수락했는데 실행이 안 멈춤	실행 콜백에 is_cancel_requested 검사 없음	실행 루프 코드 확인	매 단계 취소 검사 후 canceled() 처리
취소 요청이 실행이 끝날 때까지 처리 안 됨	단일 스레드 실행기에서 긴 실행 콜백이 독점	main()의 spin 방식 확인	Python: MultiThreadedExecutor + ReentrantCallbackGroup, C++: 실행 스레드 분리
ModuleNotFoundError: example_interfaces	의존성 누락	package.xml 확인	<depend>example_interfaces</depend> 추가 후 재빌드
서버 종료 후 클라이언트가 계속 대기	서버가 사라져 result future가 완료되지 않음	ros2 action info로 서버 수 확인	서버 재실행, 실무에서는 타임아웃 설계 고려

표 5-3. 액션 구현 시 자주 발생하는 오류와 해결 방법

정리

- 액션은 목표를 수락한 서버가 오랜 작업을 비동기로 수행하며 피드백을 보고하고 결과를 반환하는 통신 방식으로, 도중 취소가 가능하다는 점이 토픽·서비스와 구별되는 핵심입니다.

• 액션 서버는 goal/cancel/execute(수락 후 실행) 콜백으로 역할을 나누며, rclpy는 취소를 기본 거부하므로 취소 콜백 등록과 실행 중 취소 검사가 반드시 필요합니다.

• 긴 실행 콜백이 다른 콜백 처리를 막지 않도록, Python은 MultiThreadedExecutor와 ReentrantCallbackGroup, C++은 실행 스레드 분리를 사용합니다.

• 클라이언트는 goal future로 수락 여부를, goal handle로 결과·취소 요청 창구를, result future로 최종 결과를 얻는 2단계 비동기 구조입니다.

• 2부에서 배운 세 가지 통신 방식은 다음 표와 같이 분업합니다.

기준	토픽	서비스	액션
방향	단방향	요청 → 응답	목표 → (피드백) → 결과
횟수	지속 스트림	호출당 1회	목표당 1회 + 진행 피드백
소요 시간	계속	짧음	김
취소	불가	불가	가능
대표 사례	/scan, /cmd_vel	두 수 더하기, 지도 저장	목적지 이동(Nav2), 팔 동작

표 5-4. 토픽·서비스·액션 비교



그림 5-4. 상황에 맞는 통신 방식을 고르는 기준

핵심 용어

- **액션(action)**: 목표·피드백·결과·취소로 이루어지는, 오래 걸리는 작업용 양방향 통신 방식
- **목표(goal)**: 클라이언트가 서버에 요청하는 작업의 내용
- **피드백(feedback)**: 작업 수행 중 서버가 주기적으로 보내는 진행 상황
- **결과(result)**: 작업 종료 시 서버가 반환하는 최종 데이터와 상태
- **취소(cancel)**: 수행 중인 목표를 중단해 달라는 클라이언트의 요청
- **액션 서버 / 액션 클라이언트**: 목표를 수행하는 쪽 / 목표를 보내고 경과를 받는 쪽
- **목표 핸들(goal handle)**: 접수된 목표를 가리키는 참조로, 피드백 발행·상태 확정(서버)과 결과·취소 요청(클라이언트)의 창구
- **future**: 비동기 호출의 완료 시점에 값이 채워지는 상자. 액션에서는 goal·result·cancel future가 쓰인다
- **실행기(executor)**: 콜백의 실행 스레드와 순서를 관리하는 구성 요소
- **콜백 그룹(callback group)**: 콜백들의 병행 실행 허용 여부를 묶어서 지정하는 단위

연습문제

1. [개념 확인] 10m 전진 명령을 서비스가 아닌 액션으로 만들어야 하는 이유를 세 가지 쓰시오.
2. [개념 확인] goal future와 result future가 각각 무엇을 완료할 때까지 기다리는 상자인지 구분해 설명하시오.
3. [코드 수정] 서버의 `time.sleep(1.0)`을 0.5초로 줄이고, `--feedback` 옵션으로 피드백 간격이 짧아졌는지 확인하시오.
4. [코드 수정] 서버의 `goal_callback`이 `order`가 30을 초과하는 목표도 거부하도록 수정하고, CLI로 `{order: 50}`을 보내 거부되는지 확인하시오.
5. [실행 결과 예상] `timed_cancel_client`로 `order=4`(약 2초 만에 완료)인 목표를

보내면 3초 뒤 취소 요청은 어떤 응답을 받겠는가?

6. [오류 원인 찾기] 어떤 학습자가 서버에서 `cancel_callback`을 지웠더니 취소가 되지 않는다고 합니다. 클라이언트 코드는 그대로다. 원인을 설명하시오.

7. [CLI 실습] `ros2 action send_goal`에 `--feedback`을 붙여 `{order: 12}`를 보내고, 출력에서 목표 ID·피드백·최종 상태를 각각 찾아보시오.

8. [Python·C++ 비교] Python 서버가 `MultiThreadedExecutor`를 쓰는 문제 상황을, C++ 서버는 어떤 방법으로 해결하는지 쓰시오.

9. [통신 방식 선택] 다음 상황에 토픽·서비스·액션 중 적절한 방식을 고르고 근거를 쓰시오. (가) 로봇 팔로 상자를 집어 올리기 (나) 현재 관절 각도의 지속적 보고 (다) 그리퍼(집게)를 즉시 열기/닫기 (라) 충전 스테이션까지 복귀

10. [응용 구현] 클라이언트가 피드백을 받을 때마다 수열의 마지막 값이 1000을 넘으면 즉시 취소를 요청하도록 `timed_cancel_client`를 수정하시오.

연습문제 해설

• 1번 첫째, 완료까지 오래 걸려 응답 대기가 부적절하다. 둘째, 이동 중 진행 상황(피드백)을 알아야 합니다. 셋째, 위험 상황에서 중단(취소)할 수 있어야 합니다.

• 2번 `goal future`는 서버가 목표를 수락/거부했는지의 완료를, `result future`는 수락된 목표의 실행이 끝나 결과가 나왔는지의 완료를 기다립니다.

• 3번 `time.sleep(0.5)`로 수정 후 재빌드. 피드백이 0.5초 간격으로 출력됩니다.

• 4번 `goal_callback`에 `if goal_request.order > 30: return GoalResponse.REJECT` 분기를 추가합니다. CLI 출력에 `Goal was rejected`가 나타납니다.

• 5번 목표가 이미 성공으로 종료된 뒤이므로 취소할 목표가 없습니다. `goals_canceling`이 비어 있어 취소할 목표가 없습니다(이미 완료됨) 로그가 출력됩니다.

• 6번 `rcldpy` 액션 서버는 취소 콜백이 없으면 취소 요청을 기본적으로 거부합니다. 클라이언트의 요청은 도착하지만 서버가 받아들이지 않는 것입니다.

- **7번** Goal accepted with ID: ... 줄, Feedback: 블록들, 마지막 Goal finished with status: SUCCEEDED 줄에서 각각 확인합니다.

- **8번** C++ 서버는 accepted callback에서 실행 함수를 std::thread로 분리해 spin 스레드를 비워 둡니다. 문제(긴 실행의 콜백 독점)는 같고 해결 수단만 다릅니다.

- **9번** (가) 액션 수 초 이상 걸리고 중단 가능해야 합니다. (나) 토픽. 지속 스트림입니다. (다) 서비스. 짧은 일회성 명령과 성공 여부 확인입니다. (라) 액션. 장 시간 이동에 진행 보고와 취소가 필요합니다.

- **10번** feedback_callback에서 partial[-1] > 1000이면 self.cancel_goal()을 호출 하도록 수정합니다. 타이머 대신 데이터 조건으로 취소를 촉발하는 구조로, 센서 값이 위험 수준이면 중단하는 실전 패턴과 같습니다.

다음 장 예고 지금까지는 발행 주기 하나를 바꾸는 데도 코드를 고쳐 다시 빌드 해야 했고, 표준 타입에 없는 데이터는 표현할 방법이 없었습니다. 6장에서는 코드를 고치지 않고 노드의 동작을 바꾸는 파라미터와, 나만의 msg/srv/action 타입을 정의하는 커스텀 인터페이스를 다룹니다. 2부 통신의 마지막 퍼즐입니다.

참고 자료

- ROS 2 Jazzy 공식 문서, Understanding actions:
<https://docs.ros.org/en/jazzy/Concepts/Basic/About-Actions.html>

- ROS 2 Jazzy 공식 문서, Understanding actions (CLI tutorial):
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-CLI-Tools/Understanding-ROS2-Actions/Understanding-ROS2-Actions.html>

- ROS 2 Jazzy 공식 문서, Writing an action server and client (Python):
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Writing-an-Action-Server-Client/Py.html>

- ROS 2 Jazzy 공식 문서, Writing an action server and client (C++):
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Writing-an-Action-Server-Client/C++.html>

ient/Cpp.html

- ROS 2 Jazzy 공식 문서, Creating an action:
<https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Creating-an-Action.html>
- example_interfaces 패키지, Fibonacci.action 정의:
https://github.com/ros2/example_interfaces
- 표윤석·임태훈, 『ROS 2로 시작하는 로봇 프로그래밍』, 루비페이퍼

마무리 점검

- ☐ 액션이 필요한 상황(오래 걸림·경과 보고·취소 기능)을 서비스와 대비해 설명할 수 있습니다.
- ☐ .action 파일의 목표/결과/피드백 3구역 구조를 압니다.
- ☐ Python으로 액션 서버·클라이언트를 작성하고 피드백과 결과를 수신했습니다.
- ☐ C++로 같은 기능을 구현하고 빌드·실행했습니다.
- ☐ goal future, goal handle, result future, cancel future의 역할을 구분할 수 있습니다.
- ☐ 취소가 동작하기 위한 세 가지 조건(취소 콜백 등록, 취소 검사, 실행 구조)을 압니다.
- ☐ `ros2 action list/type/info/send_goal --feedback` 명령으로 액션을 진단할 수 있습니다.

06

파라미터와 커스텀 인터페이스

파라미터를 선언하고 콜백으로 바꾼다.
커스텀 메시지·서비스·액션을 정의해
빌드하고 사용한다.

이 장에서 다루는 내용

- 파라미터 선언·주입·조회의 생명주기와 YAML 파일 활용
- Python/C++ 노드에서 파라미터를 선언하고 콜백으로 동적 변경
- .msg/.srv/.action 파일로 커스텀 인터페이스 패키지 빌드
- 인터페이스 패키지를 다른 패키지에서 의존성으로 사용

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 파라미터(parameter)로 노드를 재컴파일 없이 설정할 수 있습니다.
2. CLI와 YAML 두 가지 방법으로 파라미터를 주입하고 런타임에 변경할 수 있습니다.
3. Python과 C++ 양쪽에서 파라미터를 선언하고 사용할 수 있습니다.
4. 나만의 메시지/서비스/액션 타입(.msg/.srv/.action)을 만들고 빌드할 수 있습니다.

6.1 파라미터란 — 코드를 안 고치고 바꾸기

파라미터는 노드의 **설정값**입니다. 발행 주기, 로봇 이름, 속도 한계처럼 바꿀 수 있는 값을

코드에 박지 않고 밖에서 주입합니다. 같은 노드를 다른 설정으로 재사용할 수 있습니다.

파라미터의 흐름을 그림으로 보겠습니다.

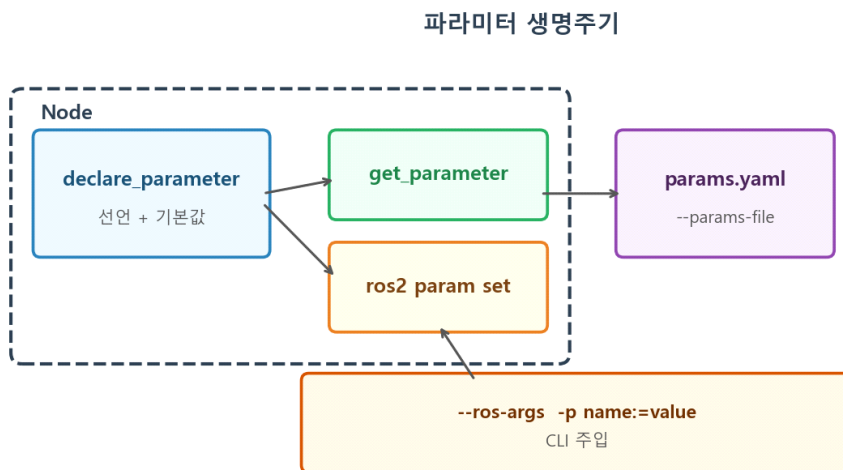


그림 6-1. 파라미터 생명주기

파라미터를 쓰는 흐름은 세 단계입니다.

1. 노드 코드에서 **선언**(declare_parameter) — 이름과 기본값을 정합니다
 2. 실행 시 **주입** — CLI(-p name:=value) 또는 YAML 파일로 기본값을 덮어씁니다
 3. 실행 중 **조회/변경** — ros2 param get/set 으로 런타임에 읽고 바꿉니다
- Jazzy에서 선언하지 않은 파라미터를 set 하면 ParameterNotDeclaredException 이 발생합니다.

반드시 declare_parameter 로 먼저 선언해야 합니다.

6.2 [따라하기] 파라미터 사용 — Python

2장에서 만든 my_py_pkg 에 파라미터를 쓰는 노드를 추가합니다.

① 노드 파일 작성

~/ros2_ws/src/my_py_pkg/my_py_pkg/configurable_publisher.py 를 만듭니다.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int64

class ConfigurablePublisher(Node):
    def __init__(self):
        super().__init__("configurable_publisher")

        self.declare_parameter("publish_period", 1.0)
        self.declare_parameter("robot_name", "turtle")

        period = self.get_parameter("publish_period").value
        name = self.get_parameter("robot_name").value
        self.get_logger().info(f"{name} / 주기 {period}s")

        self.pub_ = self.create_publisher(Int64, "number", 10)
        self.create_timer(period, self.tick)
        self.counter_ = 0

    def tick(self):
        msg = Int64()
        msg.data = self.counter_
        self.pub_.publish(msg)
        self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = ConfigurablePublisher()
    rclpy.spin(node)
    rclpy.shutdown()
```

② setup.py 에 노드 등록

setup.py 의 console_scripts 에 추가합니다.

```
entry_points={
```

```
'console_scripts': [
    'configurable_publisher =
my_py_pkg.configurable_publisher:main',
],
},
```

③ 빌드 및 실행

```
cd ~/ros2_ws
colcon build --packages-select my_py_pkg
source ~/ros2_ws/install/setup.bash
```

기본값으로 실행하면 1초 간격으로 발행합니다.

```
ros2 run my_py_pkg configurable_publisher
```

CLI로 파라미터를 주입하면 기본값을 덮어씁니다.

```
ros2 run my_py_pkg configurable_publisher --ros-args -p
publish_period:=0.5 -p robot_name:=bot
```

④ 런타임 조회와 변경

노드가 실행 중인 상태에서 다른 터미널을 열고 아래를 실행합니다.

새 터미널에서는 `source ~/ros2_ws/install/setup.bash` 를 잊지 않습니다.

```
ros2 param list
ros2 param get /configurable_publisher publish_period
ros2 param set /configurable_publisher robot_name bot2
```

`ros2 param set` 으로 값을 바꿔도, 위 코드처럼 `__init__` 에서 한 번만 읽는 구조면 실제 동작은 바뀌지 않습니다. 런타임 변경을 반영하려면 콜백에서 매번 `get_parameter` 를

호출하거나 `add_on_set_parameters_callback` 을 사용해야 합니다. 이 부분은 6.7절 워크드 예제에서 다룹니다.

6.3 [따라하기] 파라미터 사용 — C++

같은 기능을 C++ 로 만들어 봅니다. Python과 API 구조가 거의 같습니다.

`~/ros2_ws/src/my_cpp_pkg/src/configurable_publisher.cpp` 를 만듭니다.

```

#include <chrono>
#include <cstdint>
#include <functional>
#include <memory>
#include <string>
#include "rclcpp/rclcpp.hpp"
#include "std_msgs/msg/int64.hpp"

class ConfigurablePublisher : public rclcpp::Node
{
public:
    ConfigurablePublisher() : Node("configurable_publisher"),
        counter_(0)
    {
        this->declare_parameter("publish_period", 1.0);
        this->declare_parameter("robot_name", std::string("turtle"));

        double period =
this->get_parameter("publish_period").as_double();
        std::string name =
this->get_parameter("robot_name").as_string();
        RCLCPP_INFO(this->get_logger(), "%s / 주기 %.1fs", name.c_str(),
period);

        pub_ = this->create_publisher<std_msgs::msg::Int64>("number",
10);
        timer_ = this->create_wall_timer(
            std::chrono::milliseconds(static_cast<int>(period * 1000)),
            std::bind(&ConfigurablePublisher::tick, this));
    }

private:
    void tick()
    {
        auto msg = std_msgs::msg::Int64();
        msg.data = counter_++;
        pub_->publish(msg);
    }
}

```

```

    rclcpp::Publisher<std_msgs::msg::Int64>::SharedPtr pub_;
    rclcpp::TimerBase::SharedPtr timer_;
    int64_t counter_;
};

int main(int argc, char **argv)
{
    rclcpp::init(argc, argv);
    rclcpp::spin(std::make_shared<ConfigurablePublisher>());
    rclcpp::shutdown();
    return 0;
}

```

CMakeLists.txt 에 빌드 대상과 의존성을 추가합니다.

```

find_package(std_msgs REQUIRED)

add_executable(configurable_publisher src/configurable_publisher.cpp)
ament_target_dependencies(configurable_publisher rclcpp std_msgs)

install(TARGETS configurable_publisher
        DESTINATION lib/${PROJECT_NAME})

```

package.xml 에 의존성을 추가합니다.

```
<depend>std_msgs</depend>
```

빌드하고 실행합니다.

```

cd ~/ros2_ws
colcon build --packages-select my_cpp_pkg
source ~/ros2_ws/install/setup.bash
ros2 run my_cpp_pkg configurable_publisher --ros-args -p
publish_period:=0.3

```

Python vs C++ 파라미터 API 비교

작업	Python	C++
선언	self.declare_parameter("name", default)	this->declare_parameter("name", default)
읽기	self.get_parameter("name").value	this->get_parameter("name").as_double()
타입	.value 가 자동 추론	.as_double(), .as_string() 등 명시

표 6-1. 파라미터 API의 Python·C++ 대응 관계

6.4 YAML로 파라미터 일괄 주입

파라미터가 많아지면 CLI가 길어집니다. YAML 파일에 한꺼번에 적고 불러오는 것이 실무에서 일반적입니다.

① YAML 파일 작성

~/ros2_ws/src/my_py_pkg/config/params.yaml 을 만듭니다.

```
configurable_publisher:
  ros__parameters:
    publish_period: 0.2
    robot_name: "fast_bot"
```

YAML 최상위 키는 노드 이름과 정확히 일치해야 합니다. 이름이 다르면 파라미터가 적용되지 않습니다. 가장 흔한 실수이므로 `ros2 node list` 로 노드 이름을 먼저 확인합니다.

② YAML로 실행

```
ros2 run my_py_pkg configurable_publisher --ros-args --params-file
~/ros2_ws/src/my_py_pkg/config/params.yaml
```

7장 런치 파일에서 이 YAML을 노드에 자동으로 묶어 띄우는 법을 배웁니다.

Foxy → Jazzy 차이

파라미터 API 자체는 Foxy 와 동일합니다. 다만 Jazzy 에서는 `declare_parameter` 호출 시

타입 힌트(`ParameterDescriptor`)를 넣으면 잘못된 타입의 값이 들어올 때 자동으로 거부합니다. Foxy 에서는 이 검증이 느슨했습니다. 또한 Jazzy 의 기본 동작은 선언되지 않은

파라미터의 `set` 을 거부(`allow_undeclared_parameters=False`)하므로, 반드시 `declare_parameter` 를 먼저 호출해야 합니다.

6.5 커스텀 인터페이스 — 나만의 타입

지금까지는 `std_msgs`, `example_interfaces` 같은 표준 타입을 썼습니다. 실제 프로젝트에서는

도메인에 맞는 타입이 필요합니다. 예를 들어 로봇 하드웨어 상태, 회전 명령, 미로 목표

좌표 같은 것입니다. 이를 위해 인터페이스 전용 패키지를 따로 만드는 것이 관례입니다.

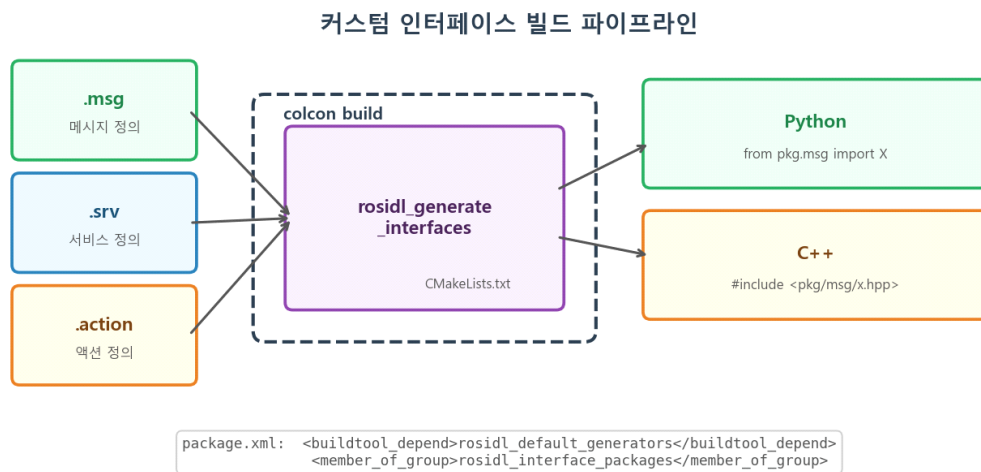


그림 6-2. 커스텀 인터페이스 빌드 파이프라인

`.msg`, `.srv`, `.action` 파일에 필드를 정의하면 `rosidl` 이 Python과 C++ 코드를 자동 생성합니다.

이 과정이 CMake 기반이므로 인터페이스 패키지는 항상 `ament_cmake` 로 만듭니다.

[따라하기] 인터페이스 패키지 생성

① 패키지 뼈대 만들기

```
cd ~/ros2_ws/src
ros2 pkg create my_robot_interfaces --build-type ament_cmake
```

인터페이스 패키지는 **항상** `ament_cmake` 로 만듭니다. Python 프로젝트라 해도

마찬가지입니다. 메시지 코드 생성이 CMake 기반이기 때문입니다.

② 메시지/서비스/액션 폴더와 파일 생성

```
cd my_robot_interfaces
mkdir msg srv action
```

msg/HardwareStatus.msg — 로봇 하드웨어 상태를 담는 메시지

```
float64 temperature
bool are_motors_ready
string debug_message
```

srv/TurnRobot.srv — 회전 명령 서비스 (--- 위가 요청, 아래가 응답)

```
float64 angle_deg
---
bool success
```

action/Navigate.action — 목표 좌표까지 이동하는 액션 (goal / result / feedback)

```
float64 x
float64 y
---
bool reached
---
float64 distance_left
```

③ CMakeLists.txt 설정

CMakeLists.txt 에 아래를 추가합니다.

```
find_package(rosidl_default_generators REQUIRED)

rosidl_generate_interfaces(${PROJECT_NAME}
  "msg/HardwareStatus.msg"
  "srv/TurnRobot.srv"
  "action/Navigate.action"
)
```

액션 타입이 있으면 추가로 find_package(action_msgs REQUIRED) 가 필요할 수 있습니다.

Jazzy 에서는 rosidl_default_generators 가 action_msgs 를 자동으로 끌어오므로

별도 추가 없이 빌드됩니다.

④ package.xml 설정

```
<buildtool_depend>roscpp_default_generators</buildtool_depend>
<exec_depend>roscpp_default_runtime</exec_depend>
<member_of_group>roscpp_interface_packages</member_of_group>
```

이 세 줄이 없으면 빌드가 실패합니다. 각각의 역할은 다음과 같습니다.

태그	역할
buildtool_depend	빌드 시 코드 생성기가 필요함을 선언
exec_depend	실행 시 메시지 런타임 라이브러리가 필요함을 선언
member_of_group	이 패키지가 인터페이스 패키지임을 colcon 에 알립니다

표 6-2. 인터페이스 패키지 package.xml의 핵심 태그

⑤ 빌드 및 확인

```
cd ~/ros2_ws
colcon build --packages-select my_robot_interfaces
source ~/ros2_ws/install/setup.bash
```

빌드가 성공하면 인터페이스를 조회할 수 있습니다.

```
ros2 interface show my_robot_interfaces/msg/HardwareStatus
ros2 interface show my_robot_interfaces/srv/TurnRobot
ros2 interface show my_robot_interfaces/action/Navigate
```

첫 빌드에서 roscpp 코드 생성이 포함되어 30초 이상 걸릴 수 있습니다. 이후에는 변경된 파일만 다시 생성하므로 빠릅니다.

6.6 커스텀 타입 사용하기

다른 패키지에서 커스텀 타입을 쓰려면 의존성을 선언합니다.

인터페이스 패키지 의존 관계

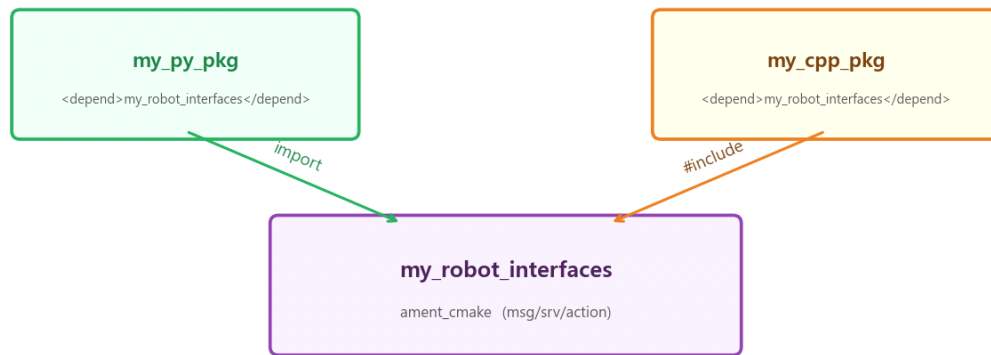


그림 6-3. 인터페이스 패키지 의존 관계

Python 에서 사용

사용할 패키지(my_py_pkg)의 package.xml 에 추가합니다.

```
<depend>my_robot_interfaces</depend>
```

코드에서 import 합니다.

```
from my_robot_interfaces.msg import HardwareStatus
from my_robot_interfaces.srv import TurnRobot
```

C++ 에서 사용

사용할 패키지(my_cpp_pkg)의 package.xml 에 동일하게 추가합니다.

```
<depend>my_robot_interfaces</depend>
```

CMakeLists.txt 에 find_package 를 추가합니다.

```
find_package(my_robot_interfaces REQUIRED)
ament_target_dependencies(노드이름 rclcpp my_robot_interfaces)
```

코드에서 include 합니다.

```
#include "my_robot_interfaces/msg/hardware_status.hpp"
#include "my_robot_interfaces/srv/turn_robot.hpp"
```

C++ 에서 메시지 헤더 경로는 CamelCase 가 아니라 snake_case 입니다.

HardwareStatus → hardware_status.hpp, TurnRobot → turn_robot.hpp

빌드 순서는 colcon 이 package.xml 의 의존성을 읽어 자동 결정합니다.

my_robot_interfaces 가 먼저 빌드되고, 그것을 쓰는 패키지가 뒤에 빌드됩니다.

의존성 선언을 빠뜨리면 빌드 순서가 보장되지 않아 import/include 실패가 발생합니다.

코드 해설 · 실행 확인 · 버전 체크

- **코드 해설 포인트:** declare_parameter 와 get_parameter 의 관계, YAML 키 구조(노드이름 > ros__parameters), rosidl_generate_interfaces 가 .msg 에서 Python/C++ 코드를 생성하는 과정을 해설합니다.
- **실행 확인 포인트:** ros2 param list/get/set, ros2 interface show, ros2 interface list | grep my_robot 으로 결과를 확인합니다.
- **버전/환경 체크:** Jazzy 에서 allow_undeclared_parameters 기본값이 False 인 점, rosidl 코드 생성 시 action_msgs 자동 포함 여부를 확인합니다.

6.7 [워크드 예제] 런타임 파라미터 변경 반영

6.2절의 코드는 __init__ 에서 파라미터를 한 번만 읽었습니다. ros2 param set 으로 값을 바꿔도

실제 동작이 바뀌지 않았습니다. 콜백 안에서 매번 읽도록 바꿔 보겠습니다.

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import Int64

class DynamicPublisher(Node):
    def __init__(self):
        super().__init__("dynamic_publisher")
        self.declare_parameter("publish_period", 1.0)
        self.declare_parameter("max_count", 10)
```

```

period = self.get_parameter("publish_period").value
self.pub_ = self.create_publisher(Int64, "number", 10)
self.timer_ = self.create_timer(period, self.tick)
self.counter_ = 0

def tick(self):
    max_count = self.get_parameter("max_count").value
    if self.counter_ >= max_count:
        self.get_logger().info("최대치 도달 - 발행 중지")
        return
    msg = Int64()
    msg.data = self.counter_
    self.pub_.publish(msg)
    self.get_logger().info(f"발행: {self.counter_}")
    self.counter_ += 1

def main(args=None):
    rclpy.init(args=args)
    node = DynamicPublisher()
    rclpy.spin(node)
    rclpy.shutdown()

ros2 run my_py_pkg dynamic_publisher --ros-args -p max_count:=3

```

0, 1, 2 를 발행한 뒤 멈춘다. 실행 중에 다른 터미널에서 값을 올려 보겠습니다.

```
ros2 param set /dynamic_publisher max_count 10
```

다시 발행이 재개됩니다. tick() 안에서 매번 get_parameter 를 호출하기 때문입니다.

더 효율적인 방법은 add_on_set_parameters_callback 으로 변경 이벤트를 받는 것입니다.

매 콜백마다 get 하는 대신, 파라미터가 바뀔 때만 내부 변수를 갱신합니다.

16장 QoS 와 함께 심화에서 다룹니다.

6.8 트러블슈팅 — 막혔을 때

증상	원인	해결
ros2 interface show 에 안 보임	빌드/소싱 안 함	colcon build 후 source ~/ros2_ws/install/setup.bash
No module named ...msg	사용 패키지에 의존성 누락	package.xml 에 \<depend\>my_robot_interfaces\</depend\> 추가
인터페이스 빌드 실패: rosidl 관련	CMakeLists.txt 설정 누락	find_package(rosidl_default_generators) + rosidl_generate_interfaces 확인
파라미터 미선언 오류	declare_parameter 안 함	사용 전 반드시 선언. Jazzy 는 미선언 파라미터 set 을 거부함
YAML 파라미터가 적용 안 됨	노드 이름 불일치	YAML 최상위 키를 ros2 node list 결과와 일치시킨다
C++ 헤더 못 찾음	snake_case 아님	HardwareStatus → hardware_status.hpp 로 변환
빌드 성공인데 interface list 에 없음	member_of_group 누락	package.xml 에 \<member_of_group\>rosidl_interface_packages\</member_of_group\> 추가
파라미터 set 했는데 동작 안 바뀔	값을 한 번만 읽음	콜백에서 매번 get_parameter 호출 (6.7절 참고)

표 6-3. 파라미터·커스텀 인터페이스에서 자주 발생하는 오류와 해결 방법

6.9 커스텀 인터페이스 — 흔한 실수 체크리스트

인터페이스 패키지는 설정이 까다로워 입문자가 가장 많이 막힙니다. 빌드 실패 시 아래를 점검합니다.

- ☐ 패키지를 `--build-type ament_cmake` 로 만들었나 (Python 패키지로 만들면 안 됨)
- ☐ CMakeLists.txt 에 `find_package(rosidl_default_generators REQUIRED)` 가 있나
- ☐ CMakeLists.txt 에 `rosidl_generate_interfaces` 로 `.msg/.srv/.action` 을 모두 나열했나
- ☐ package.xml 에 `\<buildtool_depend\>rosidl_default_generators\</buildtool_depend\>` 가 있나
- ☐ package.xml 에 `\<exec_depend\>rosidl_default_runtime\</exec_depend\>` 가 있나
- ☐ package.xml 에

\<member_of_group\>roslint_interface_packages\</member_of_group\> 가 있나
☐ 사용하는 패키지의 package.xml 에
 \<depend\>my_robot_interfaces\</depend\> 를 넣었나
☐ .msg 파일의 필드 타입이 올바른가 (float64, bool, string, int64 등 ROS 2
 기본 타입)

정리

- 파라미터는 코드를 고치지 않고 노드의 동작을 바꾸는 장치입니다. 같은 노드를 다른 설정으로 여러 번 재사용할 수 있습니다.
- 파라미터는 `declare_parameter`로 선언하고 `get_parameter`로 읽습니다. Python과 C++ 모두 같은 흐름입니다.
- 값이 많아지면 YAML 파일로 묶어 한 번에 주입합니다. 이때 YAML 최상위 키는 노드 이름과 일치해야 합니다.
- 런타임에 바뀐 값을 반영하려면 콜백에서 매번 다시 읽거나 파라미터 콜백을 등록해야 합니다. 선언 시점에 한 번만 읽으면 변경이 반영되지 않습니다.
- 표준 타입으로 표현되지 않는 데이터는 `.msg`·`.srv`·`.action`으로 직접 정의합니다. 인터페이스는 반드시 `ament_cmake` 패키지에 두어야 합니다.
- 커스텀 타입을 정의한 뒤에는 빌드하고 소싱해야 다른 패키지에서 쓸 수 있습니다. C++에서는 헤더 경로가 `snake_case`로 바뀝니다.

핵심 용어

- **파라미터(parameter)**: 노드가 실행될 때 외부에서 주입받는 설정값. 코드 수정 없이 동작을 바꿉니다.
- **declare_parameter**: 노드가 사용할 파라미터의 이름과 기본값을 선언하는 함수.
- **파라미터 YAML**: 여러 파라미터를 파일 하나로 묶어 주입하는 설정 파일. 최

상위 키가 노드 이름입니다.

- **커스텀 인터페이스**: 표준 타입으로 부족할 때 직접 정의하는 메시지·서비스·액션 타입.
- **인터페이스 패키지**: `.msg`·`.srv`·`.action` 정의를 모아 두는 `ament_cmake` 패키지. 코드 생성기가 CMake 기반이라 `ament_python`으로는 만들 수 없습니다.
- **rosidl**: 인터페이스 정의를 각 언어의 코드로 변환해 주는 ROS 2의 코드 생성기.

다음 장 예고 – 7장 **런치**. 지금까지 노드를 하나씩 손으로 썼지만, 실제 시스템은 노드가 수십 개입니다. 런치 파일로 한 번에 띄우고 파라미터 YAML까지 묶습니다.

연습문제

1. `configurable_publisher` 에 `max_count` 파라미터를 추가해 그 수까지만 발행하고 멈춰라.
2. `HardwareStatus` 메시지를 발행하는 노드를 만들어 `ros2 topic echo` 로 확인하시오.
3. `TurnRobot.srv` 를 사용하는 서비스 서버를 작성하라 (각도를 받아 `success` 반환).
4. `params.yaml` 에 두 노드의 설정을 함께 적고 런치 없이 각각 불러와 보라.
5. (생각해보기) 인터페이스 패키지를 `ament_python` 으로 만들 수 없는 이유는 무엇인가

연습문제 해설

- **1번** 6.7절 `DynamicPublisher` 가 정답. `tick()` 안에서 매번 `max_count` 를 읽어 비교합니다.
- **2번** `HardwareStatus` 메시지를 채워 발행 후 `ros2 topic echo /status` 로 확인

합니다.

필드는 `temperature`, `are_motors_ready`, `debug_message` 세 개입니다.

- **3번** `TurnRobot.srv` 로 서버 작성: 콜백에서 `request.angle_deg` 를 받아 처리하고

`response.success = True` 반환합니다. 4장 서비스 서버 구조 그대로입니다.

- **4번** YAML 최상위 키를 각 노드 이름과 일치시켜야 파라미터가 적용됩니다. 노드 이름 불일치가 가장 흔한 실수입니다.

- **5번** `rosidl` 코드 생성기가 CMake 기반이기 때문입니다. `ament_python` 에는 `CMakeLists.txt` 가

없으므로 `rosidl_generate_interfaces` 를 호출할 수 없습니다.

참고 자료

- ROS 2 Jazzy — Using parameters in a class:
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Using-Parameters-In-A-Class-Python.html>

- ROS 2 Jazzy — Creating custom msg and srv files:
<https://docs.ros.org/en/jazzy/Tutorials/Beginner-Client-Libraries/Custom-ROS2-Interfaces.html>

- ROS 2 Jazzy — Parameter API reference:
<https://docs.ros.org/en/jazzy/Concepts/Basic/About-Parameters.html>

마무리 점검

☐ 파라미터로 노드를 외부에서 설정하고 런타임에 변경할 수 있습니다.

☐ Python과 C++ 양쪽에서 `declare_parameter` / `get_parameter` 를 쓸 수 있습니다.

☐ YAML로 파라미터를 일괄 주입할 수 있습니다.

- ☐ `.msg/.srv/.action` 을 정의하고 인터페이스 패키지를 빌드했습니다.
- ☐ 커스텀 타입을 다른 패키지에서 `import/include` 해 사용할 수 있습니다.
- ☐ C++ 에서 메시지 헤더 경로가 `snake_case` 임을 압니다.

07

런치 시스템

여러 노드를 한 번에 실행한다.
파이썬 런치 파일을 작성하고
리매핑과 인자 전달을 다룬다.

지금까지는 노드를 하나 켤 때마다 터미널을 하나씩 열었습니다. 3장에서 퍼블리셔와 서브스크라이버를 띄울 때도, 4장에서 서비스 서버와 클라이언트를 확인할 때도 창을 두 개씩 벌려 놓고 명령을 따로 입력했습니다. 노드가 두세 개일 때는 그럭저럭 견딜 만합니다.

그런데 실제 로봇은 그렇게 단순하지 않습니다. 카메라 드라이버, 라이다 드라이버, 센서 융합, 좌표 변환, 경로 계획, 모터 제어까지 수십 개의 노드가 동시에 뛰어야 로봇 하나가 움직입니다. 터미널을 스무 개 열어 놓고 순서대로 명령을 치는 개발자는 없습니다. 노드 하나만 빠뜨려도 시스템 전체가 멈추는데, 그걸 매번 손으로 채길 수는 없으니까요.

런치 시스템은 바로 이 문제를 해결합니다. 어떤 노드들을 어떤 설정으로 한꺼번에 실행할지를 파일 하나에 적어 두고, 명령 한 줄로 전체를 띄웁니다. 이 장에서는 파이썬 런치 파일을 기본으로 여러 노드를 묶어 실행하고, 이름 변경과 토픽 리매핑, 파라미터 주입, 실행 인자, 그리고 런치끼리 조합하는 방법까지 차례로 익힙니다.

이 장에서 다루는 내용

- * 노드를 하나씩 실행하는 방식의 한계와 런치가 필요한 이유
- * 파이썬 런치 파일로 여러 노드를 한 번에 실행하는 방법
- * 노드 이름 변경, 토픽 리매핑, 네임스페이스, 파라미터 주입
- * 실행 시점에 값을 받는 런치 인자와 조건부 실행
- * 런치 파일끼리 포함해 조합하는 실무 패턴
- * Foxy 기준 런치를 Jazzy로 옮길 때 점검할 차이

학습 목표

이 장의 학습 목표는 다음과 같습니다.

1. 런치가 필요한 이유를 다수 노드 일괄 실행 관점에서 설명할 수 있습니다.

2. 파이썬 런치 파일을 작성해 파이썬 노드와 C++ 노드를 함께 띄울 수 있습니다.
3. 노드 이름, 리매핑, 네임스페이스, 파라미터를 런치에서 설정할 수 있습니다.
4. 런치 인자를 선언하고 실행 시점에 값을 전달하며, 조건에 따라 노드를 켜고 끌 수 있습니다.
5. 런치 파일을 포함해 큰 런치를 조립하고, 파라미터 YAML까지 설치 규칙에 맞게 묶을 수 있습니다.

7.1 런치란 무엇인가

터미널 여러 개의 한계

노드 하나당 터미널 하나를 여는 방식은 노드 수가 늘수록 감당하기 어렵습니다. 터미널을 여럿 띄우고, 각 창에서 소싱하고, 순서대로 명령을 입력하는 과정에서 실수가 생깁니다. 어느 창에서 소싱을 빠뜨렸는지, 어느 노드가 아직 안 떴는지 눈으로 확인해야 합니다. 게다가 파라미터를 바꿔 다시 실행하려면 그 과정을 처음부터 반복해야 합니다.

이 방식은 재현성도 떨어집니다. 동료에게 시스템을 넘길 때 터미널 다섯 개에서 어떤 명령을 어떤 순서로 쳤는지 말로 설명해야 한다면, 그 설명이 곧 버그의 원인이 됩니다.

런치가 하는 일

런치는 실행할 노드 목록과 각 노드의 설정을 파일 하나에 적어 둡니다. 그러면 명령 한 줄로 전체가 뜹니다. 실행 구성이 파일로 남으니 재현도 되고, Git으로 버전 관리도 되고, 동료에게 넘길 때도 파일 하나면 끝납니다.

정의(1) 런치

실행할 노드와 그 설정을 파일에 기술해 한 번에 실행하는 ROS 2의 실행 관리 도구. 노드 목록뿐 아니라 이름 변경, 토픽 리매핑, 파라미터 주입, 실행 인자, 다른 런치 파일 포함까지 한 곳에서 다룹니다.

런치가 다루는 범위는 생각보다 넓습니다. 단순히 노드를 여러 개 켜는 것을 넘어, 같은 노드를 이름만 바꿔 여러 대 띄우거나, 실행할 때 받은 값에 따라 노드 구성을 바꾸거나, 다른 팀이 만든 런치 파일을 통째로 가져다 쓰는 일까지 모두 런치의 몫입니다.

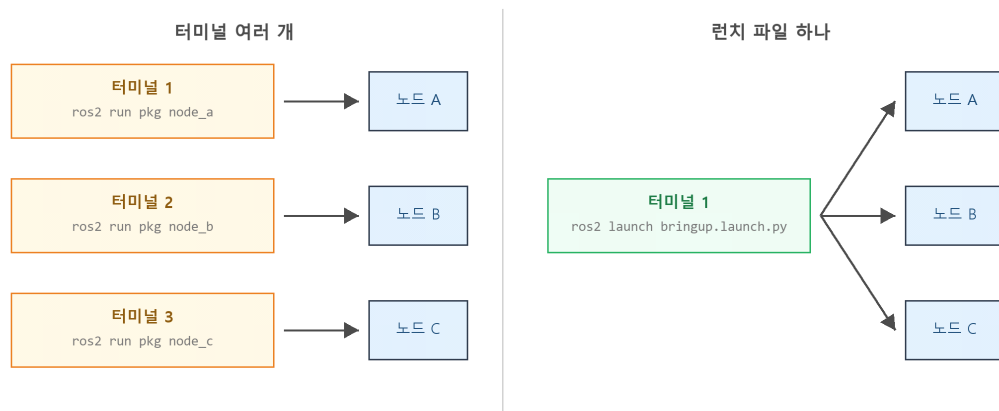


그림 7-1. 터미널 여러 개로 노드를 하나씩 띄우는 방식과 런치 파일 하나로 전체를 띄우는 방식의 비교

세 가지 런치 형식

ROS 2는 세 가지 런치 형식을 지원합니다.

형식	장점	단점
파이썬(.launch.py)	조건·반복·경로 계산 등 로직 표현	문법이 다소 장황
XML(.launch.xml)	간결, 정적 구성에 적합	동적 로직 불가
YAML(.launch.yaml)	가장 간결	동적 로직 불가

표 7-1. 세 가지 런치 형식 비교

XML은 확실히 간결합니다. 같은 노드 두 개를 띄우는 런치를 XML로 쓰면 다음과 같습니다.

```
<launch>
  <node pkg="my_py_pkg" exec="number_publisher"/>
  <node pkg="my_py_pkg" exec="number_subscriber"/>
</launch>
```

파이썬 버전보다 훨씬 짧습니다. 구성이 고정되어 있고 변할 일이 없다면 XML이 읽기도 편합니다.

왜 파이썬 런치인가

그럼에도 이 책은 파이썬 런치를 기본으로 합니다. 로봇 개발에서는 실행 인자에

따라 노드 구성이 달라지거나, 패키지 설치 경로를 계산해야 하거나, 시뮬레이션이면 A를 켜고 실제 로봇이면 B를 켜는 분기가 흔하기 때문입니다. 이런 로직은 XML이나 YAML로 표현할 수 없습니다.

파이썬 런치는 결국 파이썬 코드입니다. `if` 문을 쓸 수 있고, 반복문으로 노드 열개를 찍어 낼 수 있고, `os.path.join`으로 경로를 만들 수 있습니다. 8장 이후 시뮬레이터를 다루거나 실제 로봇을 붙일 때 이 유연함이 필요해집니다.

7.2 파이썬 런치 파일 작성

bringup 패키지 만들기

런치 파일은 보통 실행 구성만 모아 두는 별도의 패키지에 둡니다. 이런 패키지를 관례적으로 `bringup` 패키지라고 부릅니다. 노드 코드는 `my_py_pkg`나 `my_cpp_pkg`에 두고, 그것들을 어떻게 조립해 띄울지는 `bringup`이 담당하는 구조입니다.

```
cd ~/ros2_ws/src
ros2 pkg create my_robot_bringup --build-type ament_cmake
cd my_robot_bringup && mkdir launch
```

`bringup` 패키지는 파이썬 노드가 없어도 `ament_cmake`로 만듭니다. 런치와 설정 파일만 설치하면 되기 때문입니다.

첫 런치 파일

`launch/number_app.launch.py` 파일을 만들고 다음을 작성합니다. 3장에서 만든 노드 두 개를 한 번에 띄우는 구성입니다.

```
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
```

```
ld = LaunchDescription()

publisher = Node(
    package="my_py_pkg",
    executable="number_publisher",
)

subscriber = Node(
    package="my_py_pkg",
    executable="number_subscriber",
)

ld.add_action(publisher)
ld.add_action(subscriber)
return ld
```

코드를 한 줄씩 보겠습니다.

`generate_launch_description` 함수가 런치의 진입점입니다. ROS 2는 런치 파일을 실행할 때 이 이름의 함수를 찾아 호출하므로, 함수 이름을 바꾸면 안 됩니다.

`LaunchDescription`은 실행할 동작들을 담는 그릇입니다. 여기에 담긴 것들이 실행 대상이 됩니다.

`Node`는 노드 하나를 실행하는 동작입니다. `package`는 패키지 이름, `executable`은 `setup.py`의 `console_scripts`에 등록한 실행 파일 이름입니다. 3장에서 `ros2 run my_py_pkg number_publisher`로 실행했던 그 이름 그대로입니다.

`add_action`으로 담고 `return`하면 끝입니다. 담긴 순서대로 실행되지만, 런치는 노드가 완전히 뜰 때까지 기다렸다가 다음을 실행하지 않습니다. 거의 동시에 띄운다고 보면 됩니다.

설치 규칙 추가

런치 파일이 `install` 폴더에 설치되도록 `CMakeLists.txt`에 규칙을 넣습니다.

```
install(DIRECTORY launch DESTINATION share/${PROJECT_NAME})
```

이 한 줄을 빠뜨리는 실수가 대단히 흔합니다. 빌드는 성공하는데 `ros2 launch`가 파일을 못 찾는다면 십중팔구 이것입니다. 런치 파일은 소스 폴더가 아니라 설치된

share 폴더에서 읽히기 때문입니다.

빌드와 실행

빌드하고 소싱한 뒤 런치를 실행하면 노드 둘이 한 번에 뜹니다.

```
cd ~/ros2_ws && colcon build --packages-select my_robot_bringup
source install/setup.bash
ros2 launch my_robot_bringup number_app.launch.py
```

터미널 하나에 두 노드의 로그가 섞여서 올라옵니다. 터미널 두 개를 열 필요가 없어졌습니다. 종료는 **Ctrl+C** 한 번이면 런치가 띄운 노드가 모두 함께 정리됩니다. 이것도 런치의 장점입니다. 노드를 하나씩 띄웠다면 창마다 돌아다니며 꺼야 했을 테니까요.

C++ 노드도 함께 띄우기

런치는 노드가 어떤 언어로 작성되었는지 신경 쓰지 않습니다. 파이썬 노드든 C++ 노드든 **Node** 액션에 패키지명과 실행 파일 이름만 알려 주면 됩니다.

```
py_publisher = Node(
    package="my_py_pkg",
    executable="number_publisher",
)
cpp_subscriber = Node(
    package="my_cpp_pkg",
    executable="number_subscriber",
)
```

파이썬으로 발행하고 C++로 구독하는 구성이 이렇게 간단히 만들어집니다. 3장에서 두 언어가 같은 토픽으로 문제없이 통신하는 것을 확인했는데, 런치는 그 두 노드를 한 파일에서 나란히 다룹니다. 언어가 섞인 시스템을 한 줄로 띄울 수 있다는 뜻입니다.

ros2 launch 명령 옵션

런치를 실행하고 점검할 때 쓰는 명령을 정리하면 다음과 같습니다.

명령	설명
ros2 launch <pkg> <file>	런치 실행
ros2 launch <pkg> <file> -d	디버그 로그와 함께 실행
ros2 launch <pkg> <file> --show-args	이 런치가 받는 인자 목록 확인
ros2 launch <pkg> <file> <arg>:=<값>	인자에 값을 넘겨 실행

표 7-2. 자주 쓰는 ros2 launch 명령

`--show-args`는 남이 만든 런치를 쓸 때 특히 유용합니다. 어떤 인자를 받는지, 기본값이 무엇인지 코드를 열지 않고 확인할 수 있습니다.

7.3 노드 구성과 리매핑

런치는 노드를 조립하는 곳입니다. 같은 노드라도 이름을 바꾸고, 토픽을 갈아 끼우고, 파라미터를 넣어 전혀 다른 역할을 맡길 수 있습니다. 노드 코드는 그대로 두고 바깥에서 구성만 바꾸는 것이 핵심입니다.

노드 이름 바꾸기

```
publisher = Node(
    package="my_py_pkg",
    executable="number_publisher",
    name="my_number_publisher",
)
```

`name`은 노드 이름을 실행 시점에 바꿉니다. 노드 코드 안에서 `super().__init__("number_publisher")`로 지은 이름을 덮어씁니다. 같은 실행 파일을 이름만 달리해 여러 개 띄울 때 반드시 필요합니다. ROS 2는 같은 이름의 노드가 둘 이상 뜨는 것을 막지 않지만, 그렇게 되면 어느 노드에 명령이 갔는지 알 수 없어집니다.

토픽 리매핑

```
publisher = Node(
    package="my_py_pkg",
    executable="number_publisher",
    remappings=[("number", "my_number")],
)
```

remappings는 노드가 쓰는 토픽 이름을 실행 시점에 바꿉니다. 코드에서는 **number**로 발행하도록 짜 두었지만, 실행할 때 **my_number**로 내보내라고 지시하는 것입니다.

정의(2) 리매핑

노드 코드를 고치지 않고, 노드가 사용하는 토픽·서비스·액션의 이름을 실행 시점에 다른 이름으로 바꾸는 것.

리매핑이 왜 중요한지는 남의 패키지를 쓸 때 드러납니다. 어떤 드라이버가 **/scan**으로 라이다 데이터를 내보내는데 우리 시스템은 **/lidar_front**를 기대한다면, 드라이버 소스를 고칠 게 아니라 런치에서 리매핑 한 줄로 맞추면 됩니다.

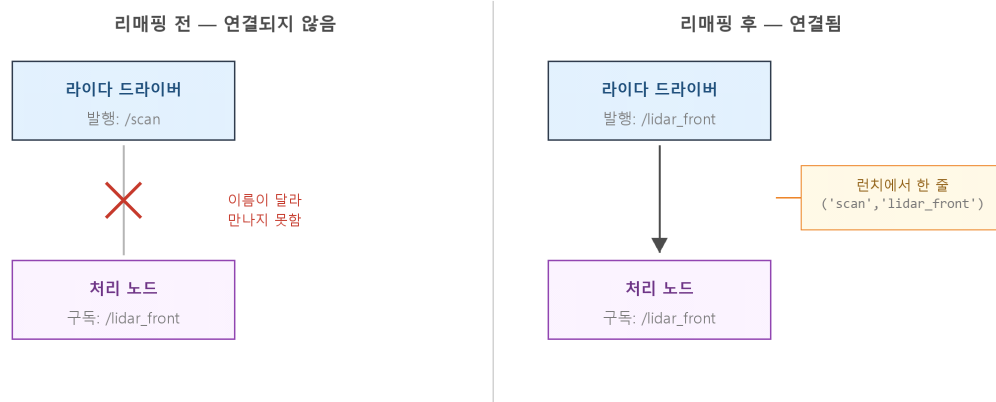


그림 7-2. 리매핑 전후의 토픽 연결 변화

파라미터 주입

```
publisher = Node(
    package="my_py_pkg",
    executable="configurable_publisher",
    parameters=[{"publish_period": 0.5, "number": 7}],
)
```

`parameters`는 6장에서 선언한 파라미터에 값을 넣습니다. 딕셔너리로 직접 적을 수도 있고, 뒤에서 볼 YAML 파일을 통째로 물릴 수도 있습니다.

네임스페이스로 여러 대 띄우기

로봇을 두 대 띄운다고 해 봅시다. 이름만 바꾸면 노드 이름은 갈라지지만 토픽은 여전히 `/number` 하나로 겹칩니다. 두 로봇이 같은 토픽에 뒤섞여 발행하게 됩니다.

이때 쓰는 것이 네임스페이스입니다. 네임스페이스를 주면 그 노드의 모든 토픽 앞에 접두어가 붙습니다.

```
robot1 = Node(
    package="my_py_pkg",
    executable="number_publisher",
    namespace="robot1",
)
robot2 = Node(
    package="my_py_pkg",
    executable="number_publisher",
    namespace="robot2",
)
```

이렇게 하면 토픽이 `/robot1/number`와 `/robot2/number`로 갈라집니다. 노드 이름도 `/robot1/number_publisher`가 됩니다. 리매핑을 토픽마다 일일이 적을 필요 없이, 한 노드가 쓰는 모든 이름이 통째로 한 칸 안으로 들어갑니다.

정의(3) 네임스페이스

노드와 그 노드가 쓰는 토픽·서비스 이름 앞에 공통 접두어를 붙여 이름 충돌을 막는 구획. 같은 노드를 여러 대 띄울 때 각각을 독립된 이름 공간에 놓습니다.

실행한 뒤 `ros2 topic list`로 확인하면 갈라진 것이 보입니다.

```
ros2 topic list
/robot1/number
/robot2/number
```

GroupAction으로 묶기

노드가 여러 개인 그룹 전체에 같은 네임스페이스를 씌우고 싶을 때가 있습니다. 로봇 한 대가 퍼블리셔와 서브스크라이버 두 노드로 이루어져 있다면, 두 노드 모두 **robot1** 안에 들어가야 합니다.

GroupAction과 **PushRosNamespace**를 쓰면 묶어서 처리할 수 있습니다.

```
from launch import LaunchDescription
from launch.actions import GroupAction
from launch_ros.actions import Node, PushRosNamespace

def generate_launch_description():
    robot1 = GroupAction([
        PushRosNamespace("robot1"),
        Node(package="my_py_pkg", executable="number_publisher"),
        Node(package="my_py_pkg", executable="number_subscriber"),
    ])
    return LaunchDescription([robot1])
```

PushRosNamespace는 그 뒤에 오는 그룹 안의 모든 노드에 네임스페이스를 씌웁니다. 노드마다 **namespace=**를 적는 것보다 훨씬 깔끔하고, 로봇을 세 대로 늘릴 때 그룹만 복사하면 됩니다.

7.4 파라미터와 실행 인자 전달

런치 인자 선언과 사용

지금까지는 값을 런치 파일 안에 적어 두었습니다. 그런데 로봇 이름이나 발행 주기처럼 실행할 때마다 달라질 수 있는 값을 파일에 고정해 두면, 값 하나 바꾸려고 파일을 열어 고쳐야 합니다.

런치 인자는 실행 시점에 값을 밖에서 받는 장치입니다.

```
from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
```

```

from launch_ros.actions import Node

def generate_launch_description():
    robot_name = LaunchConfiguration("robot_name")
    return LaunchDescription([
        DeclareLaunchArgument("robot_name", default_value="turtle"),
        Node(
            package="my_py_pkg",
            executable="configurable_publisher",
            parameters=[{"robot_name": robot_name}],
        ),
    ])

```

`DeclareLaunchArgument`는 이 런치가 `robot_name`이라는 인자를 받는다고 선언하고 기본값을 정합니다. `--show-args`로 확인할 때 보이는 것이 이 선언입니다.

`LaunchConfiguration`은 선언된 인자의 값을 가져오는 자리표입니다. 여기서 주의할 점이 있습니다. `LaunchConfiguration("robot_name")`은 값 자체가 아니라 나중에 값으로 바뀔 자리표입니다. 런치 파일이 파이썬 코드로 읽히는 시점에는 아직 값이 정해지지 않았고, 실제 실행 시점에 채워집니다. 그래서 이 값을 `if`로 비교하거나 문자열로 이어 붙이려 하면 뜻대로 되지 않습니다.

실행 시점에 값 넘기기

인자 이름과 값을 콜론 등호로 넘기면 기본값을 덮어씁니다.

```
ros2 launch my_robot_bringup number_app.launch.py robot_name:=racer
```

값을 안 넘기면 선언해 둔 기본값 `turtle`이 쓰입니다. 어떤 인자를 받는지 잊었다면 다음으로 확인합니다.

```
ros2 launch my_robot_bringup number_app.launch.py --show-args
```

파라미터 YAML 파일 물리기

파라미터가 서너 개를 넘어가면 런치 파일에 덕서너리로 나열하는 것이 지저분해 집니다. 이럴 때는 YAML 파일로 빼고 통째로 물립니다.

```
import os
from ament_index_python.packages import get_package_share_directory

config = os.path.join(
    get_package_share_directory("my_robot_bringup"), "config",
    "params.yaml")

publisher = Node(
    package="my_py_pkg", executable="configurable_publisher",
    parameters=[config],
)
```

`get_package_share_directory`는 설치된 패키지의 share 폴더 경로를 찾아 줍니다. 소스 폴더 경로를 직접 적으면 안 됩니다. 다른 사람 컴퓨터에서는 그 경로가 없기 때문입니다.

조건부 실행

시뮬레이션에서는 시뮬레이터 브리지를 켜고, 실제 로봇에서는 드라이버를 켜야 한다면 어떻게 할까요. 런치 인자로 받은 값에 따라 노드를 켜고 끄면 됩니다.

```
from launch.conditions import IfCondition, UnlessCondition

use_sim = LaunchConfiguration("use_sim")

sim_node = Node(
    package="my_py_pkg", executable="number_publisher",
    condition=IfCondition(use_sim),
)
real_node = Node(
    package="my_cpp_pkg", executable="number_publisher",
    condition=UnlessCondition(use_sim),
)
```

`IfCondition`은 값이 참일 때만, `UnlessCondition`은 거짓일 때만 그 노드를 실행합니다. 앞서 `LaunchConfiguration`은 자리표라서 파이썬 `if`로 비교할 수 없다고 했는데, 그 대신 쓰라고 마련된 것이 이 조건 클래스들입니다.

```
ros2 launch my_robot_bringup app.launch.py use_sim:=true
```

[워크드 예제] 파라미터 YAML까지 묶은 런치

실무 런치는 보통 노드 여러 개, 각 노드의 파라미터 YAML, 리매핑을 한 파일에 모읍니다. bringup 패키지에 **config** 폴더를 두고 YAML을 작성합니다.

config/number.yaml 파일은 다음과 같습니다.

```
configurable_publisher:
  ros__parameters:
    publish_period: 0.3
    robot_name: bringup_bot
```

YAML 최상위 키는 노드 이름과 일치해야 합니다. 이름이 다르면 값이 조용히 무시됩니다. 런치에서 **name="my_publisher"**로 이름을 바꿨다면 YAML의 키도 **my_publisher**여야 합니다. 파라미터가 안 먹는다는 문의의 상당수가 이 불일치입니다.

launch/full_app.launch.py에서 이 YAML을 물립니다.

```
import os
from launch import LaunchDescription
from launch_ros.actions import Node
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    cfg = os.path.join(
        get_package_share_directory("my_robot_bringup"), "config",
        "number.yaml")
    return LaunchDescription([
        Node(package="my_py_pkg", executable="configurable_publisher",
            parameters=[cfg]),
        Node(package="my_py_pkg", executable="number_subscriber",
            remappings=[("number", "number")]),
    ])
```

CMakeLists.txt에서는 launch와 config를 함께 설치합니다.

```
install(DIRECTORY launch config DESTINATION share/${PROJECT_NAME})
```

config 폴더 설치를 빠뜨리는 실수가 흔합니다. 런치는 뜨는데 파라미터만 기본값

으로 도는 증상이 나오면 이것부터 확인하세요.

7.5 런치 조합하기

런치가 런치를 부릅니다

시스템이 커지면 런치 파일 하나가 수백 줄이 됩니다. 센서 런치, 구동 런치, 내비게이션 런치를 한 파일에 다 쓰면 읽기도 고치기도 어렵습니다.

ROS 2는 런치 파일이 다른 런치 파일을 포함할 수 있게 해 줍니다. 기능별로 작은 런치를 만들어 두고, 최상위 런치가 그것들을 불러 조립하는 구조입니다. 실제 로봇 프로젝트의 bringup은 거의 이 모양입니다.

IncludeLaunchDescription

```
import os
from launch import LaunchDescription
from launch.actions import IncludeLaunchDescription
from launch.launch_description_sources import
PythonLaunchDescriptionSource
from ament_index_python.packages import get_package_share_directory

def generate_launch_description():
    bringup = get_package_share_directory("my_robot_bringup")

    number_app = IncludeLaunchDescription(
        PythonLaunchDescriptionSource(
            os.path.join(bringup, "launch", "number_app.launch.py")
        )
    )
    return LaunchDescription([number_app])
```

`IncludeLaunchDescription`이 다른 런치를 통째로 가져옵니다. `PythonLaunchDescriptionSource`는 가져올 대상이 파이썬 런치라는 뜻입니다.

XML 런치를 포함할 때는 `XMLLaunchDescriptionSource`를 씁니다.

인자를 넘기며 포함하기

포함하는 런치가 인자를 받는다면, 값을 함께 넘길 수 있습니다.

```
number_app = IncludeLaunchDescription(  
    PythonLaunchDescriptionSource(  
        os.path.join(bringup, "launch", "number_app.launch.py")  
    ),  
    launch_arguments={"robot_name": "racer"}.items(),  
)
```

`launch_arguments`는 딕셔너리를 `.items()`로 넘겨야 합니다. 이 부분은 문법이 특이해서 자주 틀리는 곳입니다.

여기에 앞서 배운 `GroupAction`과 `PushRosNamespace`를 얹으면, 같은 런치를 네임스페이스만 달리해 두 번 포함할 수 있습니다. 로봇 두 대를 띄우는 시스템이 이렇게 완성됩니다.

```
robot1 = GroupAction([  
    PushRosNamespace("robot1"),  
    IncludeLaunchDescription(  
        PythonLaunchDescriptionSource(  
            os.path.join(bringup, "launch",  
"number_app.launch.py"))),  
    ])  
robot2 = GroupAction([  
    PushRosNamespace("robot2"),  
    IncludeLaunchDescription(  
        PythonLaunchDescriptionSource(  
            os.path.join(bringup, "launch",  
"number_app.launch.py"))),  
    ])
```

런치 파일 하나를 재사용해 독립된 로봇 두 대를 띄웠습니다. 코드 줄도 고치지 않았습다.

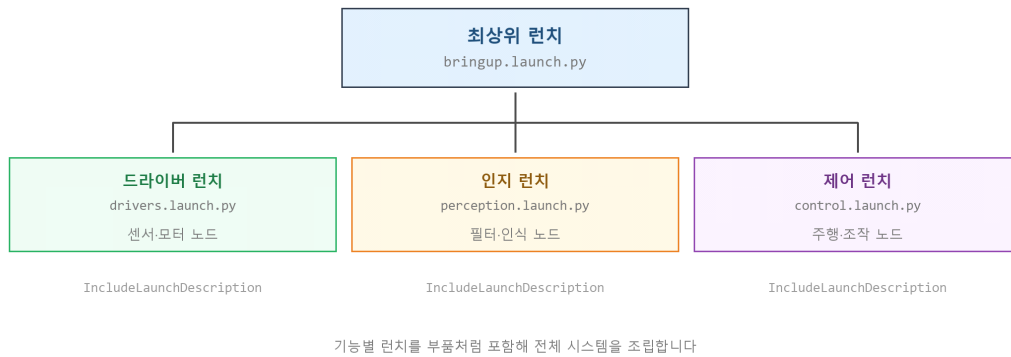


그림 7-3. 최상위 런치가 기능별 런치를 포함해 조립하는 구조

7.6 [미니 실습] 런치 하나로 시스템 띄우기

지금까지 배운 것을 하나로 묶어 봅니다. 퍼블리셔와 서브스크라이버를 네임스페이스로 갈라 두 벌 띄우고, 발행 주기는 런치 인자로 받는 구성입니다.

1단계. `my_robot_bringup/launch/two_robots.launch.py` 파일을 만들고 다음을 작성합니다.

```

from launch import LaunchDescription
from launch.actions import DeclareLaunchArgument, GroupAction
from launch.substitutions import LaunchConfiguration
from launch_ros.actions import Node, PushRosNamespace

def generate_launch_description():
    period = LaunchConfiguration("period")

    def robot(ns):
        return GroupAction([
            PushRosNamespace(ns),
            Node(package="my_py_pkg",
executable="configurable_publisher",
parameters=[{"publish_period": period}]),
            Node(package="my_py_pkg", executable="number_subscriber"),

```

```

    ])

    return LaunchDescription([
        DeclareLaunchArgument("period", default_value="1.0"),
        robot("robot1"),
        robot("robot2"),
    ])

```

2단계. 빌드하고 소싱합니다.

```

cd ~/ros2_ws && colcon build --packages-select my_robot_bringup
source install/setup.bash

```

3단계. 인자를 넘겨 실행합니다.

```

ros2 launch my_robot_bringup two_robots.launch.py period:=0.3

```

4단계. 다른 터미널에서 토픽이 갈라졌는지 확인합니다.

```

ros2 topic list

/robot1/number
/robot2/number

```

노드 네 개가 한 명령으로 떴고, 두 로봇의 토픽이 섞이지 않고 갈라졌으며, 발행 주기는 실행할 때 넘긴 0.3이 적용되었습니다. 파이썬 함수로 로봇 구성을 짰다는 점도 눈여겨보세요. XML이나 YAML로는 이렇게 못 합니다.

7.7 Foxy에서 Jazzy로 옮길 때

런치는 ROS 2 배포판이 바뀔 때 변화가 가장 큰 영역입니다. 인터넷의 예전 자료 상당수가 Foxy를 기준으로 작성되었는데, 그 런치 파일을 Jazzy에서 그대로 실행하면 특히 시뮬레이터 관련 부분이 동작하지 않습니다. 옮길 때 다음을 점검합니다.

시뮬레이터 실행 방식이 완전히 달라졌습니다. Foxy는 Gazebo Classic을 런치에서 직접 불렀습니다(`gzserver`, `gzclient`, `spawn_entity.py`). Jazzy는 최신 Gazebo를 `ros_gz_sim`으로 띄웁니다. Classic 문법으로 짠 런치는 그대로 돌지 않으므로, `ros_gz_sim`의 실행 런치를 포함하고 `create`로 로봇을 스폰하도록 다시 써야 합니다.

다.

브릿지가 바뀌었습니다. Classic은 `gazebo_ros` 플러그인으로 ROS와 시뮬레이터를 연결했지만, 최신 Gazebo는 `ros_gz_bridge`로 토픽을 잇습니다. 런치에 브리지 노드가 하나 더 들어갑니다.

임포트 경로를 정리하세요. substitution과 패키지 경로 API는 대체로 호환되지만, 더 이상 쓰지 않는 임포트 경로가 남아 있으면 경고가 뜹니다. 최신 `launch`와 `launch_ros` 임포트로 맞춰 두는 편이 안전합니다.

정리하면, 예전 런치는 참고만 하고 처음부터 다시 쓴다는 생각으로 접근하는 편이 빠릅니다. 런치는 배포판을 타는 영역이라, 검증된 Jazzy 구조와 대조하며 옮기는 것이 결국 지름길입니다.

자주 만나는 오류와 해결

증상	원인	해결
file not found	launch 폴더 미설치	CMakeLists.txt에 <code>install(DIRECTORY launch ...)</code> 추가 후 재빌드
노드가 안 뜸	실행 파일 이름 오타	<code>ros2 pkg executables <pkg></code> 로 확인
파라미터 미반영	YAML 최상위 키와 노드 이름 불일치	키를 런치의 name과 일치시킴
config를 못 찾음	config 폴더 미설치	<code>install(DIRECTORY launch config ...)</code>
토픽이 겹침	같은 노드를 이름만 바꿔 띄움	네임스페이스로 분리
인자가 안 먹음	LaunchConfiguration을 파이썬 if로 비교	IfCondition·UnlessCondition 사용

표 7-3. 런치에서 자주 만나는 오류

정리

* 런치는 여러 노드와 그 설정을 파일 하나에 적어 명령 한 줄로 실행하는 도구입니다. 실행 구성이 파일로 남아 재현되고 버전 관리도 됩니다.

* 파이썬 런치 파일은 `generate_launch_description` 함수가 `LaunchDescription`

에 Node 액션을 담아 반환하는 구조입니다.

- * 런치는 언어를 가리지 않습니다. 파이썬 노드와 C++ 노드를 한 파일에서 나란히 띄울 수 있습니다.

- * `name`, `remappings`, `parameters`, `namespace`로 같은 노드를 다른 이름, 다른 토픽, 다른 값으로 조립합니다. 그룹 전체에 네임스페이스를 씌울 때는 `GroupAction`과 `PushRosNamespace`를 씁니다.

- * `DeclareLaunchArgument`와 `LaunchConfiguration`으로 실행 시점에 값을 받고, `IfCondition`으로 조건에 따라 노드를 켜고 끕니다.

- * `IncludeLaunchDescription`으로 런치를 조합하면 큰 시스템도 기능별로 나눠 관리할 수 있습니다.

- * `launch`와 `config` 폴더는 CMake 설치 규칙에 함께 넣어야 실행 시 찾을 수 있습니다.

핵심 용어

- * **런치 파일**: 실행할 노드와 설정을 기술해 한 번에 실행하는 파일.
- * **LaunchDescription**: 실행할 액션(노드 등)을 담은 런치의 최상위 구성.
- * **리매핑**: 코드를 고치지 않고 노드가 쓰는 토픽 이름을 실행 시점에 바꾸는 것.
- * **네임스페이스**: 노드와 토픽 이름 앞에 접두어를 붙여 이름 충돌을 막는 구획.
- * **런치 인자**: 실행할 때 밖에서 값을 받아 런치에 전달하는 변수.
- * **bringup 패키지**: 실행 구성(런치·설정)을 모아 두는 패키지.

연습문제

1. 퍼블리셔와 서브스크라이버를 띄우되, 두 노드의 토픽을 `chatter`로 리매핑하는 런치를 작성하시오.
2. 런치 인자 `period`를 받아 발행 주기 파라미터에 연결하시오. `--show-args`로 인자가 보이는지 확인하시오.
3. `params.yaml`을 `config` 폴더에 두고 런치에서 불러오도록 CMake 설치 규칙을

추가하시오.

4. `GroupAction`과 `PushRosNamespace`로 로봇 세 대를 띄우고, `ros2 topic list`로 토픽이 셋으로 갈라졌는지 확인하시오.

5. (심화) 런치 인자 `use_cpp`를 받아, 참이면 `my_cpp_pkg`의 노드를, 거짓이면 `my_py_pkg`의 노드를 띄우는 런치를 작성하시오. `IfCondition`과 `UnlessCondition`을 사용합니다.

연습문제 해설

1. 두 Node 항목 모두에 `remappings=[('number', 'chatter')]`를 지정합니다. 양 쪽이 같은 이름으로 옮겨 가므로 연결이 유지됩니다.

2. `DeclareLaunchArgument('period', default_value='1.0')`을 선언하고, 노드의 `parameters=[{'publish_period': LaunchConfiguration('period')}]`로 연결합니다. `ros2 launch ... --show-args`에 인자가 나열되면 성공입니다.

3. `CMakeLists.txt`에 `install(DIRECTORY config DESTINATION share/${PROJECT_NAME})`을 추가하고, 런치에서 `get_package_share_directory`로 경로를 만들어 `parameters`에 전달합니다.

4. `GroupAction` 안에 `PushRosNamespace('robot1')`과 노드 실행을 묶고, `robot2`·`robot3`까지 반복합니다. `ros2 topic list`에 `/robot1/...` 형태로 접두어가 붙어 셋으로 갈라지면 성공입니다.

5. `DeclareLaunchArgument('use_cpp', default_value='false')`를 선언하고, 같은 역할의 노드를 두 번 적되 하나는 `condition=IfCondition(LaunchConfiguration('use_cpp'))`, 다른 하나는 `UnlessCondition`으로 감쌉니다. 조건에 맞는 쪽만 실행됩니다.

마무리 점검

- ☐ 런치가 해결하는 문제(터미널 여러 개의 한계)를 설명할 수 있습니다.
- ☐ 파이썬 런치 파일을 작성해 여러 노드를 한 번에 띄웠습니다.

- ☐ 리매핑으로 노드 코드를 고치지 않고 토픽 이름을 맞출 수 있습니다.
- ☐ 런치에서 파라미터와 실행 인자를 전달할 수 있습니다.
- ☐ IncludeLaunchDescription 으로 런치 파일을 조합할 수 있습니다.
- ☐ Foxy 기준 자료를 만나면 7.7절의 대응표로 옮길 수 있습니다.

2권 예고

– 여기까지가 1권입니다. 2권 ROS2 시뮬레이션 첫걸음에서는 URDF로 로봇을 모델링하고 Gazebo에서 시뮬레이션하며, 지금 익힌 다섯 가지 통신 위에 자율주행을 엮습니다.

부록. ros2 명령어 치트시트

본문에서 쓴 명령을 한곳에 모았습니다. 실습 중 막히면 여기부터 펴 보세요.
대괄호 자리는 실제 이름으로 바꿔 넣습니다.

환경·워크스페이스

- `source /opt/ros/jazzy/setup.bash` - ROS2 기본 환경 소싱(언더레이, 1장)
- `source ~/ros2_ws/install/setup.bash` - 내 워크스페이스 소싱(오버레이, 2장)
- `printenv | grep ROS` - ROS 환경 변수 확인

패키지·빌드

- `ros2 pkg create [이름] --build-type ament_python` - Python 패키지 생성
- `ros2 pkg create [이름] --build-type ament_cmake` - C++ 패키지 생성
- `colcon build` - 워크스페이스 전체 빌드
- `colcon build --packages-select [패키지]` - 지정 패키지만 빌드
- `colcon build --symlink-install` - Python 수정을 재빌드 없이 반영
- `ros2 pkg executables [패키지]` - 패키지의 실행 파일 목록

노드

- `ros2 run [패키지] [노드]` - 노드 실행
- `ros2 node list` - 실행 중인 노드 목록
- `ros2 node info [노드]` - 노드의 토픽·서비스·액션 연결 정보
- `rqt_graph` - 노드와 토픽의 연결 구조를 그림으로 확인

토픽 (3장)

- `ros2 topic list` - 활성 토픽 목록
- `ros2 topic type [토픽]` - 토픽의 메시지 타입
- `ros2 topic info [토픽]` - 발행자·구독자 수 확인
- `ros2 topic echo [토픽]` - 흐르는 메시지 출력
- `ros2 topic hz [토픽]` - 발행 주기 측정
- `ros2 topic pub [토픽] [타입] [값]` - 터미널에서 직접 발행

서비스 (4장)

- `ros2 service list` - 서비스 목록
- `ros2 service type [서비스]` - 서비스 타입
- `ros2 service call [서비스] [타입] [요청값]` - 터미널에서 호출

액션 (5장)

- `ros2 action list` - 액션 목록
- `ros2 action info [액션]` - 서버·클라이언트 확인
- `ros2 action send_goal [액션] [타입] [목표값]` - 목표 전송

- `ros2 action send_goal` [액션] [타입] [목표값] `--feedback` - 피드백까지 출력

파라미터 (6장)

- `ros2 param list` - 파라미터 목록
- `ros2 param get` [노드] [이름] - 값 읽기
- `ros2 param set` [노드] [이름] [값] - 런타임 변경
- `ros2 param dump` [노드] - 현재 값을 YAML로 출력
- `ros2 run ... --ros-args -p` [이름]:=[값] - 실행 시 값 주입
- `ros2 run ... --ros-args --params-file` [파일] - YAML 일괄 주입

인터페이스·런치 (6·7장)

- `ros2 interface list` - 사용 가능한 타입 목록
- `ros2 interface show` [타입] - 타입 정의(필드) 확인
- `ros2 launch` [패키지] [런치파일] - 런치 실행
- `ros2 launch` [패키지] [런치파일] [인자]:=[값] - 런치 인자 전달

참고문헌

여기에는 각 분야의 공식 문서와 표준, 더 깊이 공부하려는 사람에게 도움이 될 자료를 골라 실었습니다. 항목마다 본문 어느 장과 닿는지 한 줄씩 붙여 두었습니다. 확인 날짜가 붙은 URL은 그 시점 기준입니다.

공식 문서

- ROS 2 Documentation (Jazzy Jalisco). <https://docs.ros.org/en/jazzy/> (2026년 8월 확인) — 전 장의 기준 문서입니다. 개념(Concepts)과 튜토리얼(Tutorials)로 나뉘며, 이 책의 용어와 명령은 이 문서를 따릅니다.
- ROS 2 Tutorials — Beginner: CLI Tools·Client Libraries. <https://docs.ros.org/en/jazzy/Tutorials.html> (2026년 8월 확인) — 2~5장의 따라하기와 같은 흐름입니다. 책을 마친 뒤 공식 튜토리얼로 한 번 더 손을 익히면 좋습니다.
- About Quality of Service Settings. <https://docs.ros.org/en/jazzy/Concepts/Intermediate/About-Quality-of-Service-Settings.html> (2026년 8월 확인) — 3.5절에서 두 가지로 줄여 설명한 QoS의 전체 옵션이 표로 정리되어 있습니다.
- About Interfaces (msg·srv·action). <https://docs.ros.org/en/jazzy/Concepts/Basic/About-Interfaces.html> (2026년 8월 확인) — 인터페이스 정의 규칙의 원전으로, 6.5절 커스텀 인터페이스 작성 규칙이 여기에 근거합니다.
- Launch 튜토리얼. <https://docs.ros.org/en/jazzy/Tutorials/Intermediate/Launch/> (2026년 8월 확인) — 7장 런치 시스템의 공식 안내입니다. 이벤트 핸들러 등 이 책에서 다루지 않은 고급 기능도 있습니다.
- colcon documentation. <https://colcon.readthedocs.io/> (2026년 8월 확인) — 2장 빌드 시스템의 공식 문서입니다. 2.10절에서 추천 옵션의 전체 목록과 동작 원리를 볼 수 있습니다.
- rclpy API Reference. <https://docs.ros.org/en/jazzy/p/rclpy/> (2026년 8월 확인) — 3~6장 Python 코드에서 쓴 Node·Publisher·Client 클래스의 시그니처를 확인할 때 찾는 문서입니다.
- rclcpp API Reference. <https://docs.ros.org/en/jazzy/p/rclcpp/> (2026년 8월 확인) — 같은 내용의 C++ 대응입니다. 3.4절 Python·C++ 대응표와 함께 보면 위치를 잡기 쉽습니다.
- ROS 2 Design. <https://design.ros2.org> (2026년 8월 확인) — ROS2가 왜 DDS를 채택했는지 등 설계 배경 문서 모음입니다. 1.2~1.3절의 서술이 이 문서들과 닿아 있습니다.
- REP 2000 — ROS 2 Releases and Target Platforms. <https://www.ros.org/reps/rep-2000.html> (2026년 8월 확인) — 배포판별 지원 플랫폼과 지원 기간의 공식 표입니다. 1.4절에서 Jazzy를 고른 근거입니다.

도서

- 표윤석·임태훈. 『ROS 2로 시작하는 로봇 프로그래밍』. 루비페이퍼, 2021. — 한국어 대표 입문서입니다. 이 책의 범위를 넘어 SLAM·내비게이션까지 다루므로 2권 주제로 넘어가기 전에 훑어보기 좋습니다.
- Edouard Renard. 『ROS 2 from Scratch』. Packt, 2024. — 노드에서 런치까지 이 책과 같은 순서로 진행하는 영문 입문서입니다. 예제 패키지 구성이 본문 실습과 결이 가깝습니다.
- Francisco Martín Rico. 『A Concise Introduction to Robot Programming with ROS2』. CRC Press, 2022. — C++ 중심의 압축적인 입문서입니다. 3~5장의 C++ 구현을 더 파고들 때 다음 단계로 삼을 만합니다.
- Morgan Quigley·Brian Gerkey·William D. Smart. 『Programming Robots with ROS』. O'Reilly, 2015. — ROS1 기준이라 명령은 다르지만, 토픽·서비스라는 통신 모델이 어디서 왔는지 뿌리를 보여 줍니다. 읽을 때 배포판 차이를 염두에 두어야 합니다.

소스 코드·패키지

- ros2/examples. <https://github.com/ros2/examples> (2026년 8월 확인) — rclpy·rclcpp 공식 최소 예제 모음입니다. 3~5장에서 만든 노드의 공식 대응을 찾아 비교해 볼 수 있습니다.
- ros2/demos. <https://github.com/ros2/demos> (2026년 8월 확인) — demo_nodes_py·demo_nodes_cpp가 여기 있습니다. 1.9절 첫 통신에서 실행한 talker·listener의 원본 코드입니다.
- ros2/example_interfaces. https://github.com/ros2/example_interfaces (2026년 8월 확인) — 4장 AddTwoInts, 5장 Fibonacci 등 이 책이 쓴 표준 연습용 인터페이스의 정의입니다.
- OMG Data Distribution Service (DDS) Specification. <https://www.omg.org/spec/DDS/> (2026년 8월 확인) — ROS2 통신 미들웨어의 국제 표준 원전입니다. 1.3절의 배경이 궁금할 때 찾아봅니다.

커뮤니티

- Robotics Stack Exchange. <https://robotics.stackexchange.com> (2026년 8월 확인) — ROS 공식 질문·답변 사이트입니다. 2.8절·6.8절 트러블슈팅으로 풀리지 않는 오류는 증상 메시지 그대로 여기서 검색합니다.
- ROS Discourse. <https://discourse.ros.org> (2026년 8월 확인) — 배포판 출시 공지와 로드맵이 올라오는 공식 포럼입니다. 다음 배포판 소식을 접하는 창구입니다.

자료를 고를 때

ROS2는 배포판에 따라 API와 명령이 조금씩 다릅니다. 자료를 펴기 전에 어느 배포판 기준인지부터 확인하는 습관을 들입니다. 이 책은 Jazzy(Ubuntu 24.04) 기준이고, 공식 문서는 URL의 en/jazzy 부분으로 판을 구분합니다. Foxy 기준

자료를 만났다면 7.7절의 대응표가 번역기 역할을 합니다.

검색 결과보다 공식 문서를 먼저 봅니다. 오래된 블로그의 코드는 옛 배포판 기준이라 그대로 돌지 않는 일이 잦고, 그때 잃는 시간이 문서를 읽는 시간보다 깁니다. 막히면 공식 문서 → 이 책의 트러블슈팅 절 → 커뮤니티 검색 순서를 권합니다.